



ספר פרויקט לעבודת גמר

י"ד הנדסאי תוכנה (שאלון 714918)
הצפנת מסרים בקבצים (סטגנוגרפיה)

Stegno Mail



שם הסטודנט: יהונתן אליעזר עובדיה

ת"ז: 215556994

מנחה: אילן פרץ

תאריך הגשה: י"א סיוון תשפ"ו 26/05/2026

שם המכללה: כנפי רוח קרית נוער ירושלים (סמל מוסד: 140129)

תוכן עניינים

3	1. הצעת הפרויקט שאושרה ע"י משרד החינוך
4	2. מבוא
4	2.1 הרקע לפרויקט
4	2.2 תהליך המחקר
4	2.3 רקע תיאורטי וסקירת ספרות מקצועית
6	2.4 אתגרים מרכזיים במחקר
8	3. מטרות ויעדים לפיתוח המערכת
9	4. אתגרים בבניית המערכת
10	5. הגדרת מדדי הצלחה למערכת
11	6. תיאור פתרונות קיימים לבעיה וניתוח חלופות
12	7. תיאור החלופה הנבחרת ונימוקים לבחירתה
13	8. אפיון המערכת המוצעת
13	8.1 ניתוח הדרישות מהמערכת
14	8.3 אפיון פונקציונלי וביצועים עיקריים
15	8.4 אילוצי המערכת
18	9. תיאור הארכיטקטורה של המערכת
18	9.1 ארכיטקטורת המערכת
20	9.2 מודל שרת לקוח
20	9.3 תהליכים של מע' ההפעלה
21	9.4 תיאור פרוטוקולי תקשורת
21	9.5 שירותים חיצוניים
22	10. ניתוח תרחישים וזרימת המידע במערכת המוצעת
22	10.1 תרשימי תרחיש Use case Diagram
23	10.2 תרשימי רצף Sequence Diagram
30	11. תיאור מסכים וממשק משתמש
30	11.1 תרשים היררכיית המסכים
31	11.2 תיאור המסכים
46	12. תיאור התוכנה
46	12.1 סביבת הפיתוח של התוכנה
48	12.2 תיאור מבנה הפרויקט והמחלקות
82	12.3 מבני נתונים בהם נעשה שימוש
85	12.4 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים
93	13. תיאור מסד הנתונים
95	14. מדריך למשתמש
96	15. בדיקות והערכה
98	16. מסקנות
99	17. פיתוחים עתידיים
100	18. ביבליוגרפיה

1. הצעת הפרויקט שאושרה ע"י משרד החינוך

תיאור הנושא הפרויקט

פרויקט זה עוסק בפיתוח מערכת הפועלת כאתר אינטרנט, אשר ייעודה הוא הסתרת הודעות טקסטואליות בתוך קבצי מדיה שגרתיים, כגון תמונות או קובצי שמע. הפעולה הסטגוגרפית שמתבצעת במערכת גורמת לקבצים אלו להיראות רגילים לחלוטין כלפי כל מתבונן או מאזין חיצוני, ובכך מבטיחה כי המסר המוטמע יישמר בסוד מוחלט.

מעבר ליכולות ההצפנה וההסתרה, המערכת מספקת מעטפת שלמה לניהול תקשורת חשאית:

- **העברת נתונים:** המערכת מאפשרת להעביר את הקבצים המוצפנים בין המשתמשים בצורה מאובטחת דרך תיבת דואר פנימית וסגורה.
- **מעקב ובקרה:** המערכת שומרת רישום מסודר של כלל הפעולות שבוצעו ומציגה אותן ביומן היסטוריה אישי.
- **נגישות:** כלל הפעולות מתבצעות ישירות דרך רשת האינטרנט (דפדפן), ללא כל צורך בהתקנת תוכנה ייעודית על מחשב הקצה.

הגדרת הבעיה האלגוריתמית

האתגר המרכזי והמורכב ביותר שעומד בבסיס הפרויקט הוא היכולת להטמיע מסר סמוי בתוך קובץ כיסוי קיים, תוך עמידה קפדנית בשני תנאים הנדסיים מנוגדים:

1. **שמירה על איכות הקובץ (חמקנות):** ביצוע שינויים מזעריים בלבד, כך שהעין או האוזן האנושית לא ירגישו כלל בשינוי שנעשה במדיה המקורית.
2. **הבטחת עמידות (Robustness):** יצירת חסיונות גבוהה מפני ניסיונות של גורמים עוינים לגלות את המסר המוסתר או למחוק אותו מתוך הקובץ.

כדי לפתור בעיה מורכבת זו בצורה האופטימלית, המערכת אינה מסתמכת על שיטת הצפנה אחת ויחידה. במקום זאת, מיושמת גישה "היברידית", המתאימה פתרון אלגוריתמי שונה וייעודי לכל סוג ופורמט של קובץ, במטרה להשיג את התוצאה הטובה והבטוחה ביותר לכל תרחיש.

2. מבוא

2.1 הרקע לפרויקט

תחום הסטגנוגרפיה (Steganography) הוא ענף מרכזי בעולם אבטחת המידע העוסק בטכנולוגיות להסתרת עצם קיומו של מידע. להבדיל מהצפנה קלאסית (Cryptography), אשר מערבלת את תוכן המידע אך מותירה אותו גלוי לעין כרצף תווים חסרי משמעות, מטרת הסטגנוגרפיה היא להטמיע מסרים בתוך מדיה דיגיטלית (כגון תמונות, קבצי שמע או וידאו) באופן שאינו מותיר עקבות גלויים, כך שמתבונן מהצד לא יחשוד כלל שמועבר מסר סמוי בתוך הקובץ. בעולם אבטחת המידע המודרני, קיים צורך מבצעי בקיומם של ערוצי תקשורת חשאים לחלוטין עבור אוכלוסיות הפועלות בסביבה עוינת. האתגר הטכנולוגי המרכזי בתחום זה הוא היכולת להחדיר מידע זר לתוך קובץ נתונים קיים בנפח משמעותי, מבלי לפגוע באיכותו הוויזואלית או השמיעתית של הקובץ, ומבלי לשנות את המאפיינים הסטטיסטיים שלו בצורה שתתגלה על ידי כלי ניטור ובקרה ברשת.

2.2 תהליך המחקר

במסגרת תהליך המחקר לבניית המערכת, נבחנה השפעתם של אלגוריתמים שונים על קבצי מדיה מגוונים. ליבת הפתרון ההנדסי דרשה מחקר מעמיק של מנועי עיבוד אותות ותמונה, מתוך הבנה שפורמטים שונים דורשים גישות התמודדות שונות. המערכת פותחה כך שתשתמש ברכיבי תוכנה סטנדרטיים לקריאת המבנה הבסיסי של הקבצים (כגון חילוץ פיקסלים או מקדמי תדר), ומעליהם יושמה הלוגיקה הסטגנוגרפית הייחודית לכל פורמט.

2.3 רקע תיאורטי וסקירת ספרות מקצועית

שיטות קיימות להטמעת מידע בקבצי תמונה ושמע:

LSB – שינוי הביט הפחות משמעותי

שיטה זו מבצעת החלפה של הביט האחרון בכל פיקסל בתמונה או בדגימת אודיו. היתרון המרכזי שלה הוא פשטות מימוש וקיבולת גבוהה: ניתן להטמיע כמות גדולה של מידע בקבצים גדולים יחסית, והעין האנושית בדרך כלל אינה מבחינה בשינויים זעירים בביט הפחות משמעותי.

עם זאת, החיסרון הגדול הוא פגיעות גבוהה: כל שינוי בסיסי בקובץ — דחיסה, המרת פורמט, שינוי בהירות, רעש, סיבוב או שמירה מחדש — עלול למחוק את המידע או לחשוף אותו. בנוסף, השיטה יוצרת תבניות סטטיסטיות שקל לזהות באמצעות כלים מודרניים. לכן, LSB אינה נחשבת לשיטה בטוחה בפרויקטים רציניים.

Histogram Shifting – הזזת התפלגות הפיקסלים

שיטה זו מבוססת על שינוי מינימלי של ההיסטוגרמה, כלומר התפלגות ערכי הפיקסלים בתמונה. האלגוריתם משתמש בכך כדי לפנות מקום למידע חדש מבלי ליצור עיוותים ברורים לעין. היתרון הוא חמקנות יחסית גבוהה: בדרך כלל השינויים קטנים מאוד והעין האנושית כמעט לא מבחינה בהם.

החיסרון הוא שהיכולת להטמיע מידע מוגבלת יחסית לשיטות אחרות. כל שינוי בעיבוד התמונה — כמו פילטר החלקה, שינוי קונטרסט או דחיסה — עלול לפגוע במסר המוטמע. לכן השיטה פחות מתאימה לתמונות שעשויות לעבור עריכה נוספת.

JSteg – החבאה במקדמי DCT של JPEG

JSteg פועל ישירות על מקדמי ה-DCT (הייצוג הפנימי של JPEG). היתרון המרכזי שלה היה אפשרות להחביא מידע מבלי לשנות את ערכי הפיקסלים עצמם. בשנות ה-90 ותחילת שנות ה-2000, היא הייתה אחת השיטות הפופולריות להטמעה בקבצי JPEG.

החולשה העיקרית היא יצירת חתימה סטטיסטית ברורה: הביטים משתנים במקדם DCT בצורה ניתנת לזיהוי. כיום, JSteg נחשבת מיושנת ומסוכנת לשימוש במקרים בהם נדרשת חמקנות או עמידות.

F3 ו-F4 – גרסאות מוקדמות של החבאה ב-JPEG

גרסאות אלה שיפרו מעט את JSteg כדי להפחית את הסימנים הסטטיסטיים. האלגוריתמים טיפלו בערכים חיוביים ושליליים בצורה שונה והקטינו במעט את הסיכון לגילוי המסר.

למרות זאת, התוצאות אינן מספקות: שני האלגוריתמים עדיין משאירים תבניות שקל לזהות. F3 נוטה לאבד מידע כאשר מקדם הפוך לאפס, ו-F4 מפחיתה את התבניות במעט בלבד. לכן הן אינן עמידות מספיק לשימוש בפרויקטים אמיתיים.

F5 – Matrix Encoding (גרסה מתקדמת של JSteg)

F5 היא אחת השיטות החזקות ביותר להטמעה ב-JPEG. השיטה משתמשת בקידוד מטריצוני שמאפשר להטמיע מידע רב תוך שינוי מינימלי של מקדמי DCT. האלגוריתם משנה מספר מועט של מקדמים במקום כל מקדם שבו יש מידע, מה שמקטין את החתימה הסטטיסטית ומעלה את החמקנות.

יתרון נוסף הוא שימוש בבחירה פסבדו-אקראית של המקדמים שבהם יוטמע המידע, מה שמקשה על גילוי סטטיסטי. השיטה ידועה כסטנדרטית בפרויקטים מחקריים ומאפשרת איזון טוב בין קיבולת, חמקנות ועמידות לדחיסה חוזרת.

PVD – Pixel Value Differencing (הבדלי פיקסלים)

PVD בוחנת זוגות פיקסלים סמוכים ומזהה את רמת השונות ביניהם. באזורים מורכבים כמו שיער, דשא, בניינים וטקסטורות – ניתן להטמיע הרבה מידע מבלי שאפשר להבחין בשינוי. באזורים חלקים כמו שמיים או קירות נקיים, האלגוריתם מטמיע מעט מידע בלבד כדי למנוע כתמים.

השיטה מנצלת את העובדה שהעין האנושית רגישה פחות לשינויים באזורים עמוסים בפרטים, ולכן מספקת חמקנות טובה מאוד. היא נחשבת לבטוחה יותר מ-LSB ומאפשרת קיבולת טובה עם איכות תמונה גבוהה.

Phase Coding – קידוד פאזה בקבצי שמע

Phase Coding מטמיע מידע בפאזה של גל הקול במקום לשנות עוצמה או תדר. היתרון הוא חמקנות טובה: השינוי כמעט בלתי מורגש לאוזן האנושית.

החיסרון הוא שכל עיבוד על הקובץ כמו פילטר, נורמליזציה או שינוי עוצמה מוחק את המידע. לכן שיטה זו אינה עמידה מספיק לשימוש בקבצי שמע שעוברים עריכה או המרה.

Echo Hiding – הוספת הד עדין

שיטה זו מוסיפה הד חלש מאוד בתדרים מסוימים, שהאוזן שומעת כחלק מהאקוסטיקה. היתרון הוא חמקנות מסוימת והטמעה פשוטה.

החיסרון הוא שהד יוצר חתימה שקל לגלות באמצעות ניתוח ספקטרלי, וכל שינוי במבנה הקובץ, דחיסה או פילטרים, מוחק את המידע לחלוטין.

DSSS – Spread Spectrum (שיטה צבאית)

DSSS מפזר את המסר על פני ספקטרום רחב, כך שהמסר נשמע כמו רעש לבן טבעי. השיטה עמידה מאוד: כמעט כל שינוי בקובץ דחיסה, פילטרים, שינוי עוצמה או המרת פורמט אינו משפיע על המידע.

השיטה דורשת רצף פיזור שמוכר רק למי שמנסה לשחזר את המסר. DSSS משמשת במערכות צבאיות, לוויינים ותקשורת מוצפנת, ומספקת חמקנות ועמידות גבוהות יותר משאר השיטות.

סקירת ספרות

כדי לממש את האלגוריתמים בפרויקט, בוצע תהליך מחקר שהתבסס על ספרות מקצועית ומפרטים טכניים הקשורים לאלגוריתמים שבחרתי.

להלן הנושאים המרכזיים שנלמדו

1. אלגוריתם PVD (Pixel Value Differencing) ומרחב הפיקסלים

נחקר אלגוריתם PVD המנצל אזורי ניגודיות בתמונה להטמעת מידע רב מבלי לפגוע באיכותה הוויזואלית. הלמידה התמקדה בבניית טבלאות טווחים אקספוננציאליות ובביצוע פעולות בינאריות (Bitwise) על ערוצי צבע. המחקר התבסס על המאמר המקורי של החוקרים Tsai ו- Wu ועל התייעוד הרשמי של Java לעיבוד תמונה [1].

2. ארכיטקטורת JPEG, מקדמי DCT ואלגוריתם F5

נלמד לעומק תהליך הכיווץ המורכב של קובץ JPEG, הכולל חלוקה לבלוקים, התמרת DCT, קוונטיזציה וקידוד אך התמקד בהטמעה עצמה, המחקר התבסס על ההבנה שכל שינוי, עריכה או שמירה מחדש של הקובץ לאחר הטמעה רגילה יגררו בעיות קשות וימחקו את המידע לחלוטין בשל אופי הכיווץ מאבד הנתונים. בעקבות זאת, הלמידה התמקדה בבניית מנגנון כתיבה עצמאי שמבצע את שלבי הכיווץ הללו במדויק ומחדיר את המידע ישירות למקדמי התדר, על בסיס עבודתו של Andreas Westfel.

3. אלגוריתם DSSS (Spread Spectrum) ועיבוד אותות שמע

נחקר אלגוריתם DSSS אשר מפזר את אות המידע מתחת לרצפת הרעש הטבעית של קובץ השמע. הלמידה כללה יצירת רצפים פסאודו-אקראיים (PN) ופענוח מתמטי של דגימות קול גולמיות (PCM) מתוך קבצי WAV. המחקר נשען על ספרות מקצועית בעיבוד אותות דיגיטליים (DSP) ועל התייעוד של ספריית Java Sound API.

2.4 אתגרים מרכזיים במחקר

שלב המחקר המקדים דרש הבנה תיאורטית של שיטות ההצפנה והסתרת המידע השונות, והציב מספר אתגרים מרכזיים שאותם היה צורך לפתור לפני תחילת הפיתוח עצמו:

1. מציאת האיזון הנכון (משולש הסטגנוגרפיה) האתגר המחקרי הראשון היה להבין את הקשר המורכב בין שלושה גורמים מנוגדים:

- **קיבולת (Capacity):** כמות המידע הסודי שאפשר להחביא בתוך הקובץ
- **חמקנות (Imperceptibility):** היכולת להסתיר את המידע כך שהקובץ ייראה או יישמע טבעי לחלוטין, מבלי לעורר חשד.
- **עמידות (Robustness):** היכולת של המסר לשרוד שינויים בקובץ (כמו דחיסה או שליחה ברשת). האתגר היה לחקור ולמצוא את שיטות העבודה שיאפשרו להחביא מסר גדול מספיק, מבלי לפגוע באיכות הקובץ או להרוס את המסר במידה והקובץ יעבור דחיסה מחדש.

התמודדות עם כלי גילוי וניתוח סטטיסטי

כיום, מערכות אבטחה לא מסתמכות רק על העין האנושית, אלא משתמשות בכלים שמנתחים את הקובץ מבחינה סטטיסטית ומחפשים בו חריגות. האתגר במחקר היה להבין את החולשות של שיטות פשוטות (כמו שיטת LSB שמשאירה תבניות קבועות שקל מאוד לגלות, ולחקור אלגוריתמים חכמים יותר) כמו PVD ו-F5. המטרה הייתה להבין כיצד ניתן לפזר את המידע הסודי בתוך ה"רעש" הטבעי של הקובץ (למשל, באזורי צל או בטקסטורות מורכבות בתמונה), כך שההתפלגות הסטטיסטית שלו תיראה טבעית לחלוטין.

הבדלים בין סוגי הקבצים (תמונות רגילות לעומת תמונות דחוסות)

אתגר תיאורטי נוסף היה להבין את ההבדל המבני בין קבצי PNG לקבצי JPEG. בעוד שב-PNG-הנתונים נשמרים כפיקסלים רגילים ואפשר לשנות אותם ישירות, ב-JPEG-המידע עובר דחיסה מתמטית מורכבת שמורכבת מ"מקדמי תדר" (DCT). המחקר דרש להבין לעומק כיצד פועל מנגנון הדחיסה הזה, במטרה לגלות איך אפשר להחדיר את המידע פנימה בדיוק בשלב הנכון של הדחיסה, כך שהמסר לא ייהרס כשהתמונה נשמרת בסוף התהליך.

התמודדות עם קבצי שמע ורגישות השמיעה

בכל הנוגע לקבצי קול (WAV), האתגר המרכזי היה הרגישות הגבוהה של האוזן האנושית לכל שינוי קטן בתדרים או בעוצמת הקול. חקירת שיטות קיימות כמו הוספת הד קל (Echo) או שינוי פאזה הראתה שהן רגישות מדי, והמסר שבהן נמחק בקלות בכל פעם שהקובץ עובר עריכה קטנה. האתגר במחקר היה להבין כיצד ליישם שיטה מתקדמת מהעולם הצבאי (DSSS) ספקטרום פרוש, (שלוקחת את המסר, מערבבת אותו עם רעש רקע אקראי, ומפזרת אותו על פני הקובץ בעוצמה חלשה מאוד. בדרך זו, המידע המוצפן נשמע כמו רעש רקע טבעי לחלוטין שהאוזן מתעלמת ממנו.

2.4.1 התמודדות עם הבעיה

ההתמודדות עם האתגרים התיאורטיים והמדעיים שהוצגו לעיל דרשה גיבוש של אסטרטגיה אלגוריתמית ומבנית מיועדת, אשר שימשה בסיס לכל שלבי הפיתוח בהמשך:

ההתמודדות עם משולש האילוצים הסטגוגרפי: כדי לפתור את המתח בין קיבולת, חמקנות ועמידות, נדחתה הגישה של שימוש באלגוריתם אחיד לכל סוגי המדיה. במקום זאת, נבחרה גישה היברידית המפצלת את המערכת לשלושה מנועים נפרדים. כל מנוע הותאם באופן ספציפי למאפיינים של פורמט הקובץ שלו, ובכך התאפשר ניצול מקסימלי של נפח ההחבאה מבלי לפגוע בעמידות או בחמקנות של הקובץ.

השגת חסינות סטטיסטית: בשביל למנוע מכלי ניטור לזהות תבניות קבועות וחריגות, שולבו באלגוריתמים מנגנונים של פיזור פסאודו-אקראי ומניפולציות אדפטיביות. באלגוריתם, PVD המערכת מנתחת את רמת השונות בין זוגות פיקסלים ומחדירה מידע רב רק באזורים עמוסי פרטים וטקסטורות (שבהם העין והכלים הסטטיסטיים רגישים פחות לשינויים), בעוד שבאזורים חלקים השינוי הוא אפסי.

פתרון בעיית התדר והדחיסה ב-JPEG: ההתמודדות עם המבנה המתמטי של קבצי JPEG ושלב הדחיסה מאבדת הנתונים הטבעיים הובילה את הצורך בעבודה במישור התדר (Transform Domain). הוחלט להשתמש באלגוריתם F5 המבצע קידוד מטריציוני (Matrix Encoding) המבוסס על קוד המינג. שיטה זו מאפשרת לשנות מקדם תדר (DCT) בודד בלבד מתוך קבוצה של מקדמים כדי להצפין מספר ביטים של מסר, מה שמקטין משמעותית את השינוי הכולל בקובץ ומבטיח שהמסר ישרוד את הדחיסה.

התגברות על רגישות השמיעה באודיו: על מנת להחביא מידע בקבצי שמע מבלי שהאוזן תבחין ברעשי רקע מלאכותיים, נבחר אלגוריתם ספקטרום פרוש (DSSS) ההתמודדות התבססה על שימוש במחולל רעש פסאודו-אקראי המרחיב את אות המסר הצר על פני כל ספקטרום התדרים של השיר, תוך הכפלתו בפקטור הנחתה קטן במיוחד. עוצמת המידע הונחתה אל מתחת לרצפת הרעש הטבעית של הקובץ, כך שהיא מתמזגת איתו לחלוטין ונשמעת כחלק בלתי נפרד מהאקוסטיקה של ההקלטה.

2.4.2 הסיבות והמוטיבציה לבחירת הנושא

המוטיבציה המרכזית לבחירת הפרויקט נבעה מהרצון להתמודד עם אתגר הנדסי מורכב, החורג מגבולות פיתוח התוכנה הסטנדרטי. עולם הסטגנוגרפיה ואבטחת המידע משך אותי מכיוון שהוא דורש ירידה לפרטים הקטנים של עולם התוכנה: הבנה עמוקה של ייצוג נתונים בינארי, שליטה בפעולות ברמת הביטים (Bitwise Operations) והיכרות עם המבנה הפנימי של קבצים ברמת ה Low-Level.

במקום להסתמך רק על ספריות מוכנות שיבצעו את כל תהליך ההצפנה כ"קופסה שחורה", הפרויקט היווה עבורי הזדמנות ליישם לוגיקה ברמת הקוד – החל מפירוק והרכבה של דגימות קול גולמיות, דרך חישובי הפרשים בערוצי צבע של פיקסלים, ועד לאינטגרציה נכונה מול מודולים מתמטיים חיצוניים. מעבר לעניין שביצירת תשתית חשאית להעברת מסרים, הפרויקט נבחר כדי לדחוף אותי ללמידה עצמאית של ניהול זיכרון יעיל בשרת והרחבת היכולות המקצועיות שלי כמפתח תוכנה לקראת השתלבות בתעשייה.

2.4.3 על איזה צורך הפרויקט עונה ?

הפרויקט נותן מענה לצורך של אנשים, ארגונים או גורמי ביטחון לתקשר באופן מאובטח וחשאי בסביבות בהן תעבורת רשת מוצפנת עלולה להיחסם או לסמן את המשתמשים כיעדי מעקב. המערכת מספקת פלטפורמה נגישה להעברת נתונים רגישים תחת מעטה של שגרת רשת רגילה לחלוטין (העברת תמונות וקבצי שמע).

3. מטרות ויעדים לפיתוח המערכת

פיתוח המערכת (Stegano Mail) הציב בפניי מספר מטרות ויעדים הנדסיים מוגדרים:

1. פיתוח פלטפורמת Web אוטונומית: בניית מערכת שרת-לקוח מלאה המאפשרת למשתמשים להצפין מסרים ולנהל תעבורת נתונים חשאית ישירות מהדפדפן, ללא תלות בתוכנות צד-שלישי או התקנות מקומיות.
2. הטמעה וחילוץ (End-to-End) מדויק: יצירת מנגנונים אלגוריתמיים יציבים שיוודעים לקחת מסר, לתרגם אותו למערכי ביטים (Bit Array), לפזר אותם בקבצי מדיה מגוונים (PNG, JPG, WAV) ולשחזר אותם שלמים לחלוטין בצד המקבל.
3. אוטומציה מלאה בניהול המידע: פיתוח מנגנון מובנה המטפל באורך המסר, (Dynamic Header) כך שהמערכת תנהל בעצמה את שלב הפענוח מבלי לדרוש מהמשתמש להזין נתונים טכניים או פרמטרים.
4. תאימות לפורמט דחוס: פיתוח רכיב תוכנה המסוגל לעבוד ישירות מול קבצי JPEG מבלי לפגוע במבנה הסטנדרטי שלהם (לרבות כתיבת מחלקה המטפלת עצמאית בדחיסת התמונה וניהול מקדמי התדר).

4. אתגרים בבניית המערכת

1. ניהול גודל המסר ואוטומציית פענוח (Header Mechanism)

אחד האתגרים בבניית הלוגיקה של שלב החילוץ, היה כיצד התוכנה תדע היכן לעצור את קריאת הביטים מתוך הקובץ, מבלי לשאוב את הנתונים המקוריים שלו כ"ג'בריש". הדרישה הייתה למנוע ממשתמש הקצה להזין ידנית את כמות התווים שהוא מצפה לקבל. כדי לפתור זאת, מימשי את מחלקות העזר (כגון F5Utils ו-DsssUtils) כך שייצרו "Header" מובנה. המערכת לוקחת את האורך המספרי של המסר וממירה אותו למערך בינארי של 32 ביטים (ב JPEG או 16 ביטים (באודיו)), ומשרשרת אותו בתחילת מערך הביטים של ההודעה (באמצעות פונקציית concatArrays). בשלב הפענוח, האלגוריתם שואב תחילה רק את הHeader, מתרגם אותו חזרה למספר שלם, ומשתמש בערך זה כדי לדעת בדיוק כמה מחזורי לולאה לבצע עבור ההודעה האמיתית.

2. טיפול באלגוריתם F5 והיעדר ספריות מתאימות ב Java

האתגר המורכב ביותר בפיתוח התוכנה היה המימוש של אלגוריתם F5 לקבצי JPEG. האלגוריתם דורש לשנות את הביט הפחות משמעותי (LSB) של מקדמי תדר מקוונטים (Quantized DCT Coefficients). במערכת ההפעלה ובשפת Java, רוב הספריות כמו ImageIO שומרות תמונת JPEG כאובייקט שלם, ולא מאפשרות גישה פנימית לחלקי הדחיסה כדי לשנות ערכים תוך כדי. הפתרון ההנדסי חייב כתיבה של ספרייה עצמאית מאפס המיוצגת על ידי מחלקות כגון F5JpegWriter ו-F5JpegReader. הספרייה הזו מחלצת את הבלוקים של התמונה, ומנהלת בעצמה את תהליך הכתיבה הבינארית של הקובץ מחדש. הספרייה שיצרתי כוללת את לוגיקת Entropy Encoding כגון אלגוריתם Huffman ו-Run Length Encoding וניהול טבלאות תדרים (DQT), כך שביטי המסר שהוטמעו ישמרו בסוף הדחיסה.

3. עבודה עם זרם בינארי ומניפולציות ברמת Bitwise

אתגר נוסף היה השמירה על יעילות הזיכרון (Memory Management) וביצוע פעולות מתמטיות על קבצים גדולים. במקום להעתיק מחרוזות ארוכות שצורכות משאבים, הפיתוח התבסס על מניפולציות ברמת ה-Low-Level. לדוגמה, במחלקת PvdUtils, חילוץ ועדכון ערוצי צבע מתוך פיקסל (ARGB) בוצע באמצעות פעולות Shift בינאריות <> ו << >> ושימוש במסכות (& 0xFF). כמו כן, בפיתוח של מחלקת האודיו DsssUtils, המערכת התמודדה עם פירוק והרכבה של דגימות קול בתצורת Little Endian המרת שני Bytes לדגימת Short אחת ובחזרה באמצעות פעולות בינאריות, כדי לשמור על דיוק מקסימלי ללא עיוות האות.

5. הגדרת מדדי הצלחה למערכת

על מנת למדוד באופן אובייקטיבי את ביצועי המערכת (Stegano Mail) ולבחון האם היא עומדת במטרות ההנדסיות שהוצגו בפרק 3, נגדיר כעת את מדדי ההצלחה המרכזיים. מדדים אלו מהווים את קריטריוני הקבלה של הפרויקט, ועל פיהם ייבחנו תוצרי התוכנה בפרק הבדיקות (פרק 15):

1. שלמות הנתונים (100% Data Integrity)

המדד הקריטי ביותר להצלחת הפרויקט הוא היכולת לשחזר את המחרוזת המקורית בשלמותה המוחלטת (Lossless Extraction). הצלחה תוגדר כחילוץ הטקסט המוסתר מתוך קובץ המדיה בדיוק כפי שהוזן, ללא אובדן תווים, ללא רעש נלווה ("ג'יבריש" בקצה המחרוזת), ותוך תמיכה מלאה ומוכחת בקידוד UTF-8 הכרחי לתמיכה תקינה בשפה העברית ובתווים מיוחדים).

2. שימור פונקציונלי של קובץ הכיסוי (Stegno-Object Validity)

הקובץ הנגוע המופק מהמערכת לאחר ההטמעה חייב לשמור על תקינותו המבנית והפונקציונלית לחלוטין:

תמונות (PNG/JPEG): התמונה הנגועה חייבת להיפתח בהצלחה בכל תוכנת צפייה סטנדרטית (ללא שגיאות Corrupted File). בנוסף, אסור שהעין האנושית תבחין בעיוותי צבע או פיקסול כתוצאה ממניפולציית הפרשים (PVD) או משינוי מקדמי ה-DCT (באלגוריתם F5).

שמע (WAV): קובץ השמע חייב להתנגן כהלכה בכל נגן מדיה, מבלי שהאוזן האנושית תשמע צפצופים או רעשים סטטיים חריגים (הצלחה זו תיוחס להנחתת האות באלגוריתם DSSS אל מתחת לרצפת הרעש הטבעית).

3. אוטומציית חילוץ שקופה (Header Functionality):

מדד הצלחה פונקציונלי הוא יכולתה של המערכת לייתר לחלוטין את הצורך של משתמש הקצה לדעת פרטים טכניים על המסר שקיבל. הצלחה תימדד בפעולה חלקה של מנגנון ה-Header-המפענח בשרת יקרא את 16 או 32 הביטים הראשונים בקובץ, יתרגם אותם לערך מספרי (אורך המסר בבתים), ויעצור את שאיבת הביטים מתוך הפיקסלים או דגימות הקול בדיוק בביט האחרון של המסר, ללא כל קלט חיצוני.

4. יציבות השרת ועמידה באילוצי זיכרון (Backend Stability):

עיבוד תמונות וגלי קול ברמת ה-Low Level-עלול להעמיס משמעותית על זיכרון השרת (RAM). הצלחה בסעיף זה תימדד ביכולת המערכת (Spring Boot) לטפל בבקשות, לפרק ולהרכיב קבצים במהירות סבירה מבלי לקרוס (מניעת שגיאות OutOfMemoryError). המדד יוכח על ידי אכיפה נכונה של חסימות שהוגדרו בקוד, כגון מניעת העלאת תמונות שמידותיהן עולות על הגבול המותר (למשל 2000 X 2000 פיקסלים), ופסילת קבצי אודיו שאינם מקודדים ב-16-bit PCM-תוך החזרת הודעת שגיאה מסודרת למשתמש ולא קריסה כללית של המערכת.

6. תיאור פתרונות קיימים לבעיה וניתוח חלופות

בבואנו לפתור את הבעיה המערכתית של העברת מסרים חשאית בסביבה עוינת או מנוטרת, נבחנו מספר חלופות וגישות הנדסיות הקיימות כיום בשוק:

חלופה 1: שימוש בהצפנה קריפטוגרפית סטנדרטית (לדוגמה: AES או PGP)

הפתרון הנפוץ ביותר כיום לאבטחת מידע הוא שימוש בתוכנות הצפנה סטנדרטיות המערבלות את הטקסט, ושליחתו כקובץ או כהודעה מוצפנת דרך ערוצי תקשורת מקובלים (דוא"ל, אפליקציות מסרים).

- יתרונות: קל יחסית ליישום, נגיש לקהל הרחב, ומספק אבטחה מתמטית חזקה מאוד נגד קריאת התוכן על ידי צד שלישי.
- חסרונות: חלופה זו אינה עונה על דרישת "החמקנות". עצם העברת מידע מוצפן (הנראה כרצף תווים חסר פשר) ברשת גלויה, מעורר מיד את החשד של גורמים מנטרים. השרתים מזהים שמדובר בתעבורה מוצפנת, מה שעלול להוביל לחסימת התקשורת או לסימון המשתמש כיעד למעקב, גם אם התוכן לא פוענח.

חלופה 2: שימוש בתוכנות סטגנוגרפיה שולחניות קיימות (כמו Steghide או OpenStego)

קיימות בשוק תוכנות קוד-פתוח המאפשרות הטמעת מידע בתוך קבצי מדיה. תוכנות אלו מבוססות ברובן על התקנה מקומית וממשק שורת פקודה (CLI) או ממשק משתמש בסיסי.

- יתרונות: מדובר במוצרי מדף מוכנים החוסכים זמן פיתוח, משתמשים באלגוריתמים אמינים ועובדים בצורה מקומית (Offline) ללא תלות בחיבור לשרת.
- חסרונות: תוכנות אלו דורשות התקנה מקומית, מה שמשאיר "טביעת אצבע" דיגיטלית על מחשב המשתמש ומסכן אותו במידה והמחשב נתפס לבדיקה. בנוסף, הן אינן מספקות פלטפורמה לשליחה וקבלה של הקבצים; המשתמש נדרש להצפין את הקובץ במחשב, ואז למצוא דרך חיצונית ופומבית (כמו אימייל רגיל או רשת חברתית) לשלוח אותו, פעולה החושפת את המשתמש לסכנות ניטור בתווך התקשורת.

חלופה 3: פיתוח מערכת Web ייעודית עם תשתית דואר פנימית (Stegano Mail)

בניית פלטפורמת שרת-לקוח אוטונומית הנגישה ישירות דרך דפדפן אינטרנט, אשר משלבת בתוכה מנועי סטגנוגרפיה מותאמים (לתמונות ואודיו) יחד עם תיבת דואר פנימית מאובטחת. הפלטפורמה מיועדת הן להטמעת הקבצים והן להעברתם בין המשתמשים הרשומים במערכת.

- יתרונות: אין צורך בהתקנה מקומית, ולכן לא נשארות עקבות על מחשב הלקוח (Zero Footprint). כל פעולות העיבוד מתבצעות בשרת מרוחק תוך שמירה על אנונימיות. בנוסף, תשתית ההודעות הפנימית (In-App Messaging) מונעת את הצורך בהוצאת הקבצים לרשתות חיצוניות שאינן מאובטחות.
- חסרונות: חלופה זו דורשת זמן פיתוח ארוך ומורכב, בניית ארכיטקטורת שרת עמידה היועדת לטפל בעומסי זיכרון (בפענוח תמונות ואודיו), ופיתוח או שילוב של מנועים עצמאיים לעיבוד הנתונים.

7. תיאור החלופה הנבחרת ונימוקים לבחירתה

החלופה הנבחרת: חלופה מספר 3 פיתוח מערכת Web היברידית ועצמאית (Stegano Mail).

נימוקים לבחירה:

הוחלט לפתח את מערכת Stegano Mail כפלטפורמת Web מלאה, מאחר והיא החלופה היחידה המספקת מענה שלם ומאובטח ("End-to-End") לבעיה המבצעית שהוגדרה:

1. **אבטחה והיעדר עקבות (Zero Footprint):** בניגוד לחלופה 2 הדורשת התקנת תוכנות מחשירות על המחשב, מערכת Web מאפשרת למשתמש לפעול מכל דפדפן סטנדרטי ולמחוק את היסטוריית הגלישה, ללא השארת עקבות פיזיים של תוכנות ייעודיות.

2. **ערוץ תקשורת סגור (In-App Messaging):** בניגוד לחלופה 1, המערכת מאפשרת למשתמשים לשלוח ולקבל את הקבצים הנגועים דרך מסד הנתונים הפנימי של המערכת (תיבת דואר). פתרון זה מייצר סביבה מבודדת שבה התעבורה אינה עוברת בשרתי דואר ציבוריים ואינה ניתנת ליירוט קל מבחוץ.

3. **התאמה הנדסית לפורמטים (Modified Implementation):** הבחירה בפיתוח מערכת עצמאית מאפשרת לבצע התאמות מדויקות של האלגוריתמים התיאורטיים לצרכי המערכת. לדוגמה, אלגוריתם PVD (Pixel Value Differencing) יושם במערכת תוך התאמה הנדסית: במקום להפעיל אותו על כל התמונה כפי שמקובל בגישה הבסיסית, הוא הוחל אך ורק על הערוץ הכחול (Blue Channel) של הפיקסל באמצעות מניפולציות בינאריות (Bitwise). מאחר שהעין האנושית רגישה פחות לתדרים כחולים, מיקוד ההטמעה בערוץ זה ממקסם את הקיבולת תוך שמירה מוחלטת על איכות התמונה הצבעונית. התאמות הנדסיות מסוג זה אינן מתאפשרות בשימוש בתוכנות מדף סגורות.

4. **אוטומציה וחווית משתמש:** המערכת תוכננה לכלול מנגנון "Header" דינמי, החוסך ממשתמש הקצה את הצורך להזין פרמטרים טכניים בעת פענוח ההודעה (כגון אורך המסר), מה שהופך את הפלטפורמה לגישה ויזואלית גם למשתמשים שאינם בעלי רקע טכני.

8. אפיון המערכת המוצעת

8.1 ניתוח הדרישות מהמערכת

המערכת נדרשת לספק מענה לפונקציות העסקיות הבאות עבור משתמש הקצה:

ניהול משתמשים והרשאות: יכולת רישום למערכת, התחברות מאובטחת באמצעות שם משתמש וסיסמה (המגובבת במסד הנתונים), וניהול Session ללקוח.

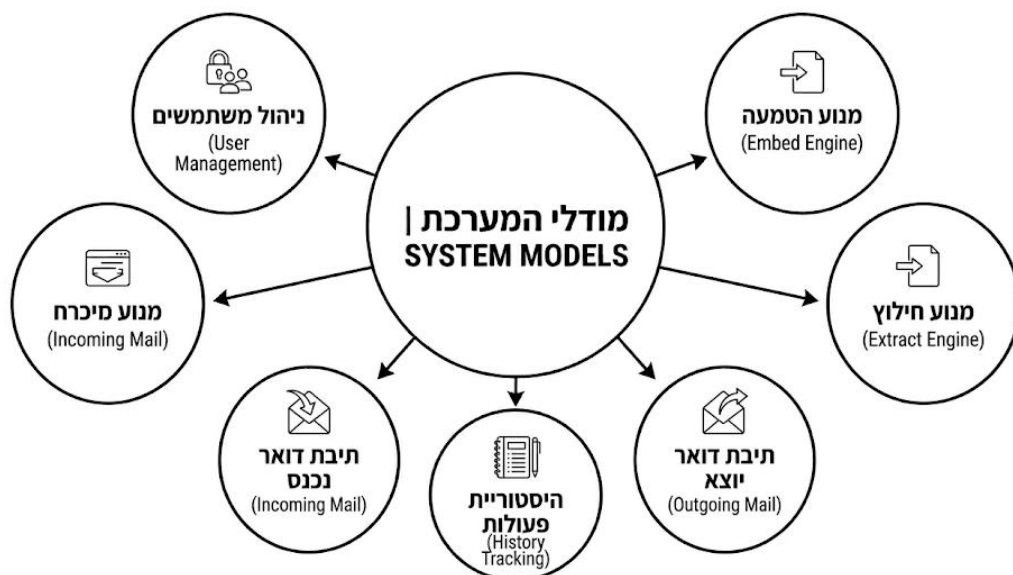
פעולת ההטמעה (Embed): קבלת קובץ כיסוי (תמונה או שמע) וטקסט סודי מהמשתמש, יצירת אובייקט נגוע (Stegno-Object) בזמן אמת בשרת, והחזרתו למשתמש להורדה או לשליחה הלאה.

פעולת חילוץ מדויקת (Extract): יכולת לפענח קובץ נגוע ולשחזר את הטקסט המקורי ב-100% דיוק. הדרישה כוללת תמיכה מלאה ומוכחת בקידוד UTF-8, כדי לאפשר הטמעה וחילוץ של שפות מורכבות, ובראשן השפה העברית.

תקשורת והודעות פנימיות: ניהול תיבת דואר אישית לכל משתמש (Inbox / Sent Messages). המערכת תאפשר לצרף את הקבצים הנגועים להודעות ולשלוח אותם למשתמשים אחרים הרשומים במסד הנתונים, מבלי לצאת מהאפליקציה.

תיעוד היסטוריה: שמירת יומן פעולות (History) המציג למשתמש את כלל פעולות ההטמעה והחילוץ שביצע, כולל גישה לקבצים שנוצרו.

8.2 מודלי המערכת



8.3 אפיון פונקציונלי וביצועים עיקריים

ארכיטקטורת התוכנה מבוססת על שישה מודלים עצמאיים בקוד. לכל מודל יש ייצוג ישיר של מחלקות תצוגה (רכיבי Java נייטיביים בתשתית Vaadin) ומחלקות שירות בשרת:

1. מודול משתמשים (User Module)

- **יישום בקוד:** מחלקות התצוגה LoginView ו-RegisterView המתממשקות מול ה-UserService ו-UserRepository.

- **פעולות המערכת:**

- **אימות נתונים:** קליטת פרטי המשתמש ובידוק מול ה-Database האם שם המשתמש קיים.
- **אבטחה וגיבוב:** הפעלת פונקציית גיבוב (Hashing) על סיסמת המשתמש טרם שמירתה ב-MongoDB.
- **ניהול Session:** יצירה ותחזוקה של תהליך התחברות מאובטח ומבודד עבור המשתמש.

2. מודול הטמעה (Embedding Module)

- **יישום בקוד:** רכיב התצוגה StegnoView (לשונית Embed) המפעיל את ה-StegnoService ונעזר במחלקות התשתית PvdUtils, DsssUtils ובמודול ה-JPEG.

- **פעולות המערכת:**

- **אכיפת אילוצים:** בדיקת קובץ הכיסוי המועלה, חסימת תמונות החורגות מ-2000x2000 פיקסלים ופסילת קבצי אודיו שאינם 16-bit PCM.
- **קידוד טקסט וניהול ה-Header:** המרת המחרוזת הסודית למערך ביטים בקידוד UTF-8, ובניית רצף של 16 ביטים בתחילת הקובץ המייצגים את אורך המסר.
- **הפעלת האלגוריתם:** פירוק מערכי פיקסלים או דגימות שמע, ביצוע מניפולציות בינאריות (Bitwise) להסתרת המידע, והרכבת הקובץ הנגוע מחדש כזרם נתונים (InputStream).

3. מודול חילוץ (Extraction Module)

- **יישום בקוד:** רכיב התצוגה StegnoView (לשונית Extract) ופונקציות הפענוח ב-StegnoService.

- **פעולות המערכת:**

- **זיהוי דינמי:** קריאת 16 הביטים הראשונים של הקובץ הנגוע, המרתם לערך נומרי, וקביעת מספר הבתים המדויק שיש לשלוף.
- **שאיבת נתונים ושחזור:** מעבר שיטתי על הפיקסלים או על דגימות הקול, שאיבת הביטים, המרתם למערך בתים, ושחזור מחרוזת הטקסט המקורית לתצוגה.

4. מודול דואר נכנס (Incoming Mail Module)

- **יישום בקוד:** מחלקת התצוגה InboxView המציגה את הנתונים ברכיבי Grid נייטיביים, בשילוב ה-MessageService וה-MessageRepository.

- **פעולות המערכת:**

- **שאלת הודעות נכנסות:** פנייה למסד הנתונים ושליפת כל המסמכים באוסף ההודעות ששדה הנמען (Receiver Username) שלהם תואם למשתמש המחובר.
- **טעינת קבצים מצורפים:** זיהוי שדה ה-FileID בהודעה, משיכת הקובץ הבינארי מתוך מסד הנתונים בענן, והנגשתו להורדה או לחילוץ ישיר.

5. מודול דואר יוצא (Outgoing Mail Module)

- **יישום בקוד:** מחלקת התצוגה SentView המבוססת על רכיבי טבלה (Grid), בשילוב ה-MessageRepository וה-MessageService.
- **פעולות המערכת:**
 - **יצירת ישות הודעה ושמירת קובץ:** בניית מסמך הודעה (Message Object) הכולל את השולח, הנמען ותוכן גלוי, והמרת הקובץ הנגוע המצורף ל-Binary Data לצורך שמירה ב-MongoDB.
 - **קישוריות נתונים:** הפקת מזהה ייחודי לקובץ וקישורו לשדה ה-FileID של ההודעה, לצד רישום זמן שליחה (Sent Timestamp).

6. מודול היסטוריה (History Module)

- **יישום בקוד:** רכיב התצוגה HistoryView וה-HistoryService.
- **פעולות המערכת:**
 - **תיעוד שקוף:** רישום אוטומטי של כל תהליך הטמעה או חילוץ מוצלח, כולל מטא-נתונים כגון סוג קובץ המקור, סוג המדיה, סוג האלגוריתם וחותמת זמן.
 - **ניהול ארכיון והורדה:** שליפת היסטוריית המשתמש והצגתה, ויצירת ממשק עבודה מול זרמי נתונים המאפשר למשתמש להוריד שוב קבצים נגועים שעובדו בעבר.

8.4 אילוצי המערכת

בשלבי התכנון, הארכיטקטורה והמימוש של מערכת הסטגנוגרפיה ומסרי המדיה המאובטחים, הוגדרו מספר אילוצים ומגבלות טכנולוגיות, תפעוליות ותשתיות. אילוצים אלו נועדו להבטיח את יציבות השרת, למנוע זליגת משאבים בעת ביצוע חישובים מורכבים, לספק אבטחת מידע היקפית ולשמור על חווית משתמש רציפה.

להלן פירוט האילוצים והמגבלות המרכזיים שבהם המערכת פועלת:

1. אילוצי טכנולוגיה (Technology Constraints)

טכנולוגיות החובה, כלי הפיתוח וה-APIs-החיצוניים שבהם נעשה שימוש לבניית הארכיטקטורה של המערכת:

- **שכבת ה-Backend (Spring Boot):** המערכת מבוססת על שפת Java ושלדת התוכנה Spring Boot לניהול שכבת הלוגיקה העסקית, ניתוב האלגוריתמים והזרקת תלויות.
- **הרציונל לקיום האילוח:** פיתוח אלגוריתמים מורכבים דורש שפה חזקה ומונחית עצמים עם ניהול זיכרון מתקדם. שלדה זו מאפשרת להפריד לחלוטין בין שכבת ממשק המשתמש (UI) לבין שכבת הסטגנוגרפיה (Separation of Concerns) ובכך מאפשרת לשדרג אלגוריתם בעתיד מבלי להשפיע על קוד התצוגה.
- **שכבת ה-Frontend (Vaadin Flow Framework):** פיתוח ממשק משתמש רכיבי מנוהל-שרת בשפת Java, המרנדרת אלמנטים ישירות לדפדפן.

- הרציונל לקיום האילוח: בשונה ממעטפות הדורשות שפות נפרדות) כמו React ב, JavaScript- שלדת Vaadin מאפשרת לפתח את ה UI-ישירות ב, Java-דבר המונע "תקורה (Overhead)" של תיאום פרוטוקולים ומבטיח כי אובייקטים ומערכי בייטים של קבצים יעברו בצורה ישירה ומאובטחת.
- **בסיס הנתונים: (MongoDB)** שימוש במסד נתונים NoSQL המאפשר אחסון מסמכים גמישים ושמירת קבצי פלט בינאריים כרכיבי BLOB.
- הרציונל לקיום האילוח: מסדי נתונים רלציוניים (SQL) מבוססים על טבלאות קשיחות שאינן אופטימליות לאחסון דינמי של קבצים MongoDB. מאפשר לשמור את המדיה המוטמעת ואת היסטוריית הפעולות בצורה טבעית ומהירה ללא ביצוע פעולות Join יקרות.
- **API חיצוני להמרת פורמטים: (ConvertService)** עבור קבצי מדיה שאינם נתמכים באופן טבעי על ידי אלגוריתמי הליבה, המערכת מחויבת להסתמך על API חיצוני ייעודי המבצע את ההמרה בענן.
- **ניהול קוד: (GitHub)** שימוש בפרטפורמת GitHub לניהול גרסאות, מעקב אחר קוד המקור וגיובי הפרויקט.

2. אילוצי פלטפורמה (Platform Constraints)

היכן וכיצד המערכת מסוגלת לרוץ ומגבלות הסביבה שלה:

- **סביבת הרצה (Web) בלבד בפורט 8080:** (האפליקציה נגישה מכל דפדפן מודרני, ומותאמת להקשבה קשיחה בפורט הרשת המקומי 8080 של השרת המארח. (server.port = 8080) אילוח ה Web-נועד להבטיח זמינות מרבית מכל סוג מכשיר ומכל מערכת הפעלה ללא צורך בהתקנה מקומית.
- **חובת חיבור אינטרנט רציף וקבוע:** הארכיטקטורה של Vaadin Flow היא ארכיטקטורה מנוהלת-שרת (Stateful Server-side) כל לחיצה בממשק שולחת אירוע לשרת וממתינה לתגובה. מאחר שפעולות ההטמעה והחילוץ רצות ברקע על תהליכונים נפרדים, השרת זקוק לקשר רציף ויציב כדי "לדחוף" באופן אקטיבי את קובץ הפלט או המסר המפוענח חזרה למסך המשתמש. (ui.access) ניתוק רשת קצר יגרום לאובדן הסנכרון וכישלון הפעולה.
- **תלות בענן: MongoDB Atlas** החיבור מבוצע מול שרת ענן חיצוני על מנת שלא להעמיס את אחסון מסד הנתונים פיזית על שרת האפליקציה המקומי. הדבר מחייב גישה חופשית לאינטרנט החיצוני דרך ה Firewall-כדי לבצע פרוטוקולי לחיצות יד ואימות מאובטח מול ה Cluster-בענן בזמן ריצה.

3. אילוצי ביצועים (Performance Constraints)

מדדי מהירות, יעילות וניהול משאבי המחשוב של המערכת:

- **הרצה בתהליכונים נפרדים: (Threads / Concurrency)** אלגוריתמים סטגוגרפיים) במיוחד F5 המבצע התמרת קוסינוס בדיסקרטיות, DCT – קוונטיזציה וסריקת זיגזג (דורשים כוח חישוב מאסיבי מהמעבד. (CPU Bound)
- הרציונל לקיום האילוח: אם היינו מריצים את האלגוריתם על גבי תהליכון הבקשה הראשי של הממשק, הדפדפן של המשתמש היה קופא לחלוטין, (Not Responding) לא היה ניתן לבצע פעולות (כמו לחיצה על כפתור "ביטול") והשרת היה עלול לנתק את החיבור עקב פסק זמן (Timeout) פיצול התהליך לתהליכון רקע נפרד שומר על ממשק תגובתי וחי.
- **שמירה כקובץ זמני בדיסק (toTempFile) וניקוי ידני:** החזקת קבצים בינאריים גדולים ישירות בזיכרון ה RAM-הדינמי עבור כל משתמש במקביל תגרום לצריכת זיכרון מופרזת ולחריגת זיכרון (Out of Memory) שתפיל את השרת כולו. הזרמת הקובץ לדיסק כקובץ זמני שומרת על ה RAM-פנוי. המחיקה הידנית המיידית בסיום התהליך חיונית כדי למנוע הצטברות של קבצי זבל שימלאו את כונן השרת וישביתו אותו.

4. אילוצי אבטחה (Security Constraints)

מנגנוני ההגנה והצפנת המידע הרגיש במערכת:

- **בקרת גישה קשיחה על הדפים: (BeforeEnterObserver)** גישה למסך הסטגנוגרפיה מותנית באופן קשיח בקיומו של ששן פעיל ומאומת. (VaadinSession) ניסיונות כניסה של אורחים אנונימיים נחסמים ומנותבים אוטומטית (Reroute) חזרה לדף הרישום וההתחברות. אילוץ הזיהוי מוודא שרק משתמש רשום, שזהותו ידועה, יכול לצרוך את משאבי השרת הכבדים ולמנוע מתקפות מניעת שירות. (DoS)
- **הצפנת סיסמאות באמצעות BCryptPasswordEncoder** שמירת סיסמאות כטקסט גלוי (Plaintext) היא כשל אבטחה חמור. המערכת מאלצת הצפנה חד-כיוונית באמצעות רכיב אבטחה ייעודי המממש את אלגוריתם BCrypt לפני הזנתן למסד הנתונים. אלגוריתם זה מבוסס על "האטת" תהליך החישוב, (Key Stretching) ומבטיח שגם אם הנתונים נגנבו, לא ניתן יהיה לפענח את הסיסמאות באמצעות מתקפות כוח גס.
- **תיעוד פעולות מאובטח: (Audit Trail)** כל פעולת הטמעה מוצלחת מחייבת זיהוי מוחלט של שם המשתמש מהששן ושמירת רשומה חתומה בהיסטוריית בסיס הנתונים לצורכי מעקב, ניטור ומניעת הכחשת פעולות.

5. אילוצי נתונים (Data Constraints)

כיצד המידע מנוהל, מוגבל ונשמר במערכת, תוך הבחנה בין סוגי הקבצים:

- **הפרדה בין קבצים נתמכים ללא-נתמכים ומכסת ה-API-**
 - **קבצים בתמיכה טבעית ומובנית (ללא הגבלת כמות):** המערכת תומכת באופן מובנה ומוחלט בקבצי תמונה מסוג PNG, JPG, JPEG ובקבצי שמע מסוג WAV. קבצים אלו מעובדים מקומית על ידי אלגוריתמי הליבה של האפליקציה ללא צורך בשירות חיצוני, ולכן המשתמש יכול להעלות ולעבד אותם ללא הגבלת כמות יומית.
 - **קבצים שאינם נתמכים (מגבלת 10 קבצים ביום):** עבור קבצי מדיה שאינם נתמכים באופן טבעי (כמו תמונות WebP או GIF), המערכת מפעילה באופן אוטומטי שירות המרה (ConvertService) המתממשק ל API-חיצוני. מאחר והפרויקט עושה שימוש בגרסה החינמית של שירות זה (Free Tier Quota) בשל אילוצי תקציב, ספק ה API-מציב מגבלה קשיחה המאפשרת עד 10 המרות בלבד ביממה. חריגה ממכסה זו תחסום את השירות החיצוני ותשבית את מנגנון ה-Fallback ולכן קיים אילוץ תוכנתי המנטר וחוסם העלאות של פורמטים אלו מעבר למכסה.
- **מגבלת נפח של 15 MB/16MB בשתי שכבות:** חסם עליון קשיח המוגדר בשתי שכבות אבטחה: מגבלה של 15 MB ברמת רכיב ה UI-ומגבלה תואמת של 16 MB ברמת ה Multipart Servlet של שרת ה-Spring Boot. חסימה זו מייצרת מנגנון "הגנה לעומק" – אם תוקף ינסה לעקוף את ה UI-ולשלוח קובץ ענק ישירות ל API-באמצעות סקריפט, השרת יזהה את החריגה מ-16 MB ויפיל את הבקשה מיד בשלב ה-Parsing, עוד לפני שהיא תגיע ללוגיקה.
- **חסימת קבצים שאינם מדיה (PDF/DOCX):** אלגוריתמים סטגנוגרפיים מסתמכים על יתירות מידע (Redundancy) הקיימת באופן טבעי בפיקסלים או בתדרים (רעש דיגיטלי עדין שהעין או האוזן אינן מבחינות בו). בקבצי טקסט משרדיים אין יתירות כזו; כל שינוי של ביט בודד ישנה את התווים עצמם או ישחית את המסמך. לכן, הרצת האלגוריתמים עליהם היא בלתי אפשרית מבחינה לוגית והם נחסמים לחלוטין במערכת.
- **חובת שלמות נתונים בהטמעה:** ביצוע תהליך ההסתרה מאלץ הזנה בו-זמנית של שני קלטים: קובץ מדיה תקין ומחרוזת טקסט המכילה את המסר המיועד להצפנה. מחסור באחד מהם יגרור עצירה של הפעולה.

6. אילוצי משתמש (User Constraints)

קהל היעד ומאפייני חווית המשתמש הנדרשים:

- **קהל יעד מוגדר:** משתמשים רשומים ומאומתים בלבד המעוניינים לבצע העברת מסרים דיסקרטית ומאובטחת באמצעות קבצי תמונה ושמע.
- **שפת הממשק:** המערכת מונגשת בממשק מלא בשפה העברית, כולל כלל הודעות המערכת, חלונות ההמתנה (Spinners) והתראות הסטטוס המוקפצות למשתמש.

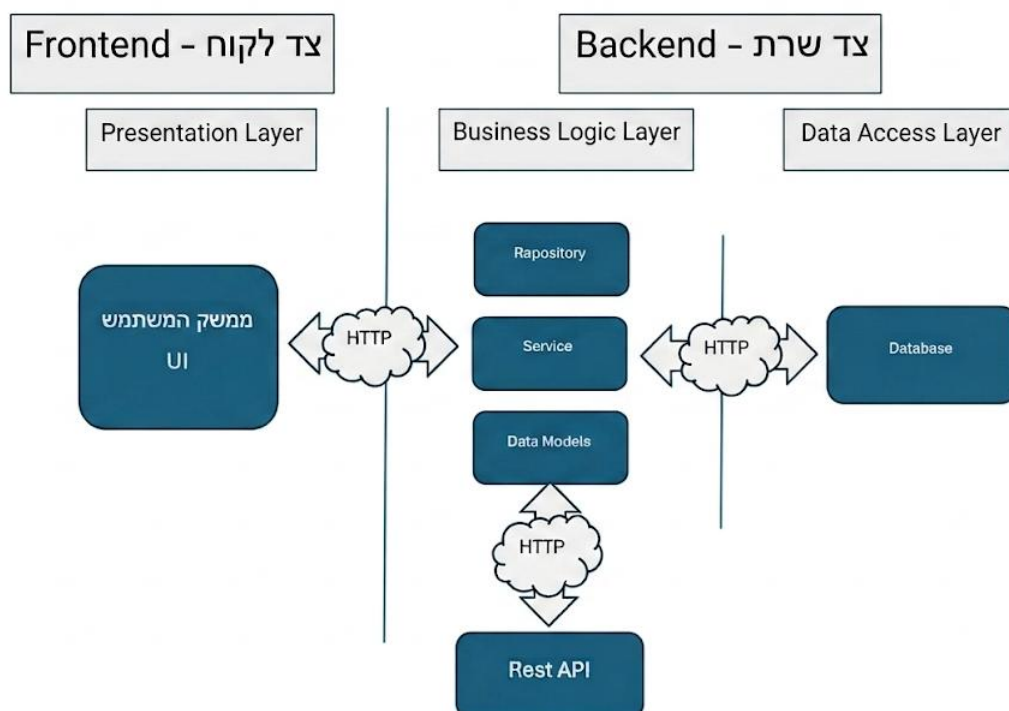
7. אילוצי משאבים (Project Constraints)

המגבלות והמציאות של פיתוח המערכת:

- **אילוץ תקציבי (כלים חינמיים בלבד):** פיתוח המערכת מתבצע בתקציב של 0 ₪ ונשען אך ורק על ספריות קוד פתוח ומסלולים חינמיים) כמו MongoDB Atlas Free Tier וגרסת ה Free Tier של Convert API המגבילה ל-10 בקשות ביום אילוץ זה מכתב את רמת צריכת המשאבים של האפליקציה ומאלץ חסימות תוכנותיות מבוקרות כדי לא לחרוג למסלולים הדורשים תשלום.

9. תיאור הארכיטקטורה של המערכת

9.1 ארכיטקטורת המערכת



מערכת Stegano Mail מבוססת על ארכיטקטורת שלוש שכבות (Three-Tier Architecture), כפי שניתן לראות בתרשים, המאפשרת הפרדה מבנית ולוגית בין ממשק המשתמש, מנוע העיבוד ואחסון הנתונים. מבנה זה, כמתואר, מבטיח אבטחה גבוהה, תחזוקה נוחה וביצועים אופטימליים.

פירוט השכבות במערכת:

1. שכבת התצוגה (The Presentation Layer):

- **שיוך בתרשים:** שכבה זו מיוצגת בתרשים כ-"Frontend - צד לקוח" ומוגדרת כ-"Presentation Layer". היא כוללת את רכיב "ממשק המשתמש UI".

- רכיב "ממשק המשתמש UI", שפותח במלואו באמצעות תשתית **Vaadin** (המאפשרת יצירת ממשק מודרני מבוסס Web סווס תוך שימוש ברכיבי Java נייטיביים בלבד), אחראי על קליטת הקלטות מהמשתמש (כגון העלאת קבצי תמונה ושמע, הזנת טקסט סודי ובקשות רישום). כמו כן, הוא אחראי על הצגת תיבות הדואר הפנימיות והקבצים לאחר חילוץ. רכיבי התצוגה אינם מבצעים עיבוד אלגוריתמי כבד, אלא מתקשרים בצורה רציפה מול שכבת ה-Backend דרך ענן תקשורת ה-HTTP.

2. שכבת הלוגיקה העסקית (Business Logic Layer):

- **שיוך בתרשים:** שכבה זו מיוצגת בתרשים כחלק מ-"Backend - צד שרת", תחת ההגדרה "Business Logic Layer". היא מורכבת מ"ערימת השרת" (Backend Stack), הכוללת את הרכיבים: "Repository", "Service", "Data Models", ו-"Rest API".
- שכבה זו, המפותחת בסביבת **Spring Boot**, היא מוח המערכת. היא מנהלת את הפעולות המורכבות והאבטחה:

- **ניהול תהליכים:** אימות משתמשים, ניהול סשנים ותקשורת מאובטחת.
- **מנוע סטגנוגרפיה (Stegano Core) ומודול ה-JPEG הפנימי:** הלוגיקה המרכזית של המערכת מתרחשת בתוך ערימת השרת. רכיבי ה-"Service" וה-"Data Models" עובדים יחד כדי לייצג ולעבד את נתוני המדיה (כגון מניפולציות במישור הפיקסלים או עיבוד אותות שמע). עבור טיפול בפורמט JPEG הדורש כיווץ, המערכת משתמשת במודול פנימי (שאינו מוצג ישירות כרכיב נפרד בתרשים אך משולב בתוך הלוגיקה של ה-Backend) המקבל את הנתונים, מבצע את ההטמעה במקדמי התדר כ"קופסה שחורה", ומחזיר לשרת את זרם הנתונים המוכן.
- **גישה לנתונים:** רכיב ה-"Repository" משמש כאינטרפייס לגשת לנתונים. רכיב ה-"Rest API" מספק ממשק תקשורת תקני לגשת לנתונים דרך HTTP, כפי שמוצג בתרשים, ומאפשר גמישות ועבודה מול מודלים מוגדרים מראש של נתונים.

3. שכבת הנתונים (Data Access Layer):

- **שיוך בתרשים:** שכבה זו מיוצגת בתרשים כחלק מ-"Backend - צד שרת", תחת ההגדרה "Data Access Layer", וכוללת את רכיב ה-"Database".
- שכבה זו מבוססת על מסד נתונים מסוג **MongoDB – NoSQL** הפועל בענן. רכיב ה-"Database" מנהל את אוספי המשתמשים, היסטוריית הפעולות ותיבות הדואר. ייחודיות השכבה היא יכולתה לשמור בצורה יעילה אובייקטים בינאריים גדולים (Binary Data), כגון קבצי התמונה והשמע הנגועים (בשימוש ב-**GridFS** של MongoDB, המאפשר לשמור קבצים גדולים מהמגבלה של מסמך בודד), לצד מטא-דאטה בתצורת מסמכי **JSON** גמישים.

זרימת המידע בין השכבות (Data Flow):

התהליך המלא של זרימת המידע, כפי שניתן לראות מהחצים וענני ה-HTTP בתרשים, הוא כדלקמן:

1. **קלט מהמשתמש:** המשתמש מזין נתונים ומעלה קובץ דרך רכיב "ממשק המשתמש UI" שבשכבת ה-Frontend.
2. **העברת נתונים ל-Backend:** הנתונים מועברים בצורה מאובטחת דרך ענן תקשורת ה-HTTP אל שכבת ה-Backend, שם הם מגיעים תחילה לערימת השרת (Backend Stack), וספציפית דרך רכיבי ה-"Rest API" וה-"Data Models" (הגורמים המייצגים את בקשות המשתמש).
3. **עיבוד:** רכיבי ה-Backend Stack (Service ו-Data Models) בשיתוף פעולה מעבדים את הקובץ. הלוגיקה של Stegano Core ומודול ה-JPEG הפנימי מופעלת כדי לבצע את מניפולציות ההטמעה.

4. **גישה לנתונים ושמירה:** בעזרת רכיב ה-"Repository", המערכת פונה דרך ענן ה-HTTP שבתוך ה-Backend אל ה-"Database". שם נשמר הקובץ הבינארי החדש ורשומת ההודעה המקושרת אליו.
5. **משוב:** ה-"Database" מחזיר תשובה המאשרת את השמירה המוצלחת בענן. ה-Backend פונה חזרה דרך ענן תקשורת ה-HTTP אל רכיב "**ממשק המשתמש UI**", אשר מעדכן את תצוגת ממשק המשתמש בזמן אמת.

9.2 מודל שרת לקוח

הארכיטקטורה הכללית של המערכת מתבססת על מודל תקשורת של שרת-לקוח (Client-Server), הפועל על פי עקרון של "בקשה ותגובה" (Request-Response).

במודל זה, הלקוח (Client) – שהוא למעשה דפדפן האינטרנט של המשתמש – מהווה את הרכיב היוזם. הלקוח שולח בקשות לקבלת שירותים או מידע (כגון העלאת קובץ להטמעה או בקשה לשליפת הודעות). מנגד, השרת (Server) מתפקד כרכיב המאזין; הוא קולט את הבקשות, מפעיל את הלוגיקה העסקית הנדרשת, ומחזיר את הפלט המעובד חזרה אל הלקוח.

יישום מודל זה במערכת Stegano Mail הוא קריטי והנדסי. הוא מאפשר לרכז את משאבי העיבוד הכבדים – כגון הרצת אלגוריתמי הסטגנוגרפיה המורכבים (PVD, F5, DSSS) וניהול אבטחת המידע – באופן בלעדי בשרת המרכזי. חלוקת תפקידים זו מותירה את צד הלקוח כ"לקוח רזה" (Thin Client), אשר נגיש לשימוש מכל דפדפן סטנדרטי ללא תלות בחומרת הקצה של המשתמש וללא צורך בהתקנת תוכנות עיבוד מקומיות.

9.3 תהליכים של מע' ההפעלה

מערכת Stegano Mail מופעלת בצד השרת כפרוסס (Process) עצמאי בתוך סביבת הריצה של Java (ה-JVM), המנוהלת על ידי מערכת ההפעלה המארחת (Host Operating System). לצורך ניהול זרימת המידע וטיפול בבקשות המשתמשים, המערכת מקיימת אינטראקציה הדוקה עם משאבי הליבה של מערכת ההפעלה:

- **ניהול תהליכונים (Thread Pool & Multithreading):** שרת האפליקציות המובנה (Apache Tomcat) מקצה תהליכונים (Threads) ייעודיים מתוך מאגר קבוע (Thread Pool) המנוהל ברמת מערכת ההפעלה. בכל פעם שמשתמש מבצע פעולת הטמעה או חילוץ, או פונה לתיבת הדואר הפנימית, מערכת ההפעלה מנתבת את הבקשה לתהליכון נפרד. מנגנון זה מונע חסימה (Non-blocking I/O) ומאפשר עיבוד מקבילי של קבצי מדיה כבדים עבור מספר משתמשים בו-זמנית.
- **הקצאת זיכרון דינמית (Heap Space & RAM):** פעולות הסטגנוגרפיה (כמו טעינת מטריצות פיקסלים של תמונות PNG/JPEG או פירוק דגימות שמע מקבצי WAV) מתבצעות ישירות על זיכרון המחשב ללא כתיבה זמנית לדיסק (In-Memory Processing). מערכת ההפעלה אחראית על הקצאה וניהול של מרחב הזיכרון (Heap Space) עבור הפרוסס של השרת, ומפעילה מנגנוני פינוי זיכרון אוטומטיים (Garbage Collection) כדי למנוע זליגות זיכרון בעת טיפול במערכי בתים (Byte Arrays) גדולים.
- **אינטראקציה עם כרטיס הרשת (NIC) ופורטים:** השרת מאזין באופן קבוע לפורט תקשורת לוגי ייעודי (כגון Port 8080 לתעבורת פיתוח או Port 443 לתעבורת HTTPS מאובטחת). כרטיס הרשת הפיזי או הווירטואלי (Network Interface Card - NIC) של השרת קולט את חבילות המידע הבינאריות המגיעות מהדפדפנים של המשתמשים ברשת האינטרנט, ומעביר אותן דרך שכבת מנהלי ההתקנים (Drivers) של מערכת ההפעלה ישירות אל ה-Socket הלוגי של האפליקציה. באותו אופן, תהליכי קלט/פלט (I/O) מנוהלים מול כרטיס הרשת לצורך שליחה וקבלת מסמכי JSON וקבצים בינאריים מול מסד הנתונים MongoDB המנוהל בענן.

9.4 תיאור פרוטוקולי תקשורת

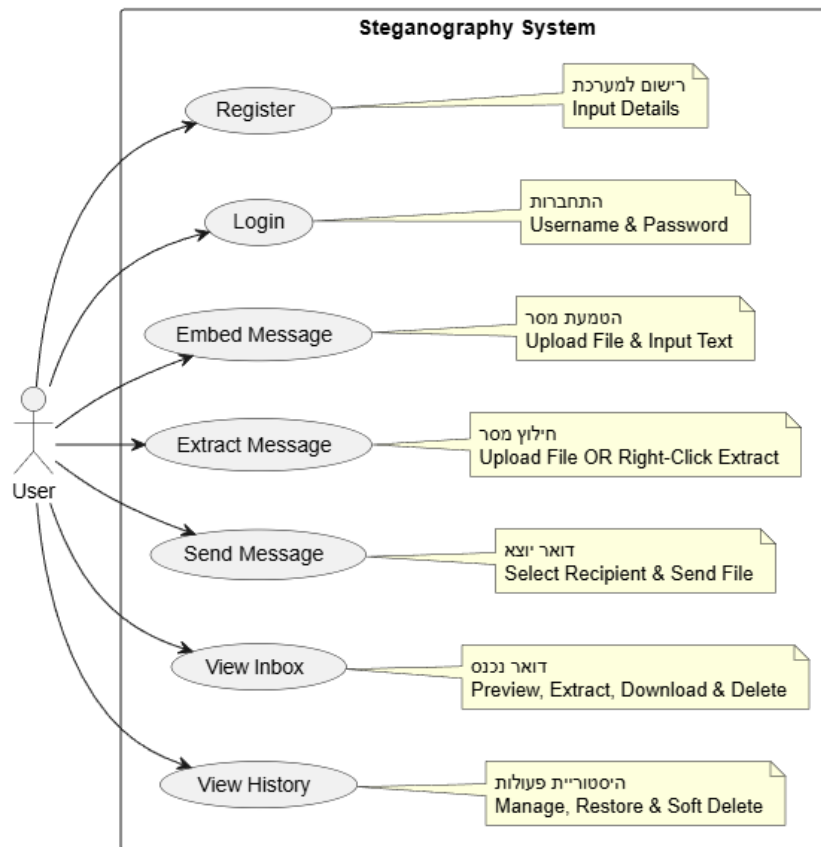
- **HTTP / HTTPS:** הוא פרוטוקול תקשורת ברשת שמאפשר העברת בקשות ותשובות בין מחשבים. HTTPS הוא הגרסה המאובטחת שלו, שמצפינה את הנתונים כדי למנוע קריאה או שינוי על ידי צדדים שלישיים. פרוטוקול זה נפוץ מאוד בשימוש ברשת האינטרנט ומאפשר במערכת תקשורת עקבית ומובנית בין השרת ללקוח.
 - **JSON:** הוא פורמט העברת נתונים פשוט, קריא ומבנה, המאפשר העברת מטא-דאטה, פרמטרים שונים של אלגוריתמים, הודעות מערכת, רשימות אוספים או תוצאות עיבוד בין הלקוח לשרת. השימוש ב-JSON מקל על פיתוח המערכת מכיוון שכל שפה מודרנית (וכמו Java) תומכת בפענוח והמרת JSON לאובייקטים פנימיים, ומבטיח אחידות בין כל חלקי המערכת.
 - **TCP/IP:** הוא מודל התקשורת שמאפשר העברת המידע ברשת בצורה אמינה ומדויקת. TCP אחראי על חלוקת המידע לחבילות, בדיקת שלמותן ושחזור סדר ההגעה שלהן בצד המקבל, מה שקריטי במיוחד במערכות סטגנוגרפיות שבהן חבילה אחת שלא הגיעה או הגיעה בסדר שגוי עלולה לשבש את הקובץ הבינארי ולמנוע חילוץ המסר. IP אחראי על ניתוב החבילות מהרכיב השולח אל הרכיב המקבל ברשת הגלובלית ומוודא שהמידע מגיע לכתובת הנכונה בצורה אופטימלית. השילוב TCP/IP מבטיח העברה אמינה ומדויקת של קבצים בינאריים, מסרים סטגנוגרפיים ומידע קריטי גם ברשתות מורכבות או לא יציבות.
- השימוש בפרוטוקולים אלו מאפשר העברה אמינה ומדויקת של קבצים בינאריים בין הלקוח לשרת, מבטיח שהמידע המתקבל יהיה עקבי, שלם ומוכן לעיבוד על ידי האלגוריתמים הסטגנוגרפיים, מאפשר ניהול יעיל של פעולות CRUD בענן, תוך שמירה על תקשורת בטוחה ומהירה, ומספק בסיס איתן להרחבות עתידיות של המערכת.

9.5 שירותים חיצוניים

- המערכת עושה שימוש בשירותים חיצוניים (Third-Party APIs) מבוססי ענן על מנת להרחיב את יכולות העיבוד שלה ולתמוך בהמרות מדיה מתקדמות:
- **CloudConvert API:** שירות ענן חיצוני המשמש את המערכת לצורך ביצוע מניפולציות והמרת פורמטים של קבצי מדיה (כגון המרת תמונות או קבצי שמע לפורמטים הקשיחים הנדרשים על ידי אלגוריתמי הסטגנוגרפיה, כדוגמת WAV ב-16 bit).
 - **אופן התקשורת והמימוש:** המערכת מתקשרת עם שירות ה-CloudConvert באמצעות ארכיטקטורת **REST API**. הלוגיקה ממומשת בצד השרת בלבד (Backend) כדי לשמור על אבטחת מפתחות ה-API (API Keys). לצורך כך, המערכת משתמשת בספריית **OkHttp3** להוצאה וניהול של בקשות HTTP (Requests) וקבלת התשובות (Responses) בצורה סינכרונית ואסינכרונית.
 - **העברת נתונים (JSON):** הגדרות ההמרה, ניתוב הקבצים וקבלת הסטטוס מה-API מתבצעים תוך שימוש בפורמט הסטנדרטי **JSON**. המערכת עושה שימוש בספריית הליבה **org.json** לשם בנייה (Serialization) ופענוח (Deserialization) של מבני הנתונים המועברים בתקשורת מול השירות החיצוני.

10. ניתוח תרחישים וזרימת המידע במערכת המוצעת

10.1 תרשימי תרחיש Use case Diagram



תרשים מקרי השימוש (Use Case Diagram) מתאר את האינטראקציה של משתמש הקצה (Actor) מול המערכת, ומציג את הפונקציות המרכזיות הפתוחות בפניו מרגע כניסתו לאתר. רישום (Register): יצירת חשבון משתמש חדש במערכת.

פירוט מקרי השימוש המרכזיים במערכת (מנקודת מבטו של המשתמש):

רישום (Register): יצירת חשבון משתמש חדש במערכת על ידי הזנת פרטים מזהים וסיסמה.

התחברות (Login): כניסה מאובטחת למערכת באמצעות אימות של שם משתמש וסיסמה.

הטמעת מסר (Embed Message): המשתמש בוחר ומעלה קובץ "כיסוי" (תמונה או קובץ שמע) ומקליד בתיבת טקסט את המסר הסודי. (המערכת מזהה את הפורמט, בוחרת אוטומטית את האלגוריתם ומבצעת את ההצפנה).

חילוץ מסר (Extract Message): המשתמש יכול להעלות קובץ נגוע למסך החילוץ, או לחלופין **לחץ מסר בזמן אמת** (בלחיצת כפתור או קליק ימני) ישירות מתוך התצוגה המקדימה בתיבת הדואר או בהיסטוריה, ללא צורך בהורדת הקובץ למחשב.

שליחת דואר יוצא (Send / Outgoing Mail): המשתמש בוחר נמען מתוך המערכת, מזין תוכן הודעה גלוי ומצרף קובץ נגוע לשליחה. במקרה של טקסט ארוך, המשתמש יכול לפתוח חלונת מודאלית לקריאה נוחה.

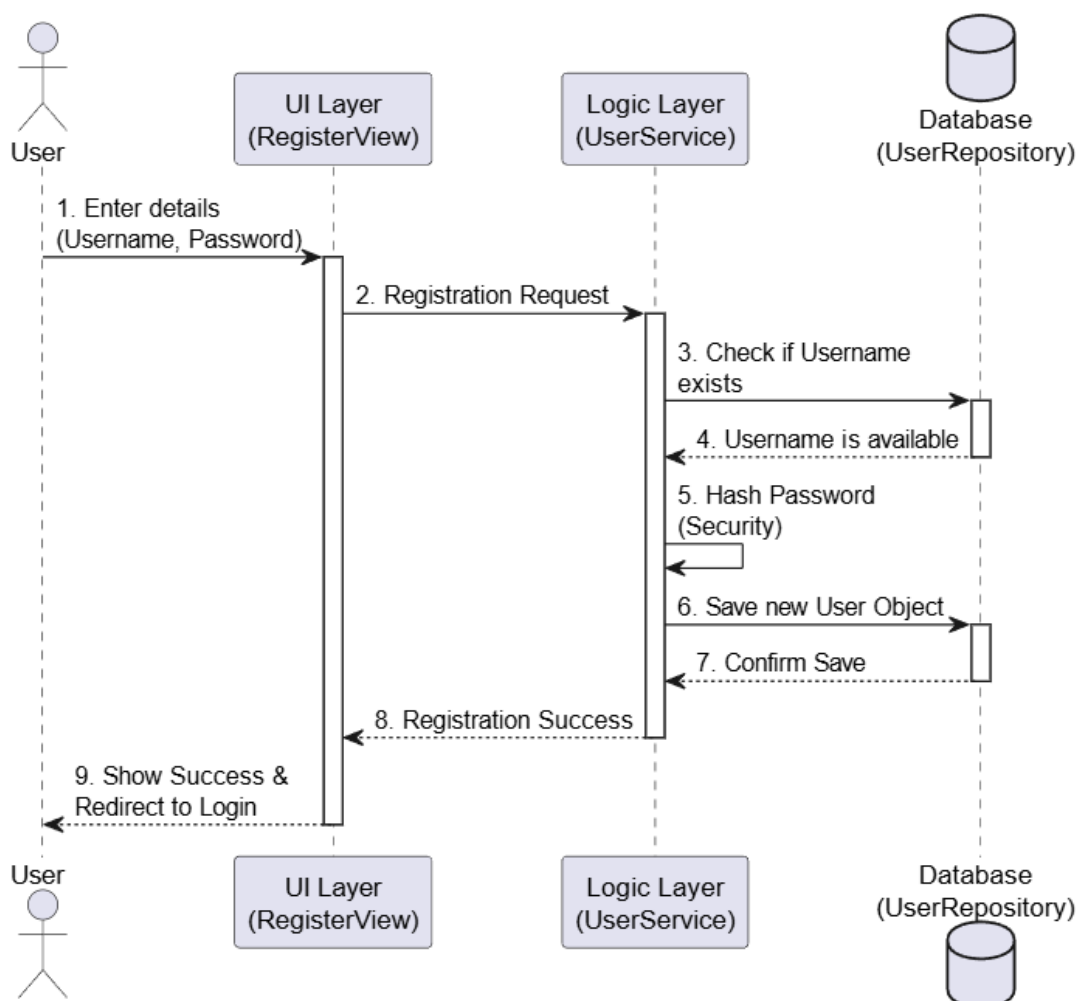
צפייה בדואר נכנס (View Inbox): המשתמש קורא הודעות שהתקבלו. הוא יכול לנגן או לצפות בקבצים המצורפים ישירות מתוך הדפדפן (Media Player) פנימי, (למחוק הודעות מהתיבה, או להוריד את הקבצים למחשבו האישי).

ניהול היסטוריה (View History): המשתמש צופה ביומן הפעולות האישי. הוא יכול להוריד קבצים מחדש, לצפות בהם, או לבצע מחיקה של קבצים (מחיקה לוגית מאובטחת הדורשת חלון אישור).

10.2 תרשימי רצף Sequence Diagram

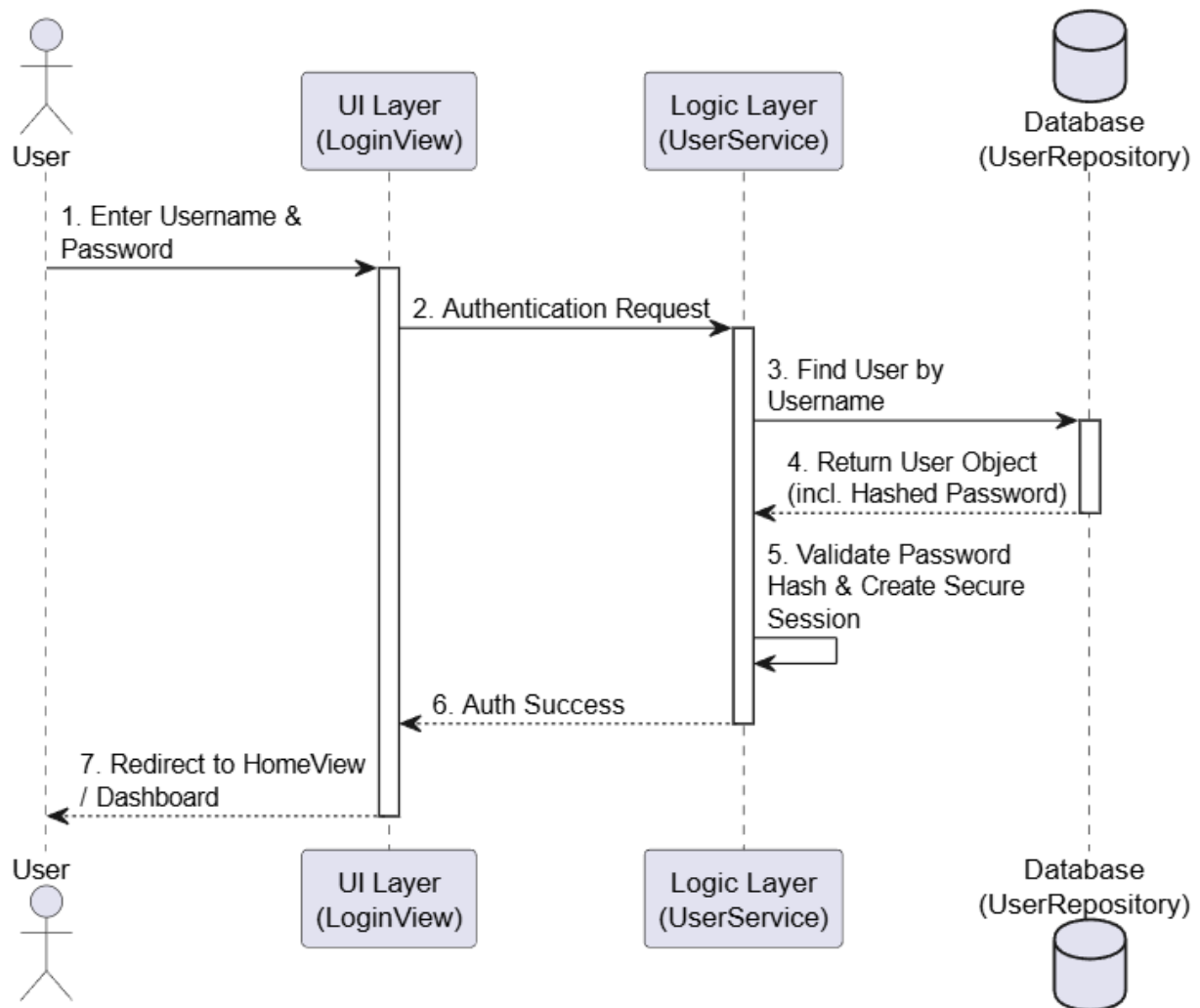
1. תרשים רישום למערכת (Register)

- **מטרת התרשים**: תיאור זרימת הפעולות ליצירת משתמש חדש והגנה על סיסמאותיו.
- **האובייקטים המשתתפים**: משתמש הקצה (User), ממשק המשתמש (RegisterView), שכת הלוגיקה (UserService), ומסד הנתונים (UserRepository).
- **פירוט שלבי התרחיש**:
 1. המשתמש מזין את פרטי ההרשמה (שם משתמש וסיסמה).
 2. ה UI-שולח את בקשת הרישום אל שכת הלוגיקה.
 3. הלוגיקה בודקת מול מסד הנתונים האם שם המשתמש כבר תפוס.
 4. מסד הנתונים מחזיר אישור כי השם פנוי.
 5. הלוגיקה מבצעת פונקציית גיבוב (Hashing) על הסיסמה שהוזנה מטעמי אבטחה.
 6. הלוגיקה שומרת את אובייקט המשתמש החדש במסד הנתונים.
 7. מסד הנתונים מחזיר אישור על הצלחת השמירה אל הלוגיקה.
 8. הלוגיקה מעבירה אישור ל UI-המציג התראת הצלחה למשתמש ומעביר אותו למסך ההתחברות.



2. תרשים אימות והתחברות משתמשים (Login)

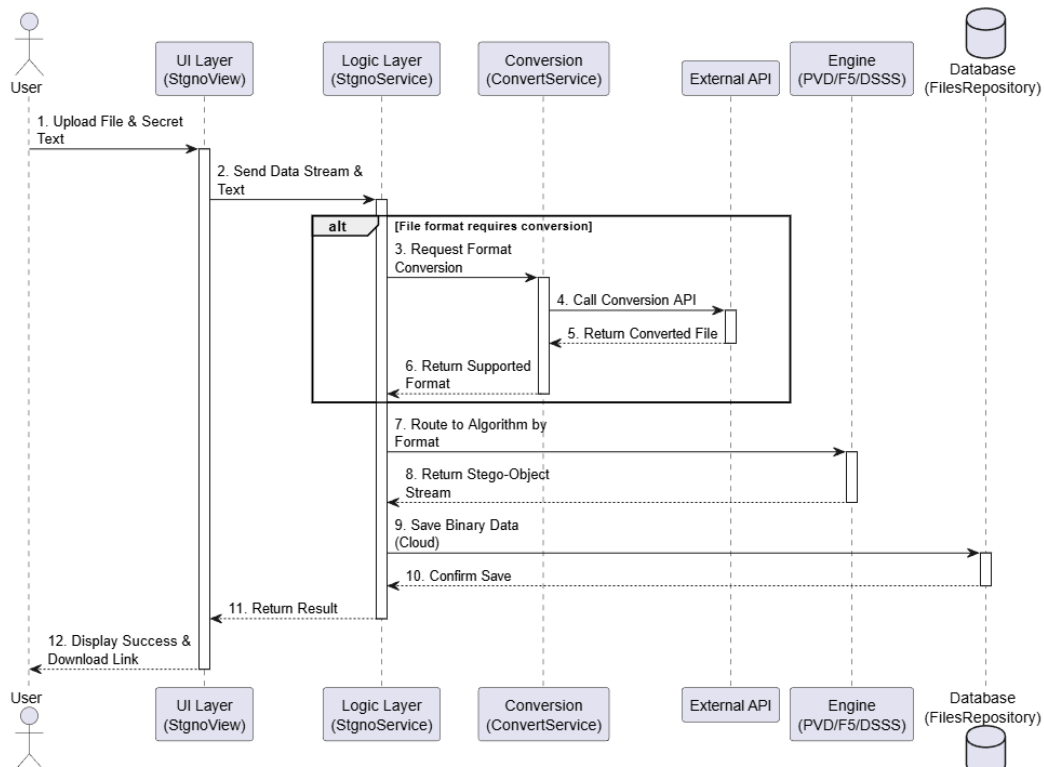
- **מטרת התרשים:** הצגת מנגנון אימות הזהות לאבטחת הגישה למסכי המערכת הפנימיים.
- **האובייקטים המשתתפים:** משתמש הקצה (User), ממשק המשתמש (LoginView), שכבת הלוגיקה (UserService), ומסד הנתונים (UserRepository).
- **פירוט שלבי התרחיש:**
 1. המשתמש מזין את שם המשתמש והסיסמה שלו.
 2. ה UI-שולח בקשת אימות (Authentication) אל שכבת הלוגיקה.
 3. שכבת הלוגיקה מחפשת את רשומת המשתמש במסד הנתונים לפי השם שהוזן.
 4. מסד הנתונים מחזיר את אובייקט המשתמש (הכולל את הסיסמה המגובבת).
 5. הלוגיקה משווה בין הסיסמה המוזנת לסיסמה המגובבת במסד, ובמקרה של התאמה - מייצרת ששן (Session) מאובטח.
 6. מוחזרת תשובה חיובית אל ה UI.
 7. ה UI-משנה ניתוב ומעביר את המשתמש אל סרגל הניווט הפנימי של המערכת.



3. תרשים הטמעת מסר סודי (Embed Message & Conversion)

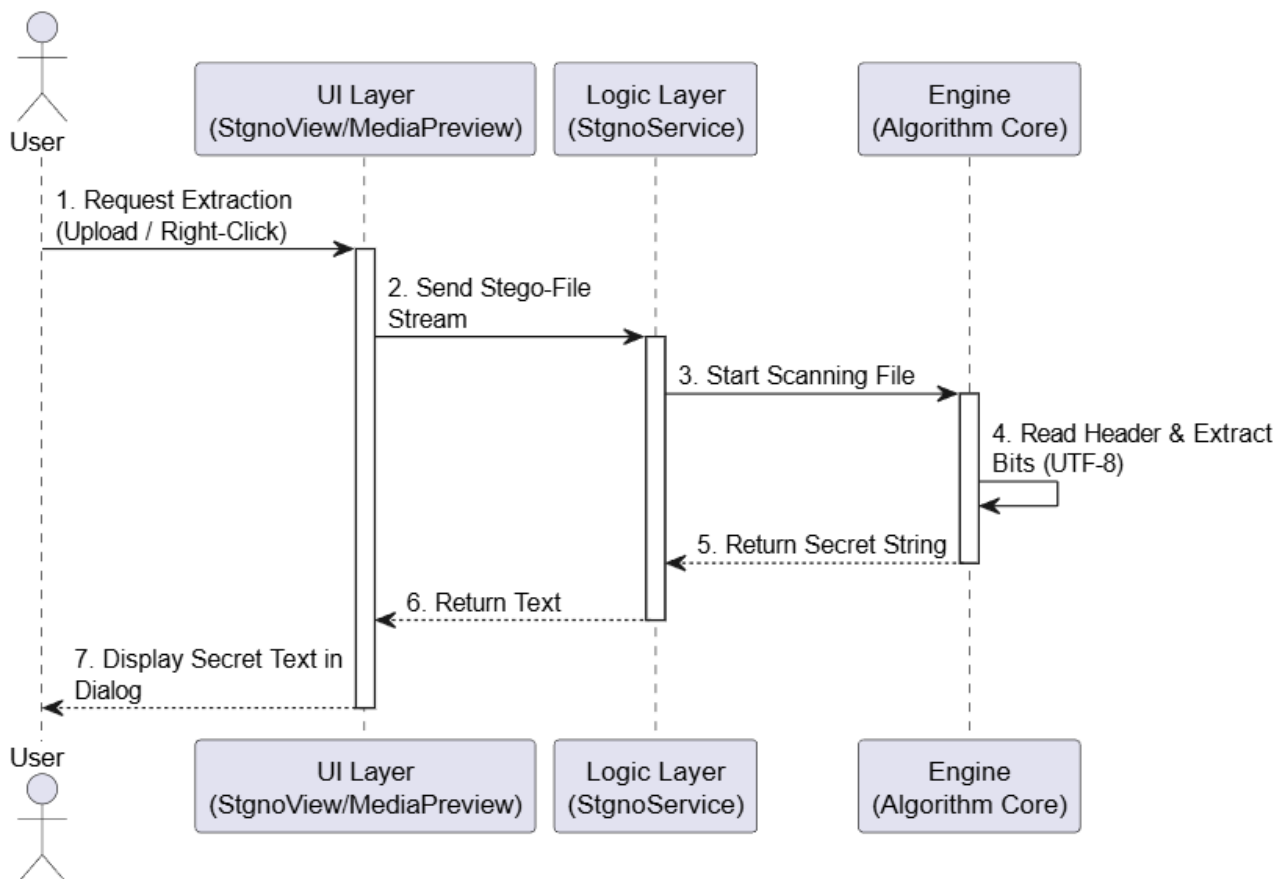
- **מטרת התרשים:** תיאור תהליך יצירת ה-Stegno-Object כולל שלב אפשרי של המרת פורמטים (כגון שמע או תמונה) למבנה הנתמך באלגוריתם באמצעות קריאה ל API-חיצוני.
- **האובייקטים המשתתפים:** משתמש הקצה (User), ממשק המשתמש (StegnoView), שכבת הלוגיקה (StegnoService), שירות ההמרה (ConvertService), שירות המרה API חיצוני, מנוע האלגוריתמים (Engine), ומסד הנתונים (FilesRepository).
- **פירוט שלבי התרחיש:**

1. המשתמש מעלה קובץ "כיסוי" למערכת ומקליד את המסר הסודי שברצונו להחביא.
2. שכבת ה UI-מעבירה את הקובץ והטקסט אל שירות הלוגיקה בשרת.
3. **תנאי - המרה:** במידה והקובץ דורש המרה לפורמט נתמך למשל המרה ל WAV-לאלגוריתם DSSS, שכבת הלוגיקה קוראת לשירות ConvertService.
4. שירות ההמרה פונה ל API החיצוני עם זרם הנתונים המקורי.
5. ה API מחזיר את הקובץ בפורמט התקין והמומר.
6. ה ConvertService מחזיר את הקובץ לשכבת הלוגיקה.
7. הלוגיקה מנתבת את הקובץ המוכן לאלגוריתם הסטגנוגרפיה הרלוונטי.
8. מנוע האלגוריתם מבצע את ההטמעה ומחזיר ללוגיקה את זרם הנתונים של ה-Stegno-Object.
9. הלוגיקה פונה למסד הנתונים כדי לשמור את המידע הבינארי בענן.
10. מסד הנתונים מחזיר אישור הצלחה.
11. שכבת הלוגיקה מחזירה את הקובץ המעובד ל UI.
12. ה UI מציג הודעת הצלחה ואת הקובץ למשתמש.



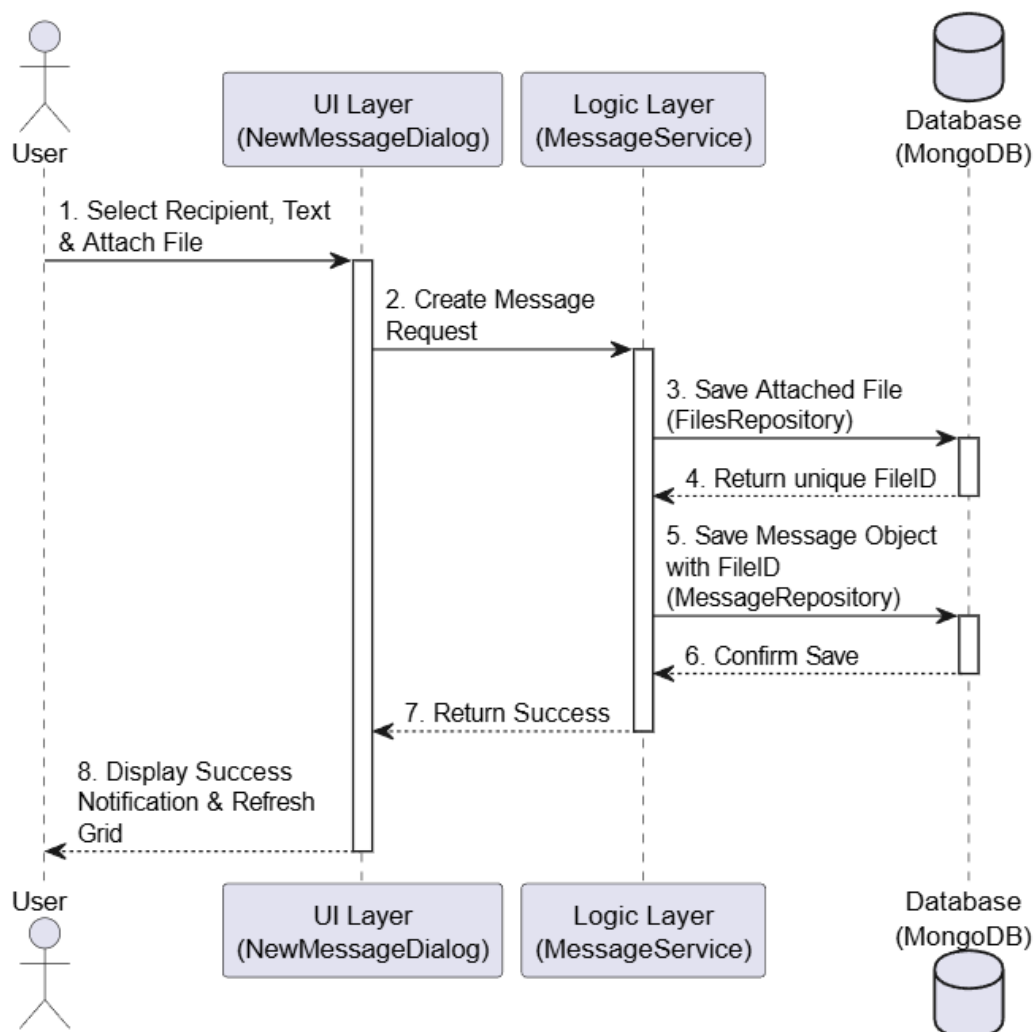
4. תרשים חילוף מסר סודי (Extract Message)

- תרשים **מטרת התרשים**: הדגמת תהליך הפענוח ושאיבת הטקסט מתוך קובץ סטגנוגרפי ללא דרישת פרמטרים מהמשתמש.
- **האובייקטים המשתתפים**: משתמש הקצה (User), ממשק המשתמש (UI Layer), שכבת הלוגיקה (StegnoService), ומנוע הפענוח (Algorithm Core).
- **פירוט שלבי התרחיש**:
 1. המשתמש מבקש לחלץ מסר (על ידי העלאת קובץ או הפעלת קליק ימני על קובץ קיימים).
 2. ה UI-שולח את הקובץ הנגוע אל שכבת הלוגיקה.
 3. הלוגיקה מפעילה את מנוע החילוץ של האלגוריתם הרלוונטי.
 4. המנוע קורא את ה-Header של הקובץ, מחשב את אורך המסר, שואב את הביטים וממיר אותם לקידוד UTF-8.
 5. המנוע מחזיר את המחרוזת הגלויה לשכבת הלוגיקה.
 6. שכבת הלוגיקה מעבירה את המסר הגלוי ל-UI.
 7. ה UI-מציג את המסר הסודי בחלונית קופצת בפני המשתמש.



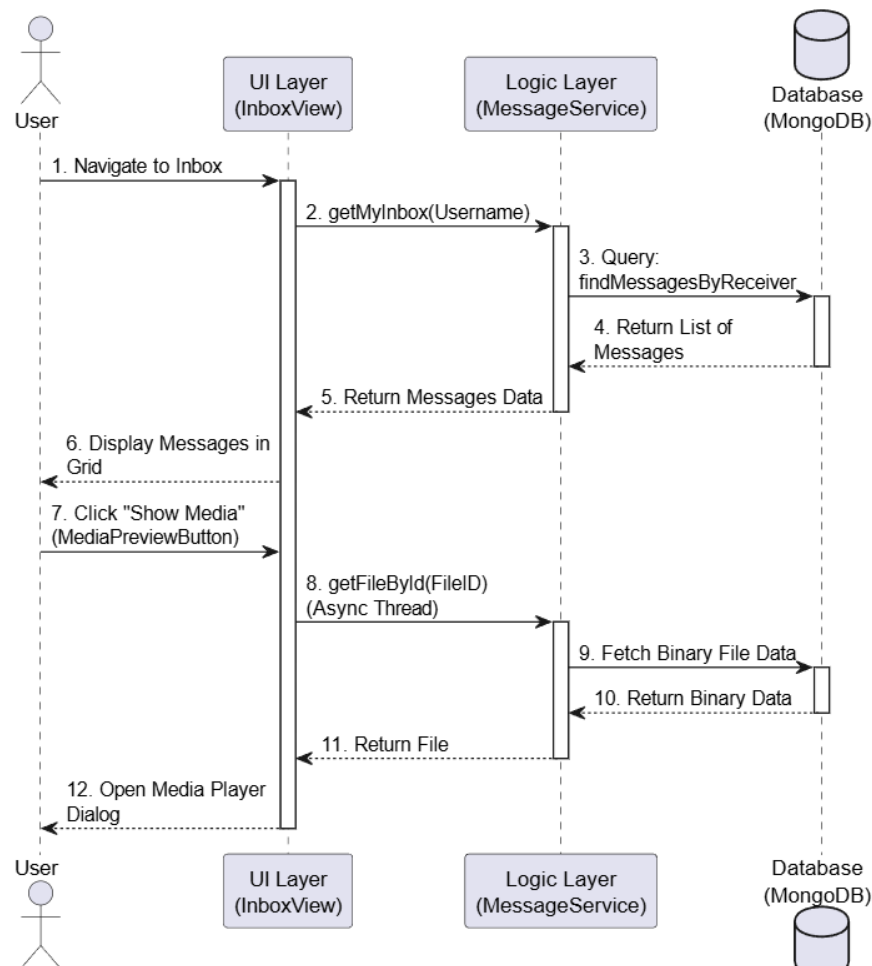
5. תרשים ניהול ושליחת דואר (Out Mail)

- **מטרת התרשים:** הצגת הלוגיקה המורכבת של שמירת קובץ מקושר להודעת טקסט במערכת הדואר.
- **האובייקטים המשתתפים:** משתמש הקצה (User), ממשק המשתמש (NewMessageDialog), שכבת הלוגיקה (MessageService), ומסד הנתונים (MongoDB).
- **פירוט שלבי התרחיש:**
 1. המשתמש בוחר נמען, מזין טקסט ומצרף קובץ נגוע לצורך שליחה.
 2. ה UI-מבקש משכבת הלוגיקה ליצור הודעה חדשה.
 3. הלוגיקה פונה תחילה לאוסף הקבצים (FilesRepository) כדי לשמור את הקובץ המצורף כמידע בינארי.
 4. מסד הנתונים מחזיר מזהה ייחודי עבור הקובץ שנוצר (FileID).
 5. הלוגיקה שומרת כעת את אובייקט ההודעה (Message) באוסף ההודעות, כאשר הוא מכיל בתוכו את ה-FileID.
 6. מסד הנתונים מאשר את שמירת ההודעה במלואה.
 7. שכבת הלוגיקה מחזירה אישור הצלחה ל-UI.
 8. ה UI-מציג התראת הצלחה למשתמש ומרענן את טבלת ההודעות.



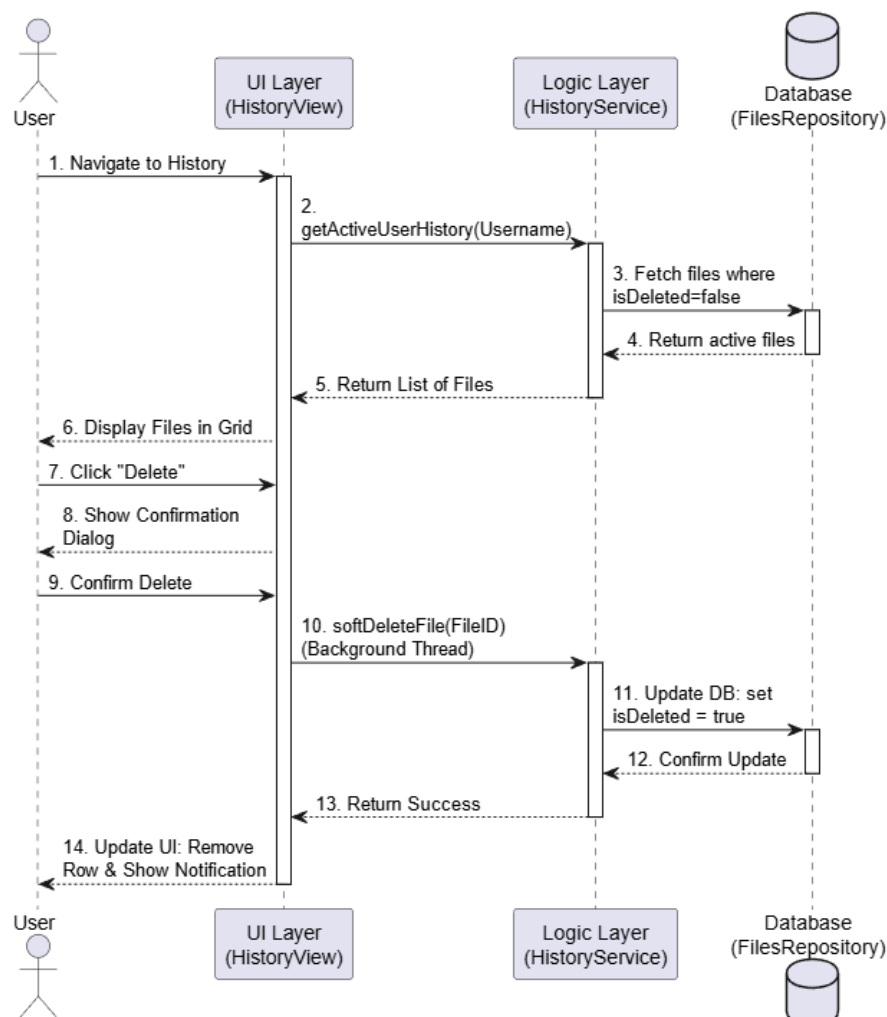
6. תרשים צפייה בדואר נכנס (View Inbox)

- **מטרת התרשים:** תיאור אופן שליפת ההודעות וניהול יעיל של משאבי שרת (Lazy Loading) בשליפת מדיה.
- **האובייקטים המשתתפים:** משתמש הקצה (User), ממשק המשתמש (InboxView), שכבת הלוגיקה (MessageService), ומסד הנתונים (MongoDB).
- **פירוט שלבי התרחיש:**
 1. המשתמש מנווט אל מסך הדואר הנכנס.
 2. ה UI-מבקש משכבת הלוגיקה לאחזר את ההודעות השייכות למשתמש.
 3. הלוגיקה מבצעת שאילתה למסד הנתונים לשליפת ההודעות.
 4. מסד הנתונים מחזיר את רשימת ההודעות.
 5. הלוגיקה מעבירה את הנתונים ל UI-.
 6. ה UI-מרנדר ומציג את ההודעות בטבלה.
 7. המשתמש לוחץ על הכפתור להצגת הקובץ המצורף ("הצג מדיה").
 8. רק בשלב זה, ה UI-יוזם תהליכון אסינכרוני (Thread) המבקש את הקובץ לפי ה FileID-.
 9. הלוגיקה פונה למסד הנתונים לשליפת הבינאריות (Binary Data).
 10. מסד הנתונים מחזיר את הקובץ במלואו.
 11. הלוגיקה מחזירה את המידע ל UI-.
 12. ה UI-פותח דיאלוג המכיל את נגן המדיה (Media Player) לחוויית צפייה נוחה.



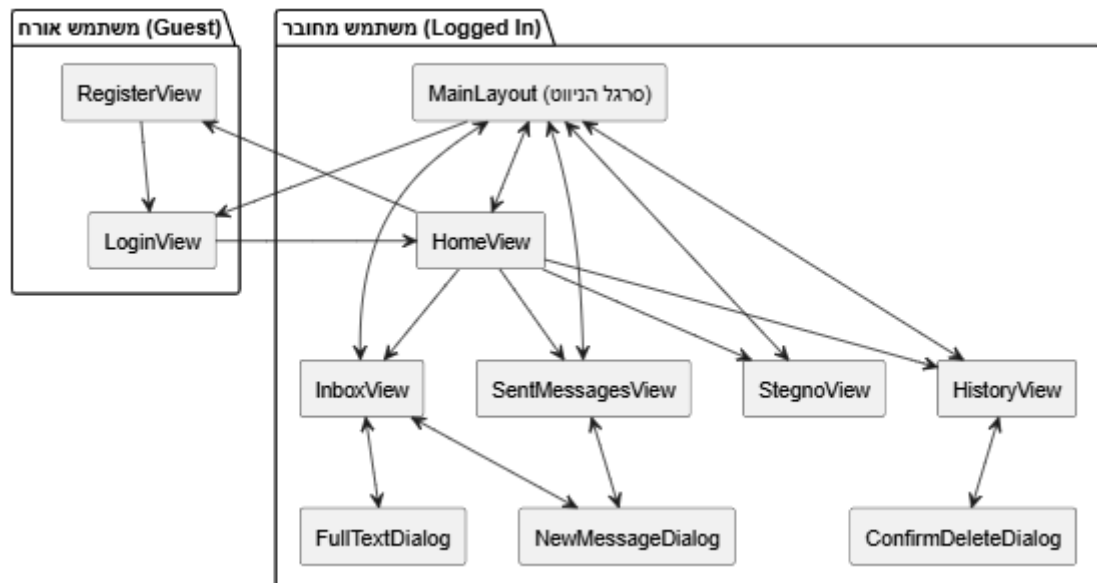
7. תרשים ניהול היסטוריה (View History)

- **מטרת התרשים:** הדגמת שליפת הנתונים במסך ההיסטוריה ואבטחת נתונים באמצעות "מחיקה לוגית".
- **האובייקטים המשתתפים:** משתמש הקצה, (User) ממשק המשתמש, (HistoryView) שכבת הלוגיקה, (HistoryService) ומועד הנתונים, (FilesRepository).
- **פירוט שלבי התרחיש:**
 1. המשתמש מנווט למסך ההיסטוריה.
 2. ה UI-מבקש מהלוגיקה את רשימת הקבצים ההיסטוריים.
 3. הלוגיקה מבצעת שאילתה ל DB-בבקשה לקבל רק קבצים שאינם מוגדרים כמחוקים (isDeleted=false).
 4. ה DB-מחזיר את הקבצים הפעילים בלבד, והלוגיקה מעבירה אותם ל UI-לתצוגה (שלבים 4-6).
 5. (שלב 7 בתרשים) המשתמש לוחץ על כפתור "מחיקה" עבור קובץ ספציפי.
 6. ה UI-מציג חלון אזהרה לאישור. עם אישור המשתמש, נשלחת הבקשה ללוגיקה (שלבים 8-10).
 7. הלוגיקה מעדכנת ב DB-את דגל המחיקה של הקובץ ל true-מבלי למחוק אותו פיזית (כדי לא לשבור קישורים בהודעות ישנות).
 8. ה DB-מאשר את הפעולה, והלוגיקה מעבירה את האישור ל UI.
 9. ה UI-בגישת (Optimistic UI מעלים את השורה מהטבלה ומציג למשתמש הודעת הצלחה על המחיקה.



11. תיאור מסכים וממשק משתמש

11.1 תרשים היררכיית המסכים



להלן תיאור מפורט המציג את הפונקציונליות המרכזית במערכת מנקודת מבטו של משתמש הקצה:

שחקנים (Actors)

- **משתמש (User):** משתמש הקצה אשר בא במגע עם המערכת כדי לבצע פעולות של תקשורת חשאית, הצפנה וחילוץ, וזאת לאחר שעבר תהליך אימות.
- **אורח (Guest):** משתמש שטרם הזדהה במערכת. גישתו מוגבלת למסך הבית המציג מידע כללי, ולביצוע פעולות של התחברות או הרשמה כסוכן חדש.

תהליכים ופעולות מרכזיות במערכת:

- **התחברות והרשמה (Authentication):** תהליך הכניסה למערכת מול מסד הנתונים ויצירת חשבון עבור סוכנים חדשים. תהליך זה מהווה תנאי סף לקבלת גישה לשאר הפונקציות המאובטחות.
- **הטמעת מסר סודי (Embed Message):** תהליך שבו המשתמש מזין מסר טקסטואלי סודי ובוחר קובץ מדיה גלוי (כגון תמונה או שמע). המערכת מטמיעה את המסר בתוך הקובץ באמצעות אלגוריתמים סטגנוגרפיים ויוצרת קובץ תקני למראה (Stego-object)
- **חילוץ מסר (Extract Message):** קבלת קובץ החשוד כמכיל מידע סמוי, פענוחו, והצגת המסר הסודי החבוי בתוכו על גבי המסך.
- **צפייה בהיסטוריה (View History):** גישה לתיעוד דינמי המציג את הקבצים והמסרים שעברו תהליכי עיבוד (הטמעה או חילוץ) במערכת לאורך זמן.
- **ניהול תכתובות פנימיות (Mailbox):** ניהול מערכת דואר מאובטחת, המאפשרת למשתמש לנווט בין הודעות שהתקבלו מסוכנים אחרים (דואר נכנס) לבין הודעות שנשלחו (דואר יוצא).

11.2 תיאור המסכים

1. מסך התחברות (Login)

Steganography System

Log in

Username •

A1

Password •

....



Log in

[Forgot password](#)

- **תפקיד המסך:** אימות זהות המשתמש מול מסד הנתונים ומתן הרשאת גישה למערכת רק למשתמשים רשומים.
- **רכיבי UI במסך ותפקידם:**
 - **תיבת טקסט (TextField) אימייל/שם משתמש:** "קליטת שם המשתמש או כתובת הדואר האלקטרוני."
 - **תיבת סיסמה (PasswordField) סיסמה:** "הזנת הסיסמה בצורה מוסתרת (נקודות)."
 - **כפתור (Button) התחבר:** "שליחת הנתונים לשרת לאימות וכניסה למערכת."
 - **קישור "הרשמה":** "מעבר למסך יצירת חשבון למשתמשים חדשים."
- **הודעות ודיאלוגים מוקפצים במסך זה:**
 1. **שגיאת פרטי התחברות שגויים:**

❗ **Incorrect username or password**
Check that you have entered the correct
username and password and try again.

- **תיאור:** הודעה שקופצת כאשר המשתמש מזין שם משתמש שלא קיים או סיסמה לא נכונה. מונעת כניסה למערכת ומבקשת לנסות שנית.

2. שגיאת חוסר בהזנת פרטים (שדות ריקים):

Username •

Username is required

Password •

Password is required

תיאור: אזהרה שמופיעה כאשר המשתמש לוחץ "התחבר" אך השאיר את שדה האימייל או הסיסמה ריקים.

2. מסך הרשמה (Register)

יצירת חשבון חדש

שם משתמש •

סיסמה •

אימות סיסמה •

הרשם למערכת

[כבר יש לך חשבון? התחבר כאן](#)

- **תפקיד המסך:** קליטת משתמשים חדשים, ביצוע בדיקות תקינות (וולידציה) על הנתונים, ושמירת המשתמש ב-MongoDB.

• רכיבי UI במסך ותפקידם:

- **תיבות טקסט (TextField):** קליטת נתונים אישיים (שם מלא, אימייל).
- **תיבת סיסמה (PasswordField):** קליטת הסיסמה החדשה.
- **תיבת אימות סיסמה (PasswordField):** הקלדה חוזרת של הסיסמה כדי למנוע שגיאות.
- **כפתור (Button) צור חשבון:** הפעלת בדיקות התקינות ושמירה בשרת.
- **כפתור חזור להתחברות:** ניווט מהיר למסך הקודם.

- הודעות ודיאלוגים מוקפצים במסך זה:

1. שגיאת סיסמאות לא תואמות:

הסיסמאות אינן תואמות

הרשם למערכת

כבר יש לך חשבון? התחבר כאן

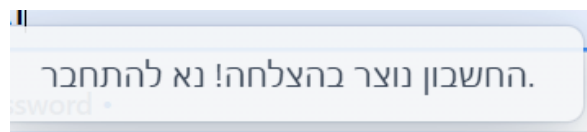
▪ **תיאור:** קופצת כאשר המשתמש מזין סיסמה שונה בשדה "סיסמה" ובשדה "אימות סיסמה".

2. שגיאת "שם המשתמש כבר קיים במערכת":

שם המשתמש כבר קיים במערכת, אנא בחר שם אחר

▪ **תיאור:** הודעה שקופצת אם משתמש מנסה להירשם עם כתובת אימייל שכבר רשומה ב.DB-

3. הודעת "הרשמה בוצעה בהצלחה":



▪ **תיאור:** חיווי למשתמש שהנתונים נשמרו תקין והוא מועבר למסך ההתחברות.

3. מסך תפריט ניווט ודף הבית (Main Layout & Home)

- **תפקיד המסך:** מעטפת הניווט המרכזית של המערכת. התפריט הוא דינמי ומתאים את עצמו אוטומטית הוא מציג אפשרויות שונות לחלוטין עבור אורח שטרם התחבר, לעומת משתמש רשום שכבר ביצע כניסה למערכת.

א. מצב אורח (Guest Menu)



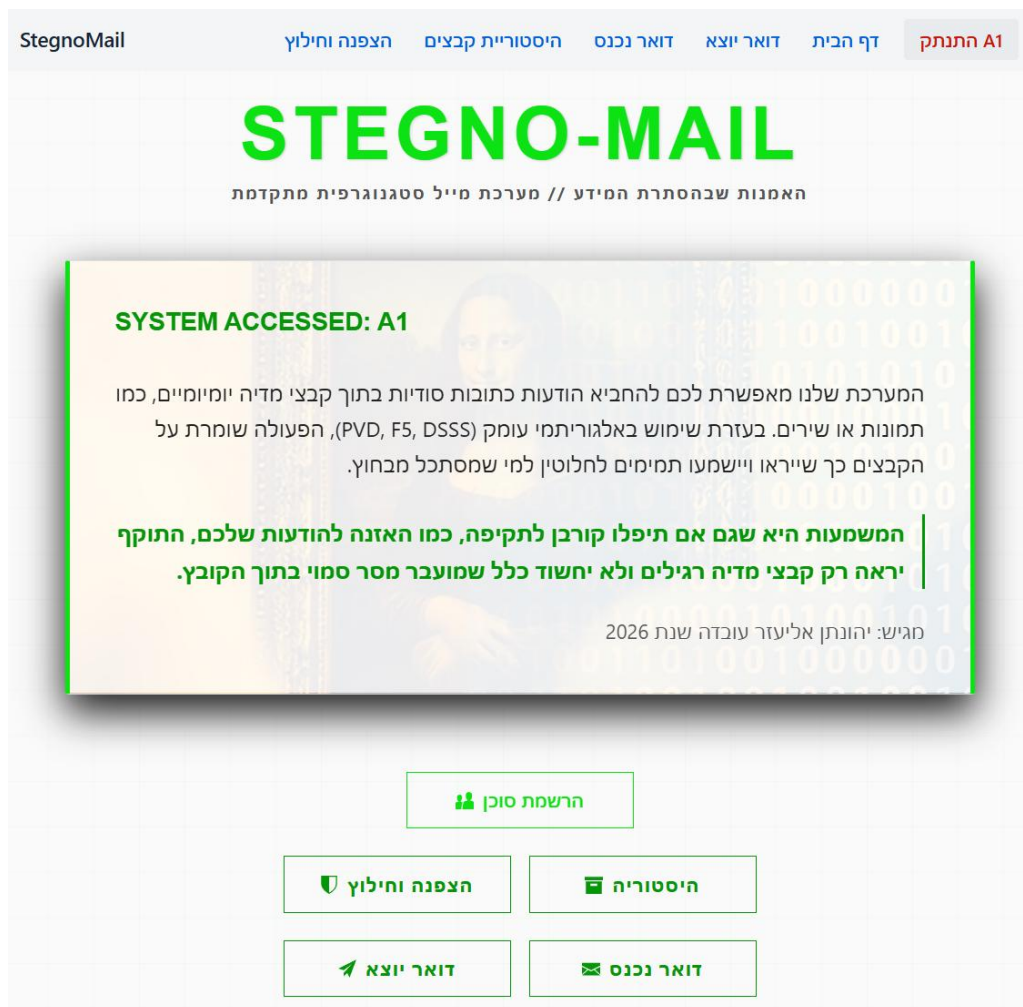
• רכיבי UI בתפריט זה ותפקידם:

- **כפתור ניווט (Button) התחברות:** כפתור המטמיע בתוכו קישור פנימי, נועד להעביר את האורח למסך ההזדהות. (Login)
- **כפתור ניווט (Button) הרשמה:** כפתור המטמיע בתוכו קישור פנימי, נועד להעביר אורח חדש למסך יצירת חשבון. (Register)

• הודעות ודיאלוגים מוקפצים במצב זה:

- במצב זה לרוב אין דיאלוג קופץ, אך אם האורח מנסה "לדלג" ולהקליד ב URL-כתובת של מסך פנימי מוגן (כמו (inbox/המערכת מזהה שאין לו Session פעיל ומעבירה אותו אוטומטית חזרה למסך ההתחברות.

ב. מצב משתמש מחובר (Authenticated User Menu)



• רכיבי UI בתפריט זה ותפקידם:

- **תווית שם משתמש (Label):** אזור טקסט המציג את השם של המשתמש המחובר כעת
- **כפתור ניווט (Button) דף הבית:** כפתור מעבר מהיר למסך הראשי של המערכת.
- **כפתור ניווט (Button) דואר נכנס:** כפתור מעבר לתיבת ההודעות לטובת צפייה בקבצים ופענוח מסרים סודיים.
- **כפתור ניווט (Button) סטגנוגרפיה:** כפתור מעבר למסך העבודה שבו מעלים קובץ מדיה, בוחרים אלגוריתם ומסתירים את המידע.
- **כפתור ניווט (Button) הודעות יוצאות:** כפתור מעבר לטבלה המציגה אילו קבצים וצפנים נשלחו לנמענים השונים.
- **כפתור ניווט (Button) היסטוריית פעולות:** כפתור מעבר ליומן התיעוד והמעקב של המערכת.
- **כפתור פעולה (Button) התנתק:** כפתור ביצוע המוחק את פרטי המשתמש מהסשן (Session) באופן מאובטח ומחזיר את האפליקציה לתצוגת אורח.

4. מסך תיבת דואר נכנס(Inbox)

StegnoMail

הצפנה וחילוץ היסטוריית קבצים דואר נכנס דואר יוצא דף הבית A1 התנתק

תיבת דואר נכנס

הודעה חדשה

מאת	תוכן גלוי	קובץ מצורף	מחיקה
A1	שלום מה קורה	הצג מדיה	מחיקה
A5	האירופאית, הנקראת...	-	מחיקה
A5	GFGDFGDFGFD	-	מחיקה
A5	שלום מה קורה	הצג מדיה	מחיקה

- תפקיד המסך: הצגת ההודעות ואז הקבצים שהתקבלו, צפייה מקדימה בהם ופענוח המסר המוסתר.
- רכיבי UI במסך ותפקידם:
 - טבלת נתונים (Grid): הצגת ההודעות לפי שולח, תאריך וסוג אלגוריתם/קובץ.
 - כפתור הצג מדיה (Media Preview): כפתור לפתיחת חלונית המציגה את הקובץ.
 - כפתור מחיקה: כפתור למחיקה של ההודעה.
- הודעות ודיאלוגים מוקפצים במסך זה:
 1. דיאלוג צפייה במדיה (Media Preview Dialog)

יפורט בהמשך

5. מסך יצירת הודעה (Stegno / New Message)

- **תפקיד המסך:** ביצוע פעולת ההצפנה וההסתרה – העלאת קובץ, כתיבת מסר, ושליחה.
- **רכיבי UI במסך ותפקידם:**
 - **רכיב העלאה (Upload):** גרירה או בחירה של קובץ תמונה/שמע המשמש כ-Cover Object-תיבה נפתחת (ComboBox) **לנמען:** בחירת המשתמש שאליו תשלח ההודעה.
 - **תיבת טקסט (TextArea):** הזנת המסר הסודי שאותו רוצים להסתיר.
 - **כפתורי רדיו / רשימה (RadioButtonGroup):** בחירת אלגוריתם ההסתרה הרצוי.
 - **כפתור "שלח/הצפן":** ביצוע תהליך ההסתרה ושמירת ההודעה למוען.
- **הודעות ודיאלוגים מוקפצים במסך זה:**

1. שגיאת "חסרים נתונים לשליחה:"

כתיבת הודעה חדשה

• אל (שם משתמש)

חובה להזין נמען

הודעה גלויה

ראר

חובה להזין את שם המשתמש של הנמען

Upload File...



Drop file here

• **תיאור:** קופצת אם המשתמש לוחץ על "שלח" מבלי שבחר נמען.

2. הודעת "הודעה נשלחה בהצלחה:"

ההודעה נשלחה בהצלחה

• **תיאור:** חיווי המאשר שהמסר הוצפן בתוך הקובץ ונשלח לנמען בהצלחה מופיע בצד ימין למטה של המסך.

3. הודעת המחיקה:

כאשר משתמש לחץ על מחק.

...מחק את ההודעה מהתיבה

6. מסך הודעות יוצאות / היסטוריה (Sent Messages & History)

StegnoMail	הצפנה וחילוץ	היסטוריית קבצים	דואר נכנס	דואר יוצא	דף הבית	A1 התנתק
הודעות שנשלחו						
הודעה חדשה						
אל	תוכן גלוי	קובץ מצורף	מחיקה			
A1	שלום מה קורה	הצג מדיה	מחיקה			
1111	11111	הצג מדיה	מחיקה			
LEVI1678	MY MSG	הצג מדיה	מחיקה			
LEVI	תראה איזה קטע	הצג מדיה	מחיקה			
יעקב	?מה נשמע	-	מחיקה			

- תפקיד המסך: הצגת כל ההודעות והקבצים שהמשתמש הנוכחי שלח למשתמשים אחרים.
- רכיבי UI במסך ותפקידם:
 - כפתור (Button) הודעה חדשה: פתיחת חלון לשליחת הודעה.
 - טבלת נתונים (Grid): דומה לתיבת הדואר הנכנס, אך במקום שולח, מוצגת עמודת "אל" (Recipient) המציגה את שם הנמען. הטבלה כוללת גם עמודת תוכן גלוי, כפתור תצוגת מדיה ("הצג מדיה"), וכפתור מחיקה.
- הודעות ודיאלוגים מוקפצים במסך זה:
 - זהים למסך ה Inbox-חלונית צפייה בטקסט ארוך, והתראות קופצות בעת תהליך מחיקת ההודעה ממסד הנתונים.

7. מסך היסטוריית פעולות וקבצים (History)

מחיקה	תצוגה	סוג קובץ	תאריך ושעה
מחיקה	הצג מדיה	audio/wav	Tue May 19 00:21:36 IDT 2026
מחיקה	הצג מדיה	audio/wav	Tue May 19 00:27:21 IDT 2026
מחיקה	הצג מדיה	image/png	Tue May 19 00:27:48 IDT 2026
מחיקה	הצג מדיה	image/avif	Tue May 19 00:27:51 IDT 2026
מחיקה	הצג מדיה	image/jpeg	Wed May 20 14:06:48 IDT 2026
מחיקה	הצג מדיה	image/jpeg	Sun May 24 21:48:39 IDT 2026
מחיקה	הצג מדיה	image/jpeg	Sun May 24 21:48:57 IDT 2026
מחיקה	הצג מדיה	image/jpeg	Sun May 24 21:51:12 IDT 2026

- תפקיד המסך: מעקב אחר כל קובצי התמונות והשמע שהשתמש העלה או עיבד במערכת, עם אפשרות לתצוגה מקדימה ולביצוע מחיקה לוגית. (Soft Delete)

- רכיבי UI במסך ותפקידם:

○ טבלת קבצים (Grid)

- עמודת תאריך ושעה מציגה מתי הקובץ עובד עמודה הניתנת למיון
- עמודת סוג קובץ: מציגה את ה Type למשל image/png
- עמודת הצגת מדיה (MediaPreviewButton) כפתור המאפשר צפייה בתמונה או נגינת השמע.
- כפתור מחיקה (Button) כפתור אדום להתחלת תהליך מחיקת הקובץ.

- הודעות ודיאלוגים מוקפצים במסך זה:

1. דיאלוג אישור מחיקה (Confirm Dialog)

אישור מחיקה

האם אתה בטוח שברצונך למחוק קובץ זה מההיסטוריה? (פעולה זו תסתיר את הקובץ)

ביטול
מחק

- תיאור: חלונית הנפתחת לאחר לחיצה על מחיקה. מכילה את הטקסט "האם אתה בטוח שברצונך למחוק קובץ זה מההיסטוריה?", וכוללת כפתור "ביטול" וכפתור "מחק".

2. הודעות סטטוס מחיקה (Notification)

- הודעה אפורה: ממתין לאישור מחיקה ממסד הנתונים...

...ממתין לאישור מחיקה ממסד הנתונים

- הודעת הצלחה ירוקה: "הקובץ נמחק בהצלחה מהמערכת!"

הקובץ נמחק בהצלחה מהמערכת

- הודעת שגיאה אדומה: "שגיאה: המחיקה במסד הנתונים נכשלה ולא בוצעה."
(במידה ויש בעיית שרת).

8. מסך הצפנה וחילוץ מסרים (StegnoView)

The screenshot shows the StegnoMail web interface with a modal window titled "StegnoView". The modal contains the following elements:

- A header bar with the StegnoMail logo and navigation links: "הצפנה וחילוץ", "היסטוריית קבצים", "דואר נכנס", "דואר יוצא", "דף הבית", and "A1 התנתק".
- A central modal window with a light gray background and a white border. It contains:
 - An "Upload File..." button and a "Drop file here" area with an upward arrow icon.
 - A "Message to embed" section with a text input field containing the placeholder "Enter your secret message here...".
 - Text indicating the message size: "מגבלת מסר סודי: עד 65,535 תווים (65-KB)".
 - Two buttons: "Embed & save To DB" (highlighted in blue) and "Extract msg".
 - Footer text: "2000 ארוזלוציה מרבית: PNG, JPG, WAV | פורמטים נתמכים | 15 MB מקסימום".

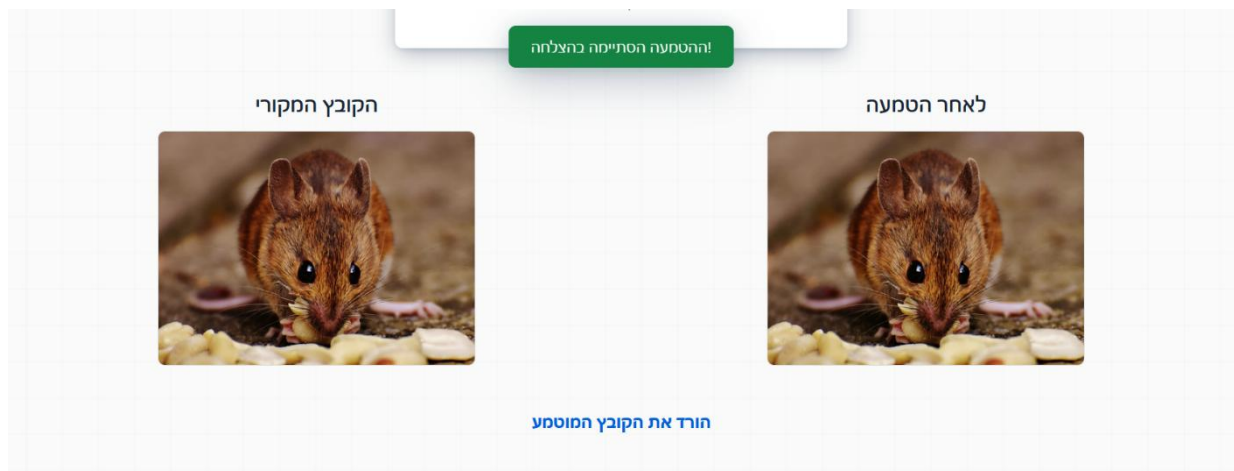
תפקיד המסך: ביצוע פעולות הליבה הסטגנוגרפיות – הטמעת מסר טקסטואלי סודי בתוך קובץ מדיה תמים, או פענוח וחילוץ של מסר מתוך קובץ נגוע. המסך מהווה את תחנת העבודה המרכזית של המערכת.

רכיבי UI במסך ותפקידם:

- **רכיב העלאה (Upload) אזור גרירה:** קליטת קובץ מדיה (PNG, JPG, WAV) עד לנפח של 15MB, תוך שמירתו כקובץ זמני בשרת למניעת עומס.
- **שדה קלט (TextField) "הודעה":** הזנת ההודעה החשאית שאותה רוצים להסתיר בקובץ.
- **כפתור Embed & (Button) save To DB:** הפעלת אלגוריתם ההטמעה, ייצור הקובץ הנגוע ושמירתו במסד הנתונים.
- **כפתור Extract msg (Button):** הפעלת אלגוריתם החילוץ על קובץ שהועלה, לצורך שחזור המסר הסודי.

התנהגות דינמית ותצוגת תוצאות (לאחר ביצוע פעולה):

מכיוון שהמערכת פועלת כ-Single Page Application, הממשק מתעדכן בזמן אמת בהתאם לפעולה שבוצעה, ומוסיף רכיבים חדשים למסך:



א. לאחר פעולת הטמעה מוצלחת:

- **אזור השוואה (HorizontalLayout) תצוגה מקדימה:** רכיב הנוצר דינמית ומציג את קובץ המקור אל מול הקובץ המוטמע זה לצד זה, לצורך בחינה חזותית ואימות שלא נוצרו עיוותים (Artifacts) במדיה.
- **קישור הורדה (Anchor):** אלמנט המופק אוטומטית ומאפשר את הורדת הקובץ המוטמע ישירות למחשב המשתמש.

ב. לאחר פעולת פענוח וחילוץ (Extract) מוצלחת:

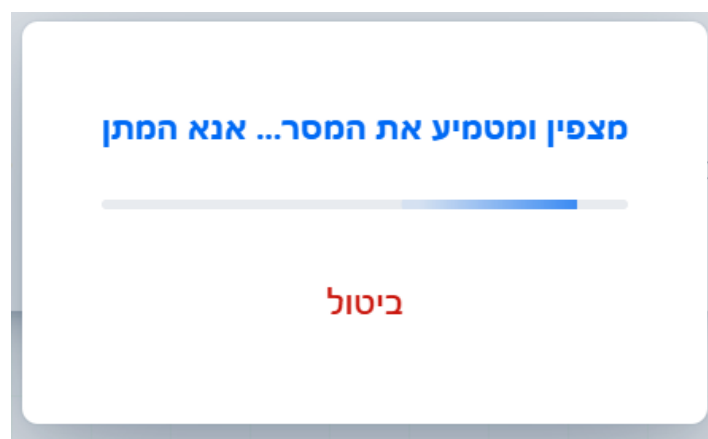
המסר הסודי שהתגלה:

אני מסתיר מידע

- **אזור חשיפת המסר (Message Reveal Container):** רכיב טקסטואלי מודגש (כותרת וטקסט בצבע הראשי של המערכת) המרונדר על המסך רק לאחר שאלגוריתם החילוץ זיהה ופענח בהצלחה את המידע הסמוי מתוך הקובץ שהועלה.

הודעות ודיאלוגים מוקפצים במסך זה:

1. חסם טעינה מודאלי (Spinner Dialog):



תיאור: חלונית חוסמת המופיעה בזמן שהמערכת מעבדת את הקובץ ברקע (תהליך אסינכרוני המצפין או מפענח). החלונית כוללת מד התקדמות וכפתור לביטול אקטיבי של הפעולה במידת הצורך.

2. התראות מערכת (Notifications):

תיאור: המערכת משתמשת במנגנון ההתראות של Vaadin כדי להציג חיווי מהיר למשתמש בתחתית או במרכז המסך, ללא קטיעת רצף העבודה. ההתראות המרכזיות כוללות:

- **"חסרים פרטים: אנא העלה קובץ והזן הודעה":** שגיאת וולידציה (באדום) המונעת הפעלת הטמעה טרם הוזנו נתוני חובה.

חסרים פרטים: אנא העלה קובץ והזן הודעה

- **"נא להעלות קובץ שמכיל מסר מוחבא":** שגיאה (באדום) המונעת ניסיון לחלץ מסר ללא קובץ מקור.

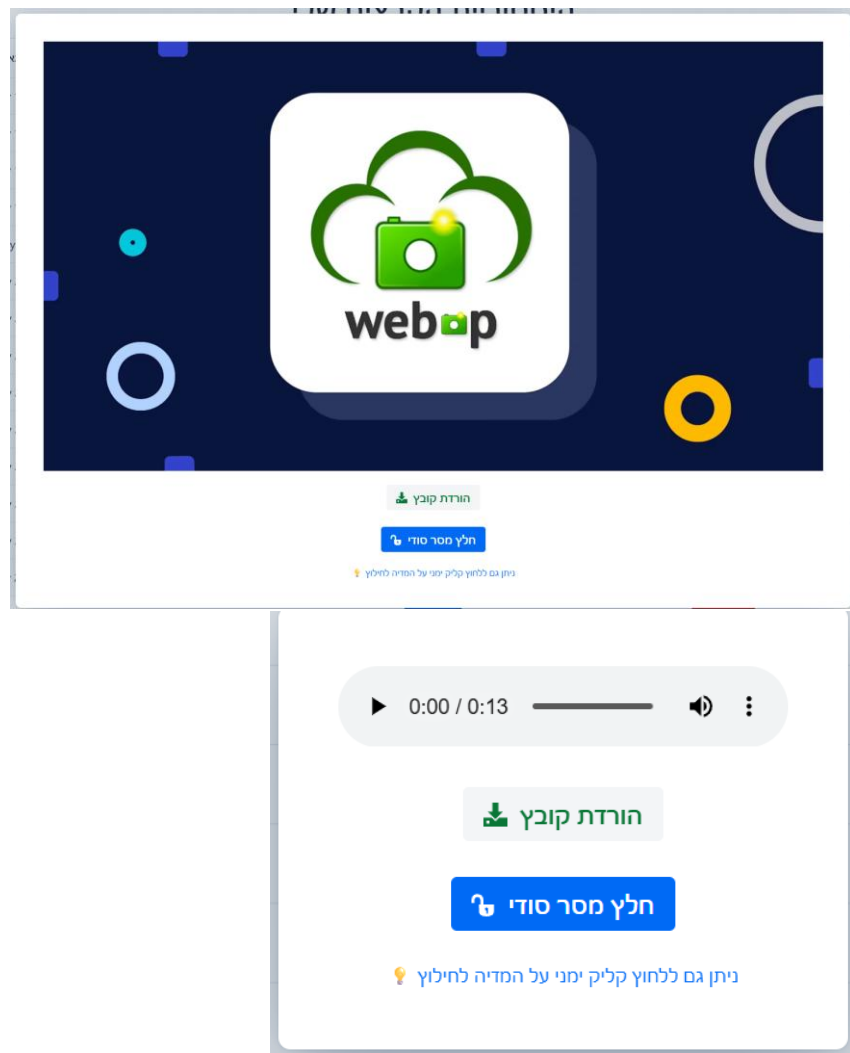
נא להעלות קובץ שמכיל מסר מוחבא

- **"ההטמעה/חילוץ הושלמה בהצלחה!":** חיווי חיובי (בירוק) המאשר את סיום התהליך התקין.

!ההטמעה הסתיימה בהצלחה!

!חילוץ הושלם בהצלחה

9. רכיב תצוגת מדיה (Media Preview)



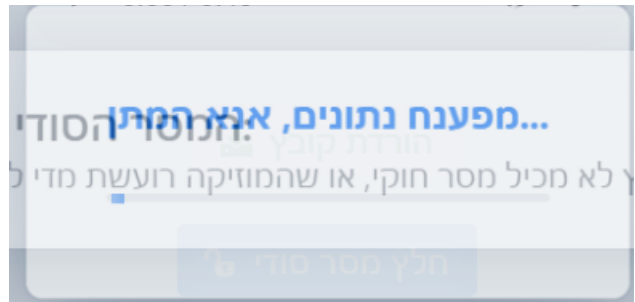
תפקיד הרכיב : ייצוג, הצגה והשמעה של קבצי המקור והקבצים המוטמעים ישירות בדפדפן (באמצעות קידוד Base64), ללא צורך בשמירתם הפיזית בשרת. רכיב זה נפתח כחלונית צפה מתוך טבלאות הנתונים במערכת, ומשמש גם כתחנת עבודה מקומית לחילוץ מסרים מוסתרים ולהורדת הקבצים.

רכיבי UI במסך ותפקידם:

- **תגית תמונה (Image) / נגן שמע (HTML5 Audio) :** אזור התצוגה הראשי המרנדר באופן דינמי את המדיה (תמונה או נגן קול עם פקדי שליטה) בהתאם לסוג הקובץ שנבחר.
- **כפתור (Button) הורדת קובץ :** לחצן הממוקם בתחתית החלונית ומשמש להורדה מיידית של קובץ המדיה המוצג ישירות למחשב המשתמש, באמצעות הזרקת הנתונים לתגית עוגן (Anchor) של HTML.
- **כפתור (Button) חלץ מסר סודי :** לחצן המופיע אך ורק עבור קבצים המוגדרים כמוטמעים. הפעלתו קוראת לשירות הסטגנוגרפיה ומתחילה את אלגוריתם פענוח וחילוץ המסר מהמדיה.
- **תפריט הקשר (Context Menu) לחילוץ :** תפריט נפתח המופיע בלחיצת קליק-ימני על המדיה עצמה, ומהווה קיצור דרך להפעלת פעולת חילוץ המסר. עבוד קבצים ללא מסר, התפריט מנוטרל ומציג חיווי "קובץ גלוי".

הודעות ודיאלוגים מוקפצים במסך זה:

1. חסם טעינה לחילוץ (Extraction Spinner):



תיאור : דיאלוג חוסם המציג מד התקדמות מונפש והודעה "מפענח נתונים, אנא המתן...", אשר מונע מהמשתמש לבצע פעולות נוספות בחלונית עד שפעולת הפענוח ברקע תושלם.

2. דיאלוג חשיפת מסר (Extraction Result Dialog):



תיאור : חלונית הנפתחת באופן אוטומטי לאחר סיום מוצלח של תהליך הפענוח, ומציגה למשתמש את המסר הסודי שחולץ מקובץ המדיה.

3. התראות מערכת קופצות (Toast Notifications):



תיאור : הודעות חיווי קצרות המופיעות למספר שניות. מופעלות כדי להתריע על שגיאות, כגון "לא נמצא מסר או שהפענוח נכשל" במקרה של ניסיון חילוץ לא תקין, או "הפעולה בוטלה על ידי המשתמש" בעת עצירת הטעינה.

- **נגן שמע דינמי (HTML5 Audio Element) :** רכיב שמע הנוצר רק במידה והקובץ הוא מסוג קול (audio/wav). הרכיב מזריק ל-DOM אלמנט HTML5 טבעי עם פקדי ניגון, עצירה, ציר זמן ושליטה

12. תיאור התוכנה

12.1 סביבת הפיתוח של התוכנה

12.1.1 מערכת הפעלה

המערכת פותחה, נבדקה והורצה על גבי מערכת הפעלה Windows 11 תוך שימוש במערכת ניהול התלויות Maven המבטיחה תאימות (Cross-Platform) לכל מערכת הפעלה המריצה את סביבת ה JVM.

12.1.2 סביבות הפיתוח (IDEs)

VS Code (גרסה 1.107.1) : סביבת פיתוח קלה ונוחה המותאמת לעבודה עם שפת Java וניהול תלויות באמצעות Maven על ידי תוספים ייעודיים. הסביבה מאפשרת כתיבת קוד יעילה ואיתור שגיאות (Debugging) בזמן אמת, כולל אפשרות לעצור את ריצת התוכנה ולבחון את הערכים השמורים בזיכרון. יכולת זו חיונית למעקב מדויק אחר עיבוד הנתונים והמניפולציות הבינאריות המבוצעות על הקבצים במערכת הסטגוגרפיה. בנוסף, הסביבה מתממשקת באופן מלא ל Spring Boot, מה שמאפשר להפעיל ולנהל את שרת האפליקציה ישירות מתוך כלי הפיתוח.

12.1.3 מסגרות לפיתוח התוכנה (Frameworks)

Spring Boot (גרסה 3.5.9) : משמשת כמסגרת העבודה המרכזית לניהול חיי האפליקציה, הלוגיקה העסקית והתקשורת בצד השרת (Backend). המסגרת כוללת שרת אפליקציה מובנה מסוג Apache Tomcat האחראי לניהול בקשות HTTP ותקשורת בין הלקוח לשרת ללא צורך בהגדרת שרת חיצוני. השימוש ב Spring Boot מאפשר הפרדה ברורה ואלגנטית בין שכבת התקשורת (Controllers) לשכבת הליבה האלגוריתמית (Services), ותומך בניהול תהליכים מתקדם להרצת פעולות אסינכרוניות.

Vaadin (גרסה 24.9.9) : מסגרת עבודה (Web Framework) לבניית ממשקי משתמש עשירים, דינמיים ומודרניים (Frontend). המסגרת מיועדת ומותאמת במיוחד למפתחי Java ומאפשרת פיתוח Full-Stack בשפה אחידה: בניית רכיבי תצוגה ולוגיקה בצד הלקוח באמצעות קוד Java בלבד, ללא צורך בכתיבה ישירה של HTML או JavaScript. גישה זו מבטיחה רצף טכנולוגי מלא, בטיחות טיפוסים (Type-Safety) ומצמצמת משמעותית את הסיכונים לבעיות המרה (Encoding) בעת העברת המידע בין השרת ללקוח.

12.1.4 שפות תכנות

Java 21 : שפת תכנות עילית מונחית עצמים (OOP) המשמשת כשפת הליבה של הפרויקט. השפה מאפשרת כתיבה ברמת הפשטה גבוהה, תוך שמירה על שליטה קפדנית במבני נתונים, במשאבי זיכרון ובזרימת המידע. התמיכה המובנית של Java בעבודה עם טיפוסים פרימיטיביים (כגון מערכי Bytes ופעולות ברמת הביט Bitwise), מאפשרת עבודה מדויקת עם נתונים בינאריים החיונית לאלגוריתמים הסטגוגרפיים.

כמו כן, סביבת ה JVM מספקת יציבות, ניהול זיכרון אוטומטי (Garbage Collection) ותמיכה בעיבוד מקבילי (Multithreading), מה שהופך אותה לאידיאלית לפיתוח מערכת הדורשת אמינות ודיוק חישובי.

12.1.5 ספריות/מערכות/מנגנוני עזר נוספים

ספריות:

המערכת עושה שימוש בספריות חיצוניות ייעודיות (Dependencies) המנוהלות באמצעות Maven, לצורך ייעול תהליכי התקשורת, ניהול הנתונים והעיבוד:

- ספריית OkHttp3 (com.squareup.okhttp3): ספריית תקשורת HTTP/HTTP2 מתקדמת ויעילה מבית Square. במערכת זו, הספרייה משמשת כלקוח ה-HTTP המרכזי להוצאת קריאות וניהול תקשורת מול ה-API החיצוני של CloudConvert לשם המרת קבצים. הספרייה נבחרה בזכות יכולתה לנהל ביעילות בקשות מורכבות, כגון העלאת מערכי בייטים (Multipart Form Data) של קבצי המדיה ישירות מהזיכרון, ניהול זמני המתנה ארוכים (Timeouts של 60 שניות המותאמים להמרת קבצים כבדים), וביצוע דגימה מחזורית (Polling) עד לקבלת התוצר המעובד.
- ספריית JSON in Java (org.json): ספרייה קלת-משקל לפיענוח ובנייה של אובייקטי JSON. הספרייה משמשת לבניית בקשות ה-Payload (הגדרות פורמט היעד ופרמטרי ההמרה) הנשלחות לשרתי CloudConvert, ולפיענוח (Deserialization) של התשובות (Responses) המתקבלות מהשרת. בעזרתה המערכת מחלצת ביעילות נתונים חיוניים מתוך עצים מסועפים של JSON, כגון מזהה משימה (Job ID), כתובות העלאה והורדה, וסטטוס העיבוד.

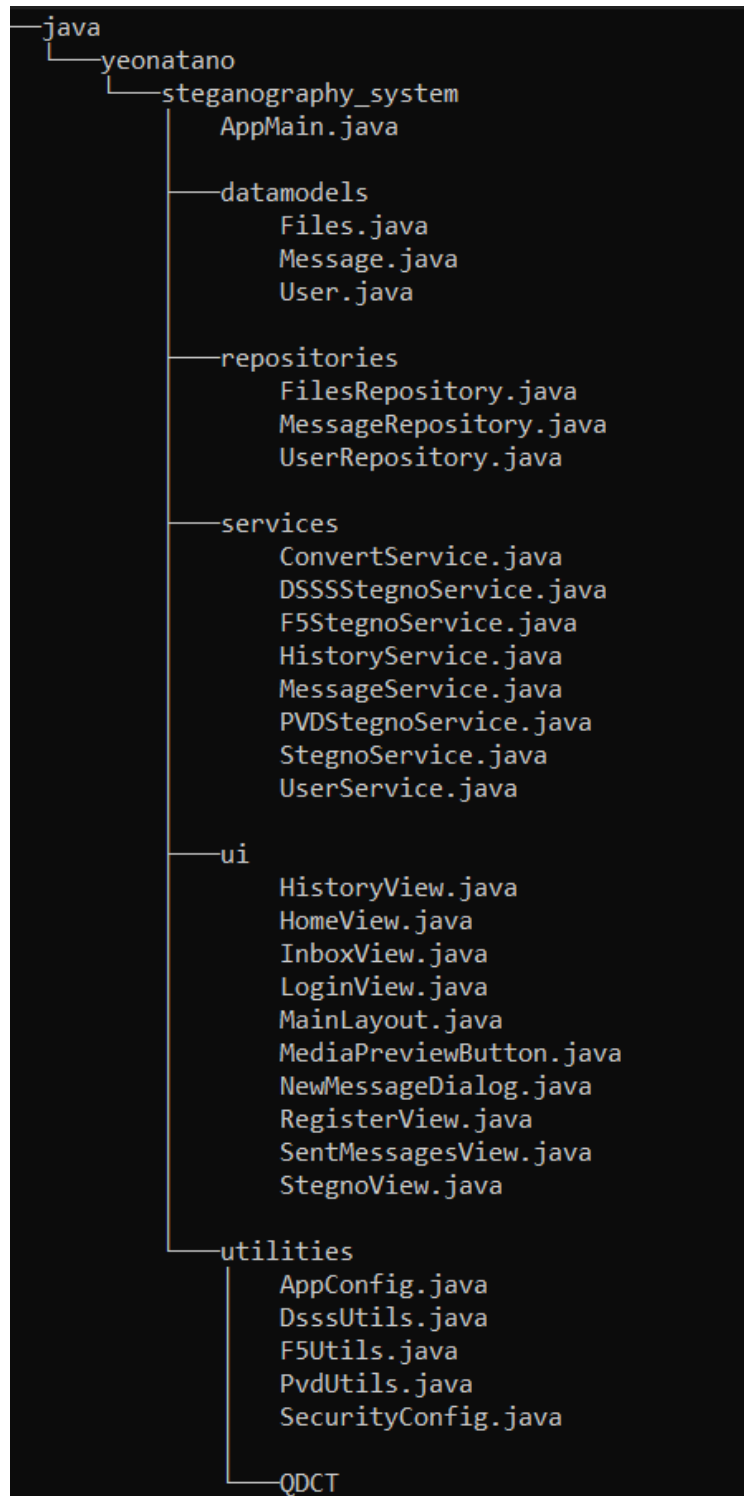
מודלים נוספים:

- Spring Data MongoDB (spring-boot-starter-data-mongodb): מודול רשמי של Spring המספק אינטגרציה מלאה מול מסד הנתונים מבוסס המסמכים (NoSQL) של MongoDB. מנגנון זה משמש כשכבת ה-Data Access Layer של המערכת (שכבת ה-Repositories), ומאפשר ביצוע יעיל של פעולות CRUD על אוספי המשתמשים, ההודעות וההיסטוריה, כולל שמירה ושליפה של קבצים בינאריים כבדים (Binary Data) ישירות מהענן.
- ספריות הליבה של Java לעיבוד מדיה (javax.imageio ו-javax.sound.sampled): המערכת אינה מסתמכת על ספריות צד-שלישי כשחורות לביצוע ההצפנה עצמה, אלא משתמשת בספריות הליבה הסטנדרטיות של Java לגישה לנתונים הגולמיים. ספריות אלו מאפשרות גישה ישירה לנתוני פיקסלים, חישובי ערוצי צבע (ARGB), ופירוק דגימות קול בתצורת PCM (עבור אלגוריתמי ה-PVD, ה-F5 וה-DSSS), לשם ביצוע מניפולציות בינאריות (Bitwise) ברמת ה-Low-Level על הקבצים.

12.2 תיאור מבנה הפרויקט והמחלקות

12.2.1 עץ המודולים של הפרויקט

עץ המודולים מציג במבט-על את החלוקה המבנית והפונקציונלית של המערכת. כדי להבטיח קוד נקי, מודולרי וקל לתחזוקה, המערכת תוכננה ופותחה על בסיס ארכיטקטורת שלוש שכבות (3-Tier Architecture) גישה זו מפרידה באופן מוחלט בין ממשק המשתמש, הלוגיקה והאלגוריתמיקה, לבין ניהול הנתונים.



1. **תיקיית ui שכבת התצוגה (Presentation Layer)** השכבה העליונה והחשופה למשתמשי הקצה. תיקייה זו מאחסנת את כל דפי האפליקציה, מעטפת הניווט, לוח הבקרה והדיאלוגים המודאליים. שכבה זו אינה מכילה לוגיקה תפעולית מורכבת או גישה למסד הנתונים, וממומשת כולה בצד השרת באמצעות ספריית Vaadin.

2. **תיקיית services ותיקיית utilities שכבת הלוגיקה העסקית (Logic Layer) (Business)** שכבת האמצע המהווה את "המוח" של המערכת. תיקיית services אחראית על ניהול התהליכים העסקיים, ניתוב המידע ואימות ההרשאות. תיקיית utilities משלימה אותה ומבודדת בתוכה את המנועים האלגוריתמיים הכבדים של הפרויקט ואת החישובים המתמטיים הטהורים לביצוע סטגנוגרפיה.

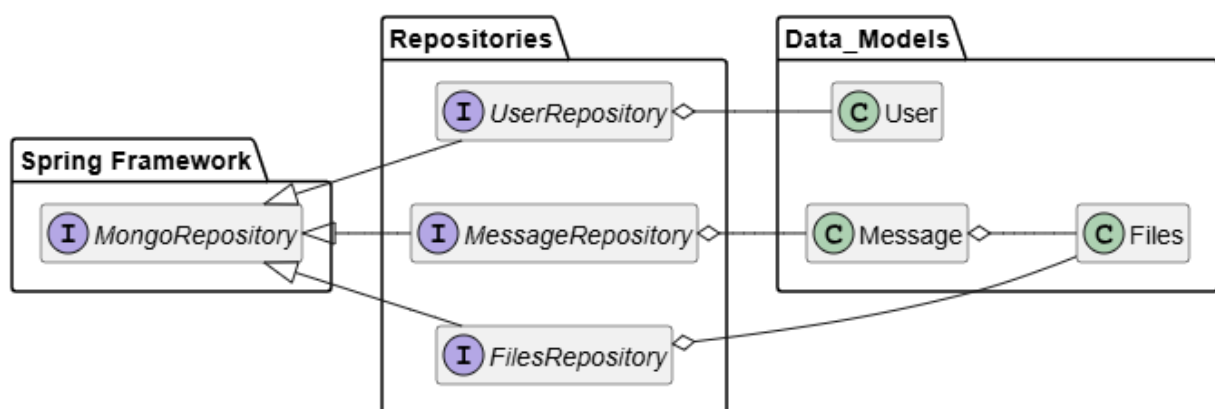
3. **תיקיית datamodels ותיקיית repositories שכבת הנתונים (Access Layer) (Data)** השכבה התחתונה ביותר במערכת. תיקיית datamodels מגדירה את מבנה הישויות של הפרויקט (משתמשים, הודעות, קבצי ארכיון), ותיקיית repositories מאחסנת את ממשקי התקשורת האחראים לביצוע פעולות שמירה, שליפה ועדכון (CRUD) מול בסיס הנתונים MongoDB.

12.2.2 תרשים המחלקות UML Class Diagram

תרשימי המחלקות (המחולקים לשלושה תרשימים נפרדים) מציגים את המבנה הארכיטקטוני הכולל של המערכת ואת חלוקת האחריות בין רכיבי התוכנה השונים. המערכת פותחה בארכיטקטורה מרובדות (3-Tier Architecture) מעל תשתית Spring Boot, ומשלבת את ספריית Vaadin לבניית ממשק המשתמש (ללא תלות ישירה ב-HTML) ואת Spring Data לניהול התקשורת מול מסד הנתונים MongoDB.

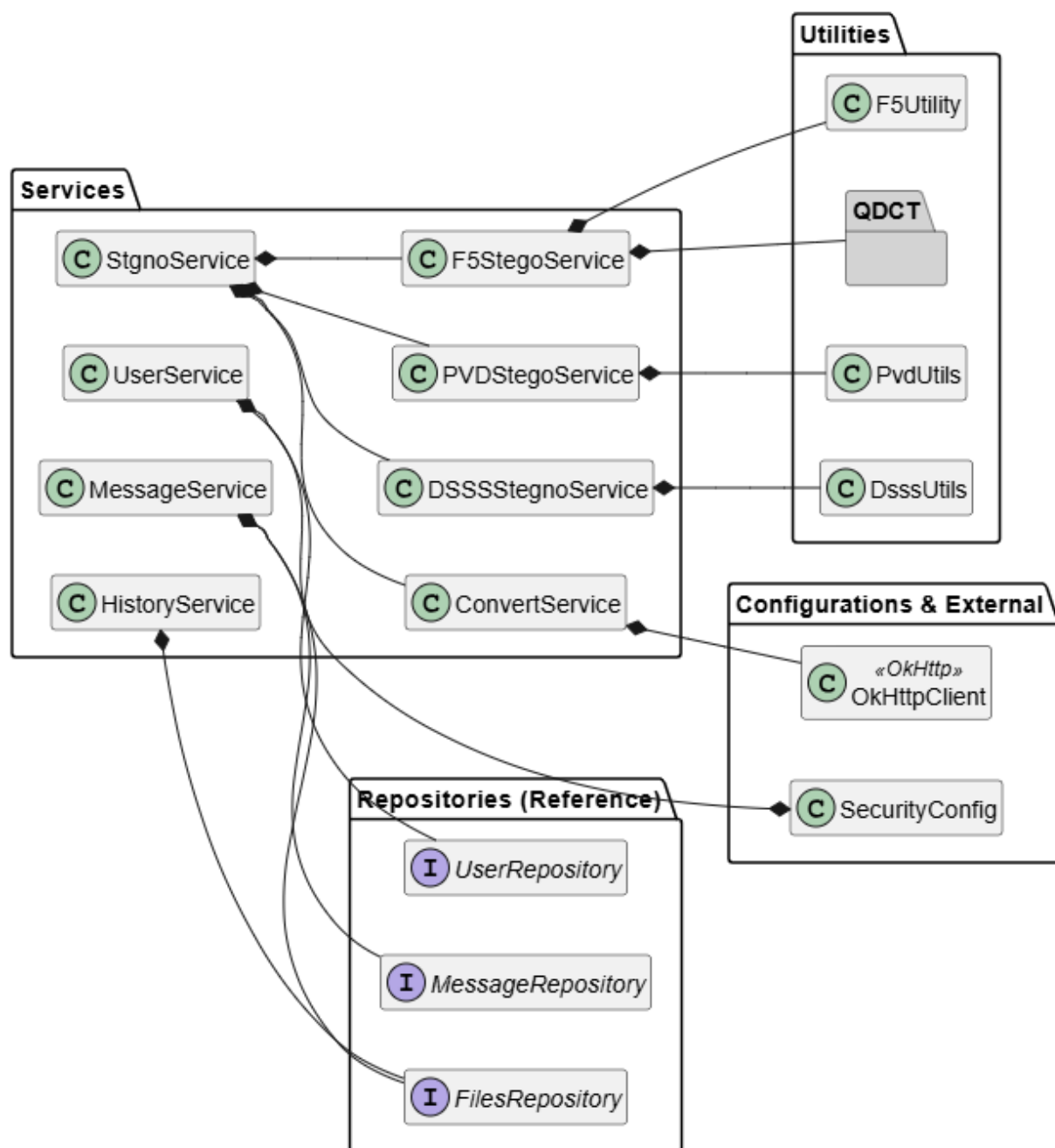
כדי לשמור על קוד מודולרי, על עקרון הפרדת הרשויות (Separation of Concerns) ועל קריאות התרשימים, הארכיטקטורה חולקה לשלוש שכבות מרכזיות, המוצגות בתרשימים נפרדים המקושרים זה לזה:

1. שכבת הנתונים (Data Layer):



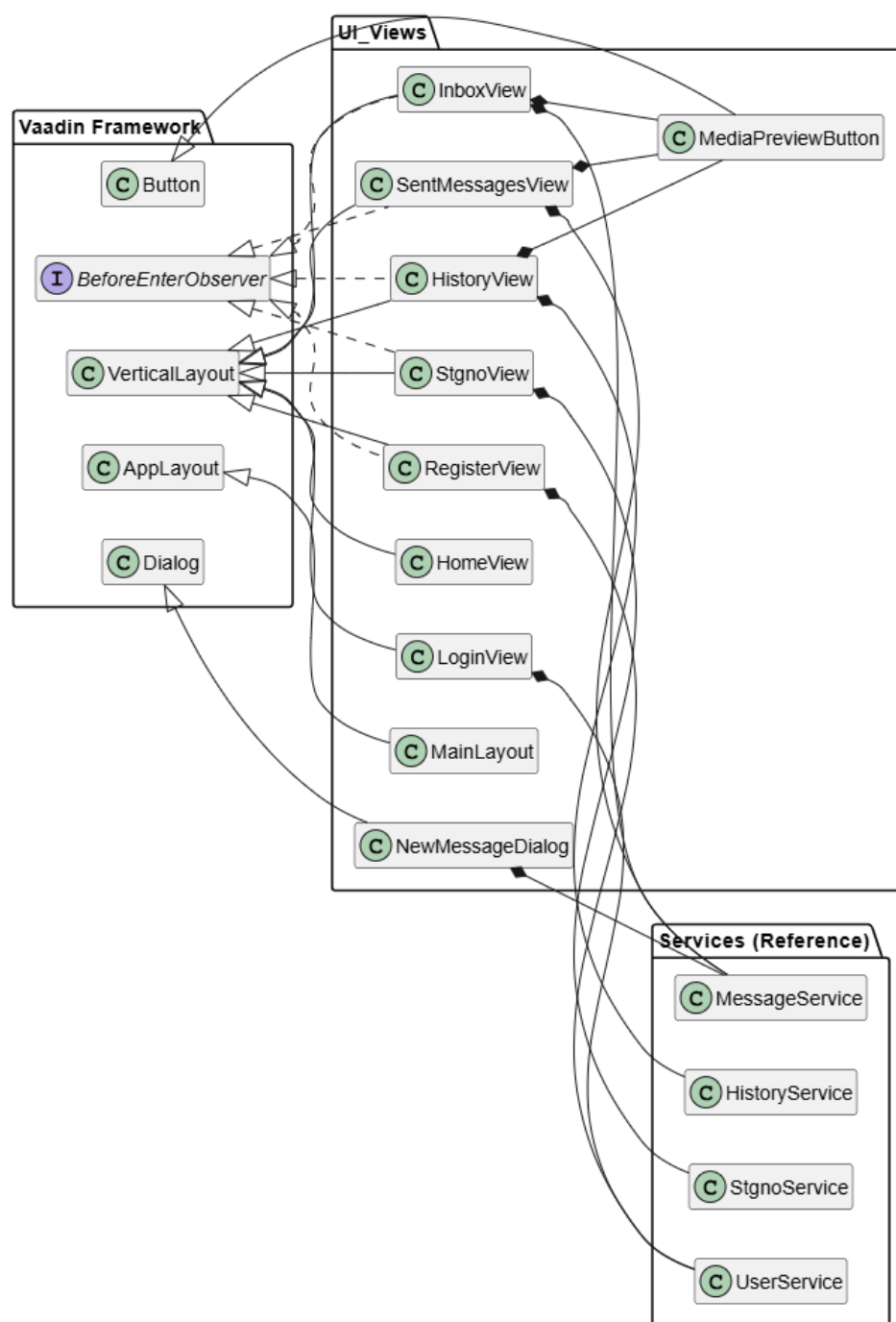
תרשים זה מתמקד במארגזי ה-Data Models וה-Repositories. הוא מייצג את מבנה הישויות במערכת (כגון משתמשים, הודעות וקבצי היסטוריה) ואת הממשקים האחראים על ביצוע פעולות ה-CRUD מול מסד הנתונים, תוך הסתרת מורכבות השאליות משאר המערכת. התרשים מציג יחסי ירושה (Inheritance) מתשתית Spring ויחסי אגרגציה (Aggregation) הממחישים כיצד ה-Repositories עובדים עם מודלי הנתונים.

2. שכבת הלוגיקה העסקית (Services & Business Logic):



תרשים המהווה את המוח של המערכת. שכבה זו מגשרת בין ממשק המשתמש לשכבת הנתונים. התרשים מרכז בתוכו את מארז ה-**Services** (ניהול התהליכים העסקיים של הדואר והמשתמשים, וניתוב שירותי הסטגנוגרפיה) ואת מארז ה-**Utilities** המבודד את המימוש המתמטי-טכני (מנועי ה-F5, DSSS, PVD וספריית ה-QDCT הפנימית) בהתאם לעקרון ההכמסה (Encapsulation). בנוסף, מוצגות כאן מחלקות תשתית חיצוניות (כמו הגדרות אבטחה). התרשים ממחיש קשרי **הכלה חזקה** (Composition) המייצגים את הזרקת ה-Repositories אל תוך השירותים.

3. שכבת התצוגה (Presentation / UI Layer):



תרשים זה מתמקד במארגז ה-UI Views, הכולל את מסכי המערכת והחלונות המודאליים, הבנויים כרכיבי צד-שרת. המסכים מנוהלים תחת מעטפת ניווט מרכזית (MainLayout). התרשים מדגים ירשה מרכיבי הספרייה של Vaadin (כמו VerticalLayout), מימוש ממשק (Realization) עבור בקרת הגישה למסכים (BeforeEnterObserver), וקשרי הכלה של השירותים העסקיים המקושרים ישירות לממשק המשתמש.

קווים מנחים בארכיטקטורה:

הקשרים בתרשימים (החצים) עושים שימוש אך ורק ביחסים תקינים של UML (ירשה, הכלה ומימוש ממשק) כדי להמחיש יישום קפדני של הזרקת תלויות (Dependency Injection). ניתן לראות כי חלוקת האחריות היא חד-כיוונית וברורה: שכבת התצוגה מכילה את שכבת ה-Services בלבד (ולעולם לא ניגשת ישירות למסד הנתונים), ושכבת ה-Services היא זו שמכילה ומתקשרת עם ה-Repositories. תכנון זה מבטיח רמת אבטחת מידע גבוהה, מודולריות, וקוד שניתן לתחזוקה ולהרחבה עתידית בקלות.

תרשים המחלקות מציג את המבנה הארכיטקטוני הכולל של המערכת ואת חלוקת האחריות בין רכיבי התוכנה השונים. המערכת פותחה בארכיטקטורה מרובדות (3-Tier Architecture) מעל תשתית Spring Boot ומשלבת את ספריית Vaadin לבניית ממשק המשתמש (ללא תלות ישירה ב HTML) ואת Spring Data לניהול התקשורת מול מסד הנתונים MongoDB.

כדי לשמור על קוד מודולרי ועל עקרון הפרדת הרשויות (Separation of Concerns) המערכת חולקה למאזרים (Packages) מרכזיים, המייצגים את השכבות השונות:

- **External Libraries & Configurations:** התשתית החיצונית עליה נשענת המערכת, יחד עם מחלקות האתחול והגדרות האבטחה (Security) המהוות את נקודת הפתיחה וסביבת הריצה של האפליקציה.
- **Data Models & Repositories (שכבת הנתונים):** ייצוג מבנה הישויות במערכת (כגון משתמשים, הודעות וקבצי היסטוריה) והממשקים האחראים על ביצוע פעולות ה CRUD מול מסד הנתונים, תוך הסתרת מורכבות השאילות משאר המערכת.
- **Services שכבת הלוגיקה העסקית:** המוח של המערכת. שכבה זו מגשרת בין ממשק המשתמש לשכבת הנתונים, ומרכזת בתוכה את שירותי הסטגנוגרפיה (כגון ניתוב למנועי ה F5, DSSS, ו-PVD) וניהול התהליכים העסקיים של הדואר והמשתמשים.
- **Utilities אלגוריתמיקה ועזרים:** מארז המבודד את המימוש המתמטי-טכני (כמו ספריית ה QDCT- הפנימית (מהלוגיקה הכללית, בהתאם לעקרון ההכמסה (Encapsulation).
- **UI Views שכבת התצוגה:** (מסכי המערכת והחלונות המודאליים, הבנויים כרכיבי צד-שרת. המסכים מנוהלים תחת מעטפת ניווט מרכזית (MainLayout) ומקושרים ישירות לשירותים העסקיים.

קווים מנחים בארכיטקטורה: הקשרים בתרשים (החצים) ממחישים יישום קפדני של **הזרקת תלויות (Dependency Injection)**. ניתן לראות כי חלוקת האחריות היא חד-כיוונית: שכבת התצוגה ניגשת אך ורק לשכבת ה Services (ולעולם לא ניגשת ישירות למסד הנתונים), ושכבת ה Services- היא זו שמתקשרת עם ה- Repositories. תכנון זה מבטיח רמת אבטחת מידע גבוהה, מודולריות, וקוד שניתן לתחזוקה ולהרחבה עתידית בקלות.

12.2.3 תיאור מחלקות מפורט

מחלקת Files

«@Document(collection="Files")» Files
-«@Id» id : String -userId : String -actionType : String -timestamp : Date -mediaType : String -imageData : byte[] -isDeleted : boolean
+Files() +Files(userId: String, actionType: String, mediaType: String, imageData: byte[]) +getId() : String +setId(id: String) : void +getUserId() : String +setUserId(userId: String) : void +getActionType() : String +setActionType(actionType: String) : void +getTimestamp() : Date +setTimestamp(timestamp: Date) : void +getMediaType() : String +setMediaType(mediaType: String) : void +getImageData() : byte[] +setImageData(imageData: byte[]) : void +isDeleted() : boolean +setDeleted(deleted: boolean) : void

מחלקת Files משמשת לאחסון וניהול קבצי המדיה במערכת. כל קובץ שעובר פעולת הסתרה (Embed) חילוצ (Extract) או העלאה סטנדרטית, נשמר במחלקה זו הממופה לאוסף "Files" ב MongoDB עיצוב זה מאפשר להפריד את העומס הבינארי מטבלאות מידע קלות יותר.

תכונות (Attributes)

- **id : String (Private):** מזהה הקובץ. נוצר אוטומטית על ידי MongoDB ומשמש כמפתח ראשי (<<@Id>>) מזהה זה הוא הערך שנשמר בשדה fileId במחלקת ההודעות (Message)
- **userId : String (Private):** שיוך הקובץ למשתמש (username) שהעלה או יצר אותו.
- **actionType : String (Private):** מתעד את הפעולה שבוצעה על הקובץ (לדוגמה "Embed": בעת הסתרת מידע "Extract", בעת חילוצ).
- **mediaType : String (Private):** שומר את סוג ה-MIME של הקובץ (למשל image/png או audio/wav) המאפשר למערכת לדעת כיצד להציג או לעבד אותו.
- **imageData : byte[] (Private):** מערך בייטים השומר בפועל את התוכן הבינארי של קובץ המדיה.
- **timestamp : Date (Private):** חותמת זמן המתעדת מתי הקובץ נוצר או נשמר במערכת.
- **isDeleted : Boolean (Private):** דגל (Flag) המיישם מנגנון "מחיקה לוגית" (Soft Delete) "מאפשר להסתיר את הקובץ מתצוגה מבלי למחוק אותו פיזית ממסד הנתונים.

בנאים (Constructors)

- **Files()**: בנאי ריק נדרש על ידי תשתית Spring Data MongoDB לשחזור אובייקט הקובץ בעת קריאת הנתונים מהמסד.
- **Files(userId, actionType, mediaType, imageData)**: בנאי ליצירת אובייקט קובץ חדש לשמירה. מקבל את מזהה המשתמש, סוג הפעולה, סוג המדיה ואת הנתונים הבינאריים. בנוסף, בנאי זה אחראי על הגדרה אוטומטית של זמן היצירה (`new Date()`) והגדרת דגל המחיקה ל `false`-כברירת מחדל.

מתודות (Methods)

- **Getters / Setters**: מתודות פומביות המאפשרות קריאה ועדכון של כלל נתוני הקובץ (כגון קריאת מערך הבייטים או שינוי סוג הפעולה).
- **isDeleted()**: מתודת אחזור ייעודית לשדה הבוליאני הבודקת האם הקובץ מסומן כמחוק לוגית במערכת.

מחלקת Message

«@Document(collection="messages")» Message
-«@Id» id : String -sender : String -receiver : String -body : String -sentTime : LocalDateTime -fileId : String
+Message() +Message(sender: String, receiver: String, body: String, fileId: String) +getId() : String +getSender() : String +getReceiver() : String +getBody() : String +getSentTime() : LocalDateTime +getFileId() : String +hasFile() : boolean +addClickListener(object: Object) : void

מחלקת Message מייצגת הודעה העוברת בין משתמשים במערכת. היא ממופה לאוסף "messages" במסד הנתונים MongoDB. מחלקה זו אחראית לאגד את המידע הטקסטואלי הגלוי, פרטי השולח והנמען, וליצור את הקישור לקובץ המדיה המצורף, במידה וקיים.

תכונות (Attributes)

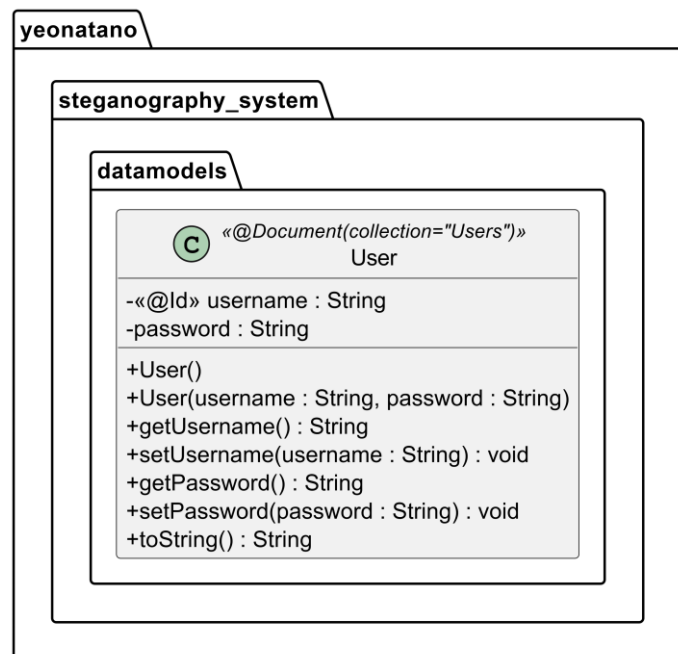
- **Id String (Private)**: מזהה ייחודי של ההודעה. מוגדר כמפתח הראשי (`<<@Id>>`) ומיוצר אוטומטית על ידי מסד הנתונים.
- **sender : String (Private)**: מחרוזת השומרת את שם המשתמש של הנמען.
- **receiver : String (Private)**: מחרוזת השומרת את שם המשתמש של השולח.
- **body : String (Private)**: תוכן ההודעה.
- **sentTime : LocalDateTime (Private)**: חותמת זמן מדויקת של מועד שליחת ההודעה. מאותחלת כברירת מחדל לזמן היצירה.
- **fileId : String (Private)**: שדה מפתח לוגי השומר את המזהה של אובייקט Files המקושר להודעה. מאפשר שמירת קבצים כבדים באוסף נפרד תוך שמירה על יעילות שלילת ההודעות.

בנאים (Constructors)

- **Message()**: בנאי ריק זה נדרש על ידי תשתית Spring Data MongoDB לצורך המרת הנתונים מהמסד לאובייקט Java בעת שליפה מהמאגר.
- **Message(sender, receiver, body, fileId)**: בנאי אתחול. מקבל את פרטי השולח, הנמען, תוכן הטקסט הגלוי ומזהה הקובץ המצורף (אם ישנו). משמש ליצירה מהירה של אובייקט הודעה חדש רגע לפני שליחתו ושמירתו במסד הנתונים.

מתודות (Methods)

- **Getters**: מתודות אחזור המספקות גישה לקריאת נתוני ההודעה מבלי לאפשר את שינויים לאחר השליחה (למעט מניפולציות מוגדרות מראש).
- **hasFile : Boolean()**: פונקציית עזר לוגית (Utility Method) המבצעת בדיקת תקינות (Validation) ומחזירה ערך בוליאני האם להודעה מצורף קובץ מדיה תקין, על ידי בדיקה שה fileId אינו ריק.

מחלקת USER

מחלקת User משמשת כמודל נתונים (Data Model) המייצג את פרטי המשתמשים במערכת ההסתרה והשליחה. המחלקה ממופה ישירות לאוסף (Collection) בשם "Users" בתוך מסד הנתונים MongoDB, כפי שמוגדר באמצעות האנוטציה <<@Document>>.

תכונות: (Attributes)

- **Username : String (Private)**: שדה זה שומר את שם המשתמש. הוא מסומן באנוטציה <<@Id>> המגדירה אותו כמפתח ראשי (Primary Key) ב-MongoDB, מה שמבטיח שכל שם משתמש במערכת יהיה ייחודי לחלוטין.
- **password : String (Private)**: שדה השומר את סיסמת המשתמש (לרוב לאחר הצפנה או גיבוב, בהתאם להגדרות האבטחה של המערכת).

בנאים (Constructors)

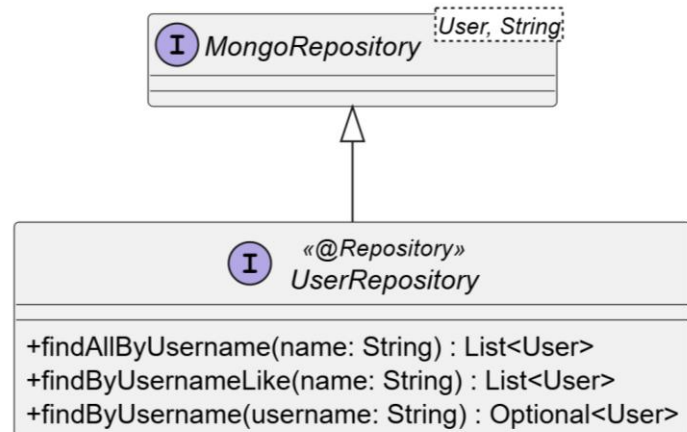
- **User()** - (בנאי ריק): בנאי זה נדרש על ידי התשתית של Spring Data MongoDB כדי לבנות מחדש את אובייקט המשתמש בעת שליפת נתונים מהמסד.

- **User (username, password)**: בנאי המקבל פרמטרים המשמש ליצירת אובייקט משתמש חדש ומהיר בעת הרשמה למערכת.

מתודות (Methods)

- **Getters / Setters** מתודות אלו (getUsername, setUsername, getPassword, setPassword) הן מתודות פומביות (Public) המאפשרות גישה מבוקרת ועדכון של השדות הפרטיים.
- **toString()** דריסה (Override) של מתודת toString המובנית ב Java הפונקציה מחזירה ייצוג מחרוזתי של פרטי המשתמש, דבר המאפשר יצירת טקסט עם פרטים האובייקט לצורך בדיקות.

ממשק UserRepository



UserRepository הוא ממשק (Interface) האחראי על ניהול התקשורת מול אוסף ה "Users" במסד הנתונים MongoDB הוא מתויג באנוטציה <<@Repository>> כדי להגדיר אותו כרכיב גישה לנתונים במערכת של Spring Boot

תכונות וירושה (Attributes & Inheritance)

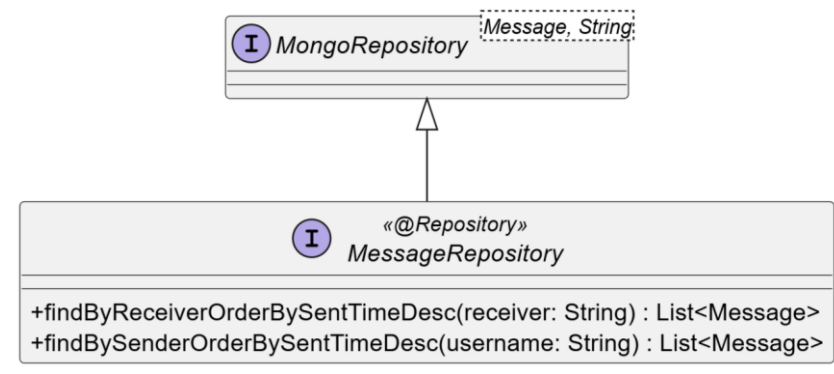
- **ירושה מ <User, String> MongoRepository**: הממשק אינו מכיל שדות (Attributes) פרטיים משלו. במקום זאת, הוא יורש יכולות עוצמתיות מתשתית Spring Data. הפרמטר הראשון (User) מציין את ישות הנתונים המנוהלת, והפרמטר השני (String) מציין את סוג הנתון של המפתח הראשי (שם המשתמש). ירושה זו מעניקה אוטומטית פונקציות CRUD (יצירה, קריאה, עדכון, מחיקה) ללא צורך בכתיבת קוד.

בנאים (Constructors)

- **הזרקת תלויות (Dependency Injection)**: מכיוון שמדובר בממשק (Interface) ולא במחלקה רגילה, אין לו בנאים (Constructors). תשתית Spring מתערבת בזמן ריצה (Runtime), מייצרת באופן אוטומטי אובייקט (Proxy) המממש ממשק זה, ומזריקה אותו לשכבת השירות (Services) באמצעות מנגנון Inversion of Control.

מתודות (Methods)

- **findAllByUsername (name) : List<User>**: מתודה השולפת רשימה של משתמשים בעלי שם משתמש זהה לחלוטין לפרמטר המבוקש.
- **findByUsernameLike (name) : List<User>**: מתודה המיועדת לחיפוש חלקי. (Like Query) היא מאפשרת למצוא משתמשים ששםם מכיל מחרוזת מסוימת, מה ששימושי למנועי חיפוש פנימיים במערכת.
- **findByUsername (username) : Optional<User>**: מתודה קריטית לתהליכי אימות (Authentication). היא מחפשת משתמש ספציפי ומחזירה אותו עטוף באובייקט Optional, שמונע שגיאות מסוג NullPointerException במקרה שהמשתמש אינו קיים במערכת.

ממשק MessageRepository

MessageRepository מנהל את שליפת ושמירת ההודעות המועברות בין המשתמשים באוסף "messages" ב-MongoDB- הממשק חיוני לטעינת תיבות הדואר הנכנס והיוצא של המשתמשים במערכת ההסברה.

תכונות וירושה (Attributes & Inheritance)

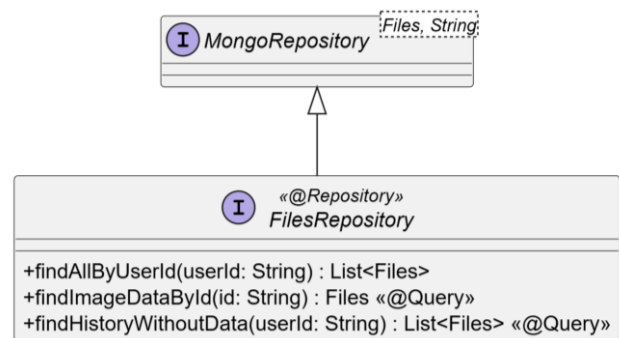
- **ירושה מ `MongoRepository <Message, String>`:** בדומה לממשק המשתמשים, ממשק זה יורש את יכולות הבסיס, כאשר הוא מוגדר לנהל ישויות מסוג `Message` המזוהות על ידי מפתח ראשי מסוג מחרוזת (String).

בנאים (Constructors)

- **ללא בנאים מפורשים:** בהיותו ממשק, יצירת האובייקט מתבצעת על ידי תשתית Spring תחת הקלעים, והוא מוזרק ישירות למחלקות השירות (למשל `MessageService`) ללא קריאה לבנאי באמצעות המילה השמורה `new`.

מתודות (Methods)

- **`findByReceiverOrderBySentTimeDesc(receiver) : List<Message>`:** מתודה ייעודית לתיבת **דואר נכנס**. היא מסננת את כל ההודעות כך שיוחזרו רק אלו שהנמען (`receiver`) שלהן תואם למשתמש המבוקש. בנוסף, מנוע Spring Data מפרש את המילה `OrderBySentTimeDesc`: וממין אוטומטית את התוצאות מההודעה החדשה ביותר לישנה ביותר.
- **`findBySenderOrderBySentTimeDesc(username) : List<Message>`:** מתודה ייעודית לתיבת **דואר יוצא**. היא פועלת באופן זהה, אך מסננת את ההודעות לפי שדה השולח, תוך מיון כרונולוגי יורד.

ממשק FilesRepository

FilesRepository מנהל את קבצי המדיה הכבדים (תמונות ואודיו) באוסף "Files" ב MongoDB מכיוון שקובץ מדיה מכיל מערך בייטים שעלול להעמיס על הזיכרון, ממשק זה כולל שאילתות מותאמות אישית (Custom Queries) לביצוע אופטימיזציה בשליפת הנתונים.

תכונות וירשה (Attributes & Inheritance)

- **ירשה מ `MongoRepository<Files, String>`:** מנהל ישויות מסוג Files, כאשר מזהה הקובץ (ה-ID שלו ב MongoDB) מוגדר כ String

בנאים (Constructors)

- **ללא בנאים מפורשים:** ממומש ומוזרק אוטומטית על ידי תשתית ה Spring Container-מה ששומר על ארכיטקטורה נקייה של הפרדות ממשקים.

מתודות (Methods)

- **`findAllByUserId(userId) : List<Files>`:** מתודה סטנדרטית השולפת את כל הקבצים המשויכים למשתמש מסוים, כולל כל הנתונים שלהם.
- **`findImageDataById(id) : Files`:** מתודה המשתמשת באנוטציה `<<@Query>>` השאילתה המוגדרת בה `{ 'imageData': 1, '_id': 0 }` מבצעת תהליך הנקר "Projection". היא שולפת אך ורק את התוכן הבינארי (imageData) של הקובץ הספציפי ממסד הנתונים, מבלי לטעון את שאר השדות, מה שמיעל משמעותית את זמן התגובה בעת חילוץ תמונה.
- **`findHistoryWithoutData(userId) : List<Files>`:** מתודה קריטית לביצועים המבוססת גם היא על `<<@Query>>` היא שולפת את היסטוריית הפעולות של המשתמש החלק החשוב כאן הוא ההגדרה `{ 'imageData': 0 }`, המורה למסד הנתונים **לא לשלוח** את מערך הבייטים הכבד אל הזיכרון. זהו תכנון הנדסי המאפשר הצגה מהירה של טבלת היסטוריה בממשק המשתמש (UI), מבלי להקריס את השרת עקב חריגת זיכרון.

מחלקת UserService

 «@Service» UserService
-passwordEncoder : PasswordEncoder -userRepository : UserRepository
+UserService(userRepository: UserRepository, passwordEncoder: PasswordEncoder) +addUserToDB(user: User) : boolean +authenticate(username: String, rawPassword: String) : User +getAllUsers() : ArrayList<User>

מחלקת UserService מרכזת את הלוגיקה העסקית הקשורה לניהול זהויות המשתמשים במערכת (Identity Management). המחלקה מסומנת באנוטציה «@Service» המגדירה אותה כרכיב (Bean) המנוהל על ידי תשתית ה Inversion of Control (IoC) של Spring Boot. מחלקה זו מפרידה לחלוטין את פעולות ההרשמה והאימות משכבת התצוגה, ואחראית על אבטחת המידע של סיסמאות המשתמשים.

תכונות (Attributes)

- **passwordEncoder : PasswordEncoder**: ממשק של Spring Security האחראי על גיבוב (Hashing) והצפנת סיסמאות.
- **userRepository : UserRepository - Private**: מופע של רכיב הגישה לנתונים, המאפשר ביצוע פעולות מול אוסף ה Users-במסד ה MongoDB

בנאים (Constructors)

- **UserService(userRepository, passwordEncoder)**: הבנאי המחלקה. הבנאי מיישם תבנית עיצוב קריטית של **הזרקת תלויות (Constructor Dependency Injection)**. במקום שהמחלקה תיצור בעצמה את התלויות שלה, תשתית Spring מזריקה אליה את רכיב מסד הנתונים ורכיב ההצפנה בעת יצירתה. גישה זו מבטיחה שהאובייקט תמיד ייווצר במצב תקין ומקלה משמעותית על ביצוע בדיקות יחידה (Unit Tests).

מתודות (Methods)

- **AddUserToDB (user) : Boolean**: מנהלת את תהליך ההרשמה. המתודה מבצעת ולידציה ששם המשתמש טרם תפוס. אם הוא פנוי, היא מבצעת מניפולציה על אובייקט המשתמש - מחליפה את סיסמת הטקסט הגלוי בסיסמה מגובבת (Hashed) בעזרת ה passwordEncoder, ורק אז שומרת את הרשומה המאובטחת במסד הנתונים.
- **Authenticate(username, rawPassword) : User**: מתודת אימות (התחברות). היא שולפת את המשתמש מהמסד (תוך שימוש ב Optional-למניעת קריסות אם המשתמש אינו קיים), ומשתמשת בפונקציית matches של רכיב ההצפנה. פונקציה זו מאמתת שהסיסמה הגלוייה שהוקלדה תואמת לגיבוב השמור במסד, תוך הגנה מפני התקפות תזמון. (Timing Attacks)
- **getAllUsers() : ArrayList<User>**: מתודת שירות השולפת את רשימת כלל המשתמשים במערכת (מומרת למבנה נתונים מסוג ArrayList).

מחלקת MessageService

 «@Service» MessageService
-messageRepository : MessageRepository -filesRepository : FilesRepository
+MessageService(messageRepository: MessageRepository, filesRepository: FilesRepository) +sendMessage(sender: String, receiver: String, body: String, fileData: byte[], mimeType: String, actionType: String) : void +getMyInbox(username: String) : List<Message> +getMySentMessages(username: String) : List<Message> +getFileById(fileId: String) : Files +deleteMessage(id: String) : void

מחלקת MessageService מתפקדת כרכיב ניהול ("Orchestrator") של תעבורת ההודעות והקבצים במערכת ההסתרה. גם היא מנוהלת כ Bean-על ידי (<<@Service>>) Spring Boot. תפקידה העיקרי הוא לנהל את הלוגיקה המורכבת של שמירת הודעות המכילות טקסט גלוי המשולב עם קבצי מדיה כבדים, תוך שמירה על ביצועים גבוהים ויעילות זיכרון.

תכונות (Attributes)

- **messageRepository : MessageRepository**: רכיב הגישה המאפשר שליפה ושמירה של ישויות הטקסט (הודעות).
- **filesRepository : FilesRepository**: רכיב הגישה המאפשר מניפולציות על אוסף הקבצים הבינאריים.

בנאים (Constructors)

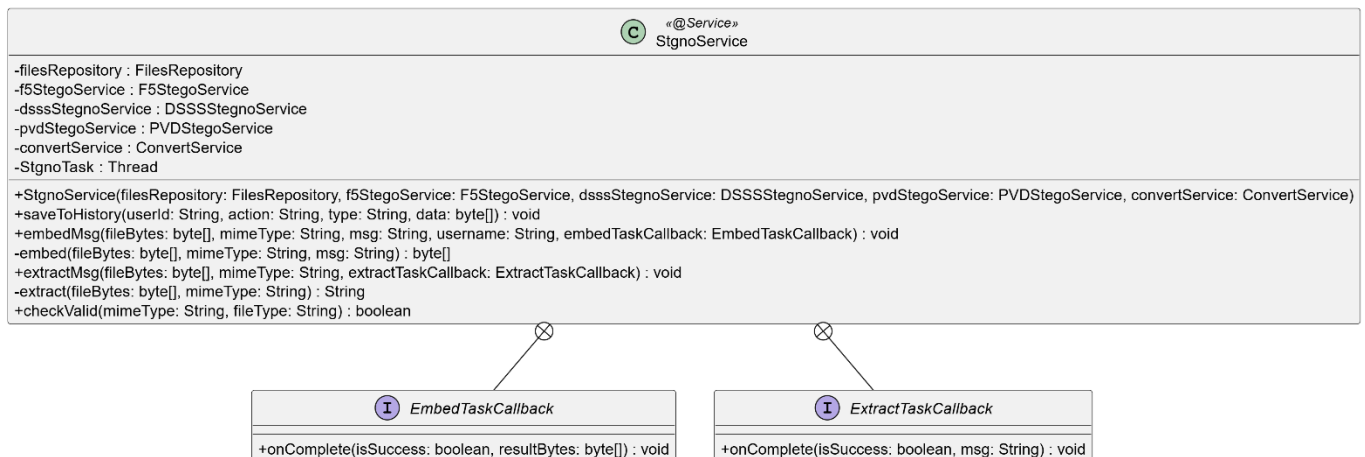
- **MessageService(messageRepository, filesRepository)**: הבנאי מיישם הזרקת תלויות הוא מבטיח ששירותי ההודעות מקבל לידיו את שני ממשקי מסד הנתונים הנדרשים לפעולתו לפני שהוא מתחיל לטפל בבקשות משתמשים.

מתודות: (Methods)

- **sendMessage(String sender, String receiver, String body, byte[] fileData, String mimeType, String actionType) : void**

זוהי מתודת הליבה של המחלקה. היא מיישמת החלטה ארכיטקטונית חשובה של נרמול נתונים. במקום לשמור את הקובץ הבינארי הענק בתוך רשומת ההודעה, המתודה שומרת קודם את הקובץ דרך ה-filesRepository. לאחר מכן, היא מחלצת את ה-ID-הייחודי שנוצר לקובץ (מפתח זר), ומציבה אותו בתוך אובייקט ה-Message-שנשמר בנפרד. הפרדה זו מונעת עומס זיכרון (Out of Memory) כאשר המשתמש טוען את תיבת הדואר שלו.

- **getMyInbox(username) : List<Message>**: שולפת את ההודעות הנכנסות עבור הנמען המבוקש. מסתמכת על שאילתות אוטומטיות של Spring Data שמחזירות את התוצאות ממוינות כרונולוגית (מהחדש לישן).
- **getMySentMessages(username) : List<Message>**: פועלת בדומה ל, Inbox-אך מסננת את מסד הנתונים לפי השולח, לצורך תצוגת דואר יוצא.
- **getFileById(fileId) : Files**: מתודת שליפה בטוחה המיישמת הגנת ולידציה מוקדמת (Fail-Fast) היא מוודאת שה-ID-אינו ריק לפני פנייה למסד, ומשתמשת במעטפת Optional כדי למנוע קריסה במידה והקובץ לא נמצא, ומחזירה בבטחה את הקובץ ל UI-לצורך חילוץ סטגוגרפי או תצוגה.
- **deleteMessage(id) : void**: מתודה המבצעת מחיקה פיזית (Hard Delete) של רשומת הודעה ספציפית לפי המזהה שלה.

מחלקת StegnoService

מחלקת StegnoService היא "המוח" והמנוע הראשי של פעולות הסטגוגרפיה במערכת. היא מיישמת את תבנית העיצוב היא מספקת ממשק יחיד ופשוט לשכבת התצוגה, תוך שהיא מסתירה ממנה את כל המורכבות של ניתוב הקבצים, בחירת אלגוריתמי ההצפנה (F5/PVD/DSSS) והמרות הפורמטים. בנוסף, המחלקה מנהלת ארכיטקטורה אסינכרונית כדי למנוע את "הקפאת" ממשק המשתמש בזמן חישובים מתמטיים כבדים.

תכונות (Attributes) וממשקים פנימיים (Inner Interfaces)

- **EmbedTaskCallback ו-ExtractTaskCallback: ממשקים פנימיים** מגדירים חוזה להתקשרות אסינכרונית (Callback). מאחר שפעולות ההצפנה רצות ב Thread-נפרד, ממשקים אלו משמשים כדי "להודיע" בחזרה לשכבת ה UI-על סיום הפעולה והחזרת התוצאה.
- **שירותים (Private): f5StegoService, dsssStegoService, pvdStegoService, convertService:** שדות אלו ומאפשרים למערכת לטפל בסוגי מדיה שונים.
- **StgnoTask : Thread:** אובייקט תהליכון המשמש להרצת אלגוריתמי ההצפנה והפענוח ברקע (Background Task), כך שהשרת יוכל להמשיך לטפל בבקשות משתמשים אחרות.

בנאים (Constructors)

- **StegnoService(...):** בנאי מרכזי המיישם הזרקת תלויות (DI) של כלל שירותי הליבה (הצפנות, המרות, ומסד הנתונים) באמצעות התשתית של Spring.

מתודות (Methods)

- **embedMsg(...): void:** פונקציית הליבה להטמעת מסר. פועלת ב Thread נפרד. היא מבצעת שרשרת פעולות: ולידציה של הקובץ, המרה אוטומטית במקרה של פורמט לא נתמך) למשל המרת PNG לWebP ניתוב לאלגוריתם המתאים, שמירה בהיסטוריה, ולבסוף הפעלת פונקציית Callback לחזרה ל UI
- **extractMsg(...): void:** פונקציית הליבה לחילוץ מסר ממדיה נגועה (Stegno-object). מפענחת את הקובץ בסביבה אסינכרונית ומחזירה את המחרוזת החשופה דרך ה Callback
- **checkValid(): Boolean:** מתודת עזר (Utility) המבצעת ולידציה לפורמטים של הקבצים לפני עיבודם, על מנת למנוע קריסות (Exceptions) באלגוריתמים מתמטיים הרגישים לסוגי מדיה.

מחלקת HistoryService

 «@Service» HistoryService
-filesRepository : FilesRepository
+HistoryService(filesRepository: FilesRepository) +getActiveUserHistory(username: String) : List<Files> +softDeleteFile(fileId: String) : void +getFileData(fileId: String) : byte[]

מחלקת HistoryService מרכזת את הלוגיקה העסקית לניהול היסטוריית הפעולות של המשתמשים במערכת. מחלקה זו תוכננה עם דגש רב על ביצועים ואופטימיזציות זיכרון, תוך הפרדה מוחלטת בין טעינת המידע הטקסטואלי של היסטוריית הפעולות לבין טעינת קבצי המדיה הכבדים עצמם.

תכונות (Attributes)

- **filesRepository : FilesRepository**: רכיב הגישה לנתונים האחראי לתקשורת מול אוסף ה"Files" במסד הנתונים MongoDB.

בנאים (Constructors)

- **HistoryService(filesRepository)**: בנאי המיישם הזרקת תלויות (Constructor DI). מבטיח שהשירות יוכל לתקשר עם מסד הנתונים מרגע הפעלת המערכת באופן בטוח ונקי מתלויות קשיחות.

מתודות (Methods)

- **getActiveUserHistory(username) : List<Files>**: מתודה ייעודית לתצוגת ההיסטוריה. היא קוראת לפונקציה מותאמת אישית במסד הנתונים ששולפת את הפעולות ללא התוכן הבינארי (imageData). גישה זו מונעת קריסות זיכרון (Out of Memory) בעת טעינת רשימת פעולות ארוכה ל UI
- **softDeleteFile(fileId) : void**: מיישמת מנגנון "מחיקה לוגית (Soft Delete)": במקום למחוק את הרשומה לצמיתות (Hard Delete) ולאבד את תיעוד הפעולה, הפונקציה שולפת את הקובץ ומשנה את סטטוס ה isDeleted שלו ל true, מה שמסתיר אותו מעיני המשתמש אך שומר על שלמות בסיס הנתונים.
- **getFileData(fileId) : byte[]**: פונקציית שליפה מדויקת תחת דרישה (On-Demand / Lazy Loading) כאשר המשתמש מבקש להוריד קובץ ספציפי מתוך ההיסטוריה, פונקציה זו ניגשת למסד הנתונים, שולפת את מערך הבייטים של אותו קובץ בלבד ומחזירה אותו, תוך טיפול בחריגות מובנה (Try-Catch) למניעת קריסות השרת במידה והקובץ פגום.

מחלקת ConvertService

 «@Service» ConvertService
-API_KEY : String -client : OkHttpClient
+ConvertService() +convertFormat(fileData: byte[], sourceMimeType: String, targetFormat: String) : byte[] -createJob(targetFormat: String) : JSONObject -uploadFile(uploadTask: JSONObject, fileData: byte[], mimeType: String, ext: String) : void -waitForResult(jobId: String) : String -downloadFileAsBytes(downloadUrl: String) : byte[]

מחלקת ConvertService משמשת כשירות המרת קבצים מבוסס ענן (CloudConvert API). ייעודה לפתור בעיות תאימות (Incompatibility) כאשר המשתמש מעלה פורמטים שאינם נתמכים ישירות על ידי אלגוריתמי המערכת. החלטה ארכיטקטונית חשובה: השירות מתוכנן לעבוד במודל **In-Memory** בלבד. הוא מקבל ומחזיר מערכי בייטים (byte[]) ללא קריאה או כתיבה לדיסק הקשיח של השרת, מה שמונע יצירת קובצי זבל, משפר את זמן התגובה, ומעלה את רמת אבטחת המידע (Data Privacy).

תכונות (Attributes)

- **API_KEY : String (Private):** מחרוזת טוקן (Bearer Token) המשמשת לאימות מאובטח מול השרתים החיצוניים של CloudConvert.
- **client : OkHttpClient:** אובייקט לניהול תקשורת HTTP סינכרונית, דרכו מתבצעות בקשות (REST API).

בנאים (Constructors)

- **ConvertService():** בנאי המחלקה. מאתחל את ה OkHttpClient עם הגדרות Timeout מותאמות אישית של 60 שניות (לקריאה, כתיבה וחיבור). הגדרה זו חיונית, כיוון שהמרת קבצי אודיו ותמונה בענן עשויה לקחת זמן, והארכת ה Timeout מונעת קריסות של השרת בטרם ההמרה הושלמה.

מתודות: (Methods)

- **convertFormat(...) : byte[]:** פונקציית העל (Orchestrator) הפומבית. היא מנהלת את מחזור החיים השלם של ההמרה באמצעות קריאה לארבע מתודות העזר הפרטיות.
- **createJob (Private):** בונה צינור נתונים (Pipeline) מול ה API-החיצוני ומגדירה את הפעולה הנדרשת. (Import -> Convert -> Export)
- **uploadFile (Private):** מעלה את המידע הבינארי לשרת האחסון הזמני באמצעות בקשת Multipart Form Data.
- **waitForResult (Private):** מיישמת מנגנון "המתנה פעילה" (Polling). "היא דוגמת את שרת ההמרה כל מספר שניות עד לסיום התהליך כדי לא להציף אותו בבקשות (Rate Limiting).
- **downloadFileAsBytes (Private):** מושכת את תוצאת ההמרה ישירות אל תוך זיכרון מערך בייטים.

F5StegnoService מחלקת

 «@Service» F5StegnoService
<pre>+F5StegnoService() +embed(fileBytes: byte[], secretMsg: String) : byte[] +extract(fileBytes: byte[]) : String</pre>

מחלקה זו אחראית על יישום אלגוריתם הסטגנוגרפיה **F5** עבור קבצי תמונה מסוג JPEG. היא מבצעת הטמעה וחילוץ של מידע במרחב התדר (Frequency Domain) על ידי שינוי מקדמי ה DCT (Discrete Cosine Transform) של התמונה, תוך שימוש בקידוד מטריצה (Matrix Encoding) למזעור הפגיעה החזותית.

תכונות (Attributes): למחלקה זו אין שדות גלובליים (Attributes), משום שכל עיבוד התמונה מתבצע באופן מקומי בתוך מתודות השירות, מה ששומר על המחלקה נטולת מצב (Stateless) ובטוחה לעבודה מקבילית (Thread-Safe).

בנאים (Constructors)

- **F5StegnoService()**: בנאי ברירת המחדל נוצר אוטומטית על ידי Spring Boot שמנהל את המחלקה כ-Singleton Bean.

מתודות (Methods)

- **embed(fileBytes, secretMsg) : byte[]**: פונקציית ההטמעה. היא קוראת את מקדמי ה DCT-מהקובץ המקורי, משרשרת את אורך המסר (32 ביט) לתוכן המסר עצמו, ומטמיעה אותם בעזרת פעולות מתמטיות (XOR) על המקדמים שאינם אפס. מחזירה את ה Stegno-Image הבינארי.
- **Extract(fileBytes) : String**: פונקציית החילוץ. פועלת באופן הפוך: תחילה שולפת את 32 הביטים הראשונים כדי לדעת את אורך המסר, ולאחר מכן מחלצת את תוכן ההודעה בעזרת פעולות XOR על הביטים הפחות משמעותיים (LSB) של המקדמים.

DSSSSStegnoService מחלקת

 «@Service» DSSSSStegnoService
<u>-FIXED_PASSWORD : String</u>
+DSSSSStegnoService() +embed(fileBytes: byte[], messageStr: String) : byte[] +extract(fileBytes: byte[]) : String

DSSSSStegnoService מיישמת אלגוריתם סטגנוגרפיה מתקדם לאודיו בפורמט WAV המבוסס על **DSSS** מחלקה זו משלבת עוצמת הטמעה אדפטיבית (Adaptive Alpha) ופיזור של ביטי המסר יחד עם רצף פסאודו אקראי (PN) מה שהופך את ההצפנה לעמידה במיוחד בפני רעשי רקע.

(Attributes) תכונות

- **FIXED_PASSWORD : String**: סיסמה קבועה המשמשת כגרעין (Seed) ליצירת רצף ה PN הזהה הן בשלב ההטמעה והן בשלב החילוץ העיוור.

(Constructors) בנאים

- **DSSSSStegnoService()**: בנאי ברירת מחדל לניהול על ידי מכולת ה Spring-LoC של.

(Methods) מתודות

- **embed(...) : byte[]**: מבצעת הנמכה קלה (Attenuation) של האודיו המקורי, ולאחר מכן מחשבת "אלפא מקומית" (עוצמת הטמעה) המשתנה בהתאם לעוצמת האודיו המקומית (למשל מוזיקה שקטה מול תופים רועשים). היא מפזרת את הביטים המומרים לפורמט ביפולרי לאורך הדגימות.
- **extract(...) : String**: מבצעת חילוץ עיוור (Blind Extraction). משתמשת בטכניקת Zero-Mean Normalization (החסרת ממוצע מקומי) כדי לסנן תדרי באס ורעשים מהמוזיקה, ומבצעת חישוב קורלציה (מכפלה פנימית) מול רצף ה PN-כדי להחליט אם הוטמע 0 או 1.

PVDStegnoService מחלקת

 «@Service» PVDStegnoService
<pre>+PVDStegnoService() +embed(fileBytes: byte[], secretMessage: String) : byte[] +extract(fileBytes: byte[]) : String</pre>

שירות זה מיישם את אלגוריתם הסטגנוגרפיה **PVD** (Pixel Value Differencing) במרחב המרחבי של תמונות (פורמט PNG). האלגוריתם מנצל את העובדה שהעין האנושית רגישה פחות לשינויים באזורים בעלי ניגודיות גבוהה (קצוות בתמונה), ומטמיע כמויות משתנות של מידע בהתאם להפרשי הצבע בין זוגות פיקסלים סמוכים.

תכונות (Attributes) למחלקה אין שדות גלובליים (Stateless Pattern), כדי להבטיח ניהול זיכרון יעיל בעת עיבוד מערכי פיקסלים של תמונות ברזולוציה גבוהה.

בנאים (Constructors)

- **PVDStegnoService()**: בנאי ברירת מחדל הנדרש להזרקת השירות (DI) למחלקת הניתוב המרכזית (StegnoService)

מתודות (Methods)

- **embed(...) : byte[]**: הפונקציה סורקת את התמונה בזוגות של פיקסלים, מחשבת את ההפרש ביניהם, ונעזרת בטבלת טווחים מוגדרת מראש כדי לקבוע כמה ביטים ניתן להטמיע באותו זוג (קיבולת דינמית). הפונקציה כוללת מנגנון חכם להגנה מפני חריגות (Overflow/Underflow), המוודא שערכי הצבע של הפיקסלים (0-255) נשמרים תקינים לאחר ההצפנה מבלי לשנות את ההפרש המחושב.
- **extract(...) : String**: פונקציה הסורקת את ה Stegno-Image באותו מסלול זוגי, מחשבת את ההפרשים מחדש, ומחלצת את ביטי המידע הסודי מתוך ההפרש שבין גודל הפער הנוכחי לגבול התחתון של הטווח שלו, וממירה אותם חזרה למחרזת טקסט קריאה.

מחלקת PvdUtils

C PvdUtils
+MAX_IMAGE_WIDTH : int = 2000 +MAX_IMAGE_HEIGHT : int = 2000 +MESSAGE_LENGTH_BITS : int = 16 -RANGE_TABLE : int[][]
+getRangeIndex(absDifference: int) : int +getRangeStart(rangeIndex: int) : int +getCapacity(rangeIndex: int) : int +getBlueValue(image: BufferedImage, x: int, y: int) : int +setBlueValue(image: BufferedImage, x: int, y: int, newBlue: int) : void +isImageSizeValid(image: BufferedImage) : boolean +valueToBits(value: int, numBits: int) : BitSet +bitsToValue(bits: BitSet, numBits: int) : int +textToBits(text: String) : BitSet +bitsToText(bits: BitSet, numBytes: int) : String

מחלקת PvdUtils היא מחלקת עזר סטטית המרכזת את כלל הלוגיקה המתמטית והבינארית הדרושה לאלגוריתם ה-Pixel Value Differencing-היא אחראית על ניהול טבלת הטווחים (Quantization Table), חילוך והטמעה בטוחה של ערוצי צבע, והמרות מדויקות בין טקסט למבנה נתונים מסוג BitSet.

תכונות (Attributes)

MAX_IMAGE_WIDTH / MAX_IMAGE_HEIGHT: קבועי מערכת המגבילים את רזולוציית התמונה כדי להגן על השרת מפני חריגות זיכרון (Out Of Memory).

• **MESSAGE_LENGTH_BITS**: קבוע המגדיר את מספר הביטים שיוקצו בראש התמונה (16) לשמירת אורך המסר.

• **RANGE_TABLE : int[][]**: טבלה דו-ממדית המהווה את לב האלגוריתם. היא מגדירה טווחים מתמטיים קבועים הקובעים כמה ביטים מותר להטמיע בכל זוג פיקסלים בהתאם ל"רעש" החזותי באותו אזור.

בנאים (Constructors)

• בהיותה מחלקת עזר (Utility Class), המחלקה אינה מיועדת ליצירת מופעים (Instances) ולכן אינה כוללת בנאי. כל הפעולות מבוצעות דרך מתודות סטטיות.

מתודות (Methods)

• **מתודות טווחים (getRangeIndex, getCapacity)**: פונקציות מתמטיות המחלצות מידע מתוך RANGE_TABLE על בסיס חישוב ההפרש בין פיקסלים סמוכים.

• **פעולות פיקסלים (getBlueValue, setBlueValue)**: פונקציות המבצעות מניפולציות ברמת הביטים (Bitwise AND / Shift) הן מחלצות את הערוץ הכחול (Blue Channel) בלבד, ושותלות אותו בחזרה לתוך הפיקסל מבלי לדרוס את ערוצי השקיפות, האדום והירוק.

• **פעולות המרה (textToBits, bitsToText)**: המרת מחרוזת טקסט למערך ביטים חכם (BitSet) תוך שימוש בקידוד UTF-8 כדי להבטיח תמיכה בשפות כמו עברית, וכן פעולות הפוכות לחילוך המסר המדויק.

F5Utils מחלקת

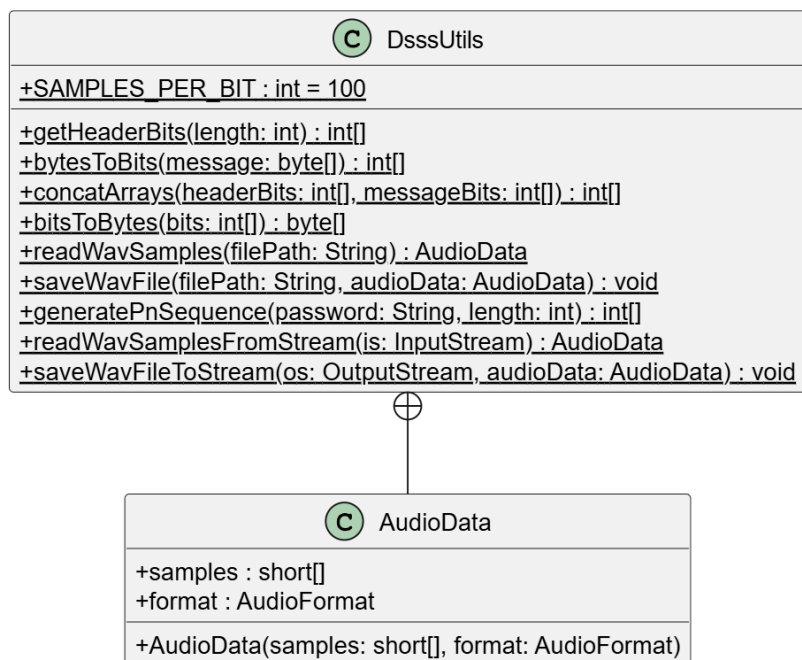
 «@Document(collection="Files")» Files
-«@Id» id : String -userId : String -actionType : String -timestamp : Date -mediaType : String -imageData : byte[] -isDeleted : boolean
+Files() +Files(userId: String, actionType: String, mediaType: String, imageData: byte[]) +getId() : String +setId(id: String) : void +getUserId() : String +setUserId(userId: String) : void +getActionType() : String +setActionType(actionType: String) : void +getTimestamp() : Date +setTimestamp(timestamp: Date) : void +getMediaType() : String +setMediaType(mediaType: String) : void +getImageData() : byte[] +setImageData(imageData: byte[]) : void +isDeleted() : boolean +setDeleted(deleted: boolean) : void

מחלקת F5Utils משמשת כתשתית להמרות בינאריות עבור אלגוריתם הסטגנוגרפיה F5. היא מטפלת בשבירת הודעות למערכי ביטים (0 ו-1) ובפעולות מניפולציה ייעודיות על מקדמי ה-DCT של תמונות JPEG.

תכונות ובנאים (Attributes & Constructors): מחלקה זו הינה נטולת מצב (Stateless) לחלוטין ואינה מכילה משתנים גלובליים או בנאים. היא מספקת פונקציות שירות טהורות (Pure Functions) לשימוש על ידי שכבת ה-Service.

מתודות (Methods)

- **getHeaderBits(length) : int[]**: מייצרת מערך המייצג ב-32 ביטים (Header) את אורך המסר הכולל.
- **bytesToBits** ו- **bitsToBytes**: פונקציות המבצעות המרה דו-כיוונית מורכבת שבירת כל Byte למערך של 8 ביטים בודדים, ודחיפת מערך ביטים בחזרה לתוך משתני Byte בעזרת פעולות Bitwise OR-לוגי. הזזה שמאלה ו-לוגי.
- **concatArrays(...): int[]**: מאחדת את ה-Header עם גוף ההודעה למערך עבודה רציף אחד המיועד להטמעה.
- **setLSB(value, targetBit) : int**: הפונקציה המרכזית במחלקה, המיישמת את עקרון הכיווץ (Shrinkage) של F5 בניגוד לאלגוריתמי LSB קלאסיים שדורסים את הביט, פונקציה זו תמיד מקטינה את הערך המוחלט של מקדם ה-DCT (מקרבת אותו לאפס) כדי להתאים את הביט הפחות משמעותי ליעד, מה ששומר על איכות התמונה.

DsssUtils מחלקת

מחלקת **DsssUtils** מספקת כלים חיוניים להטמעה וחילוץ של מידע מקבצי שמע (Audio). המחלקה מתמודדת עם המורכבות של קריאת קבצי WAV ברמת ה-Byte, המרתם לדגימות 16-bit של גלי קול מתמטיים, ויצירת מפתחות פריסה (Pseudo-Noise).

מבני נתונים פנימיים (Inner Classes)

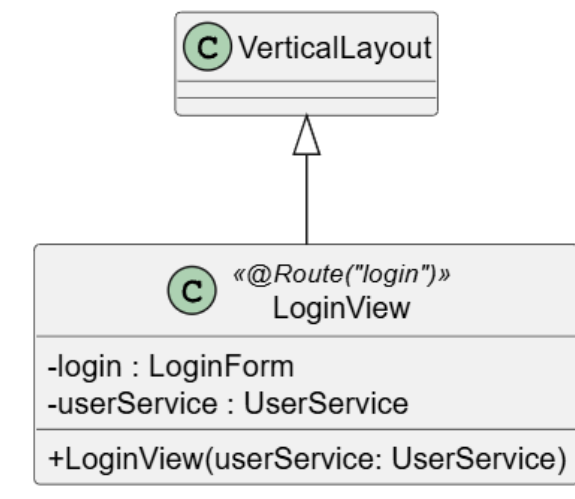
- **AudioData**: מחלקת מעטפת (Wrapper Class) המשמשת כדי להעביר את דגימות האודיו יחד עם המטא-דאטה החיוני שלהן) כמו תדר הדגימה (Sample Rate - בלעדיה, לא ניתן היה לבנות את הקובץ מחדש בצורה חוקית לאחר ההצפנה).

תכונות (Attributes)

- **SAMPLES_PER_BIT**: קבוע המגדיר על פני כמה דגימות שמע יפוזר כל ביט בודד של המידע (Spread). ערך זה (100) קובע את החוסן של ההצפנה מפני רעשי רקע, אל מול הקיבולת הכללית של הקובץ.

מתודות (Methods)

- **המרה ועיבוד בינארי**: פונקציות דומות לאלו של F5 כגון `getBytesToBits`, `getHeaderBits` ו-`bytesToBits` של 16 ביט.
- **קריאה וכתובה של שמע** (`readWavSamplesFromStream`, `saveWavFileToStream`): פונקציות המיישמות ארכיטקטורת In-Memory במקום לקרוא קבצים מהדיסק הקשיח, הפונקציות קוראות `InputStream`, מפרקות זוגות של בייטים (Little Endian) לדגימת קול שלמה מסוג `short` ובסיום מרכיבות אותן חזרה לזרם פלט חוקי של WAV.
- **generatePnSequence(password, length) : int[]**: מייצרת רצף פסאודו-אקראי (PN) של 1 ו-(-1). הרצף דטרמיניסטי ותלוי לחלוטין בגרעין אקראיות (Seed) הנגזר מסיסמת המערכת. רצף זה הכרחי לשלב הפיזור (Spreading) והחילוץ על ידי חישוב קורלציה.

מחלקת LoginView

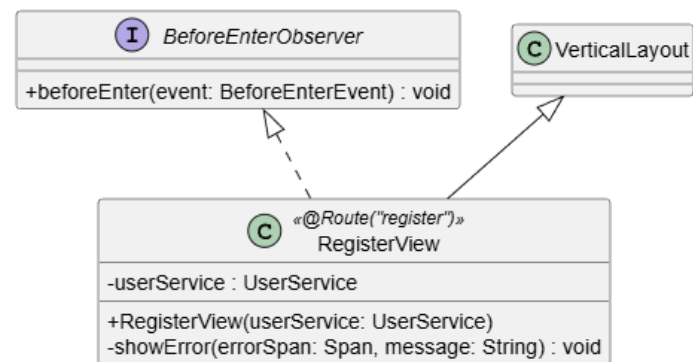
מחלקת LoginView מהווה את שער הכניסה (Authentication Gateway) של המערכת. היא מקושרת לנתיב הרשת `/login` באמצעות האנוטציה `<<@Route>>` בניגוד למערכות סטנדרטיות המסתמכות על ניתוב טפסים של Spring Security, תצוגה זו מממשת תהליך אימות מותאם אישית (Custom Auth Flow) המבוסס על אירועים (Event-Driven) כדי להתאים לארכיטקטורת ה-SPA של המערכת.

תכונות וירושה (Attributes & Inheritance)

- **ירושה מ VerticalLayout:** המחלקה יורשת את תכונות הפריסה האנכית מ Vaadin-מה שמאפשר לישור ומרכז קל של אלמנטים על המסך (Flexbox).
- **login : LoginForm (Private):** רכיב UI מוכן מראש של Vaadin המספק תבנית רספונסיבית ובטוחה לקליטת שם משתמש וסיסמה.
- **userService : UserService (Private):** אובייקט השירות המוזרק לצורך ביצוע הלוגיקה העסקית של אימות מול מסד הנתונים.

בנאים ומתודות (Constructors & Methods)

- **LoginView(userService)** בנאי המחלקה המיישם הזרקת תלויות ומגדיר את ממשק המשתמש. הלוגיקה המרכזית נמצאת בתוך מאזין האירועים (addLoginListener). בעת שליחת הטופס, המערכת מפעילה את פונקציית ה-authenticate של שכבת השירות. במידה והאימות מצליח, נשמר אובייקט המשתמש כ"אסימון גישה" בתוך ה-VaadinSession והמשתמש מנותב לעמוד הבית בצורה חלקה (UI.getCurrent().navigate). במקרה של כישלון, מודלק דגל שגיאה ויזואלי ברכיב הטופס להתרעה בפני המשתמש.

מחלקת RegisterView

מחלקת RegisterView אחראית על קליטת משתמשים חדשים למערכת. היא מבצעת אימות קלט ראשוני בצד הלקוח (Client-Side Validation) לפני פנייה לשרת, כדי לחסוך משאבי מערכת. ייחודה של המחלקה הוא במימוש ממשק BeforeEnterObserver שהופך אותה ל"שומר סף הפוך" (Reverse Auth Guard) היא מונעת ממשתמשים שכבר מחוברים למערכת לגשת לדף ההרשמה בטעות או במכוון.

תכונות וירושה (Attributes & Inheritance)

- ירושה מ VerticalLayout ומימוש BeforeEnterObserver : שילוב בין פריסה ויזואלית לבין יכולות האזנה למחזור החיים של טעינת דפים ב Vaadin
- **(private) userService : UserService** : רכיב השירות המנהל את שמירת המשתמש החדש.

בנאים ומתודות: (Constructors & Methods)

- **RegisterView(userService)** הבנאי בונה את שדות הקלט (כולל שדות סיסמה חסויים מסוג PasswordField), ומגדיר את כפתור ההרשמה הראשי. אירוע הלחיצה על הכפתור מבצע תהליך סניטציה (ניקוי רווחים) בעזרת trim, מוודא התאמת סיסמאות, ושולח את הנתונים לשמירה. במידה ושם המשתמש תפוס, מוצגת הודעת שגיאה ממוקדת.
- **(Private) showError(errorSpan, message) : void** : מתודת עזר (Helper) המרכזת את לוגיקת הצגת השגיאות הוויזואלית, במטרה למנוע שכפול קוד ולשמור על עקרון DRY (Don't Repeat Yourself).
- **beforeEnter(event) : void** : מתודה המופעלת רגע לפני רינדור הדף. היא בודקת אם קיים ערך user בuser בנכח. אם כן, היא מבצעת הפניה שקטה (Forward) לעמוד הבית, ובכך חוסמת את הגישה לדף ההרשמה עבור משתמשים מחוברים.

מחלקת MainLayout

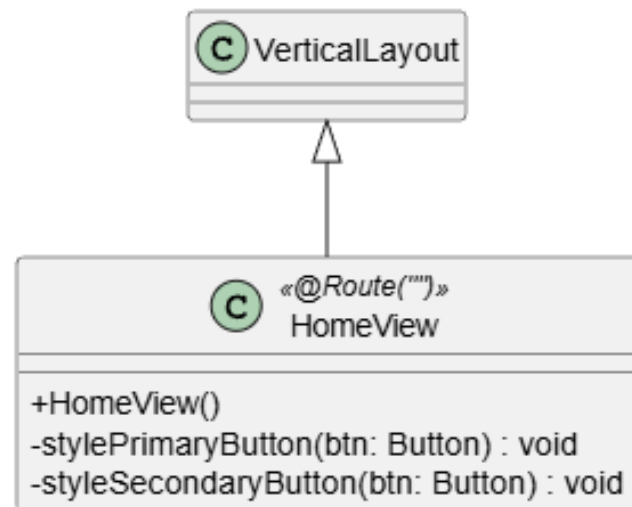
מחלקת MainLayout מתפקדת כמעטפת הגלובלית (Shell Component) של המערכת, ויורשת מרכיב ה AppLayout המובנה. כל תצוגה פנימית במערכת "מוזרקת" אל תוך מעטפת זו. המחלקה חוסכת שכפול קוד של התפריט העליון (Navbar) בין עמודים שונים, ואחראית גם על הזרקת העיצוב הגלובלי (Dark Theme) המעניק למערכת אסתטיקת סייבר-פאנק כהה (Cyberpunk Aesthetic) בעזרת הגדרות CSS מתקדמות לפריסת רשת (Grid Pattern) וניגודיות צבעים.

תכונות וירשה (Attributes & Inheritance)

- **ירשה מ AppLayout:** תבנית מבנית של Vaadin המספקת אזורים מוגדרים מראש עבור תפריט עליון ואזור תוכן דינמי שמתחלף בעת ניווט.

בנאים ומתודות (Constructors & Methods)

- **MainLayout():** בנאי המחלקה המאתחל את העיצוב הגלובלי באמצעות קריאות למנוע ה DOM של Vaadin, ומפעיל את הפונקציה הבונה את התפריט העליון.
- **void createHeader() (Private):** מתודה המיישמת רנדור מותנה (Conditional Rendering). היא קוראת את סטטוס המשתמש מתוך ה VaadinSession-ובונה את התפריט בהתאם:
 - **עבור משתמש אורח (Unauthorized):** מוצגים כפתורי כניסה והרשמה.
 - **עבור משתמש מחובר (Authorized):** מוצג תפריט הניווט הפנימי (תיבות דואר, היסטוריה, כלי ההצפנה), וכן כפתור "התנתק". פעולת ההתנתקות מלווה בלוגיקת אבטחה קריטית של השמדת הסשן בשרת (invalidate) וביצוע רענון קשיח (Hard Refresh) לדפדפן כדי לנקות שאריות זיכרון ולמנוע מתקפות השתלטות על סשנים (Session Hijacking).

מחלקת HomeView

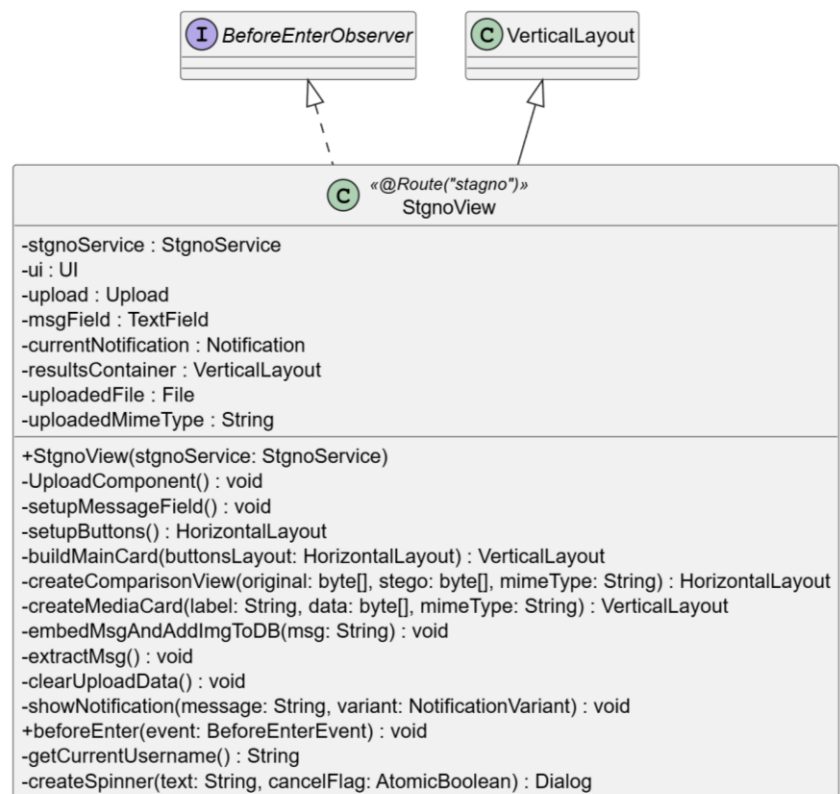
מחלקת HomeView משמשת כדף הבית וצומת הניווט המרכזי (Dashboard / Hub) של המערכת. המחלקה מקושרת לנתיב הבסיס (Root Route). עיצוב הממשק מאופיין בעיצוב סייבר הממומשת כולה באופן פרוגרמטי דרך Java Style API, ללא תלות בקבצי CSS חיצוניים.

תכונות וירשה (Attributes & Inheritance)

- **ירשה מ VerticalLayout:** המחלקה יורשת תכונות פריסה אנכית המאפשרות בנייה רספונסיבית מהירה מבוססת Flexbox של אלמנטי המסך.
- המחלקה אינה מחזיקה בשדות גלובליים (Stateless), שכן כל הנתונים (כמו שם המשתמש) נמשכים דינמית מתוך ה VaadinSession בעת טעינת הדף.

בנאים ומתודות (Constructors & Methods)

- **HomeView():** הבנאי מרכיב את עץ ה DOM של המסך: הוא קורא את נתוני המשתמש מהסשן ומציג הודעת ברכה דינמית, מייצר טקסטים מסבירים בעיצוב Glassmorphism (אפקט של זכוכית מטושטשת מעל תמונת רקע), ומרכיב את רשת כפתורי הניווט המפעילים ניווט צד-לקוח (SPA).
- **(Private) stylePrimaryButton / styleSecondaryButton:** מתודות עזר המייצרות חווית משתמש (UX) אינטראקטיבית. הן מזריקות הגדרות עיצוב מתקדמות לכפתורים ורושמות מאזיני אירועים (Event Listeners) לעכבר (mouseover, mouseout). גישה זו מאפשרת ליצור אפקטים דינמיים (כגון הצללות זוהרות ואנימציות הגדלה) ישירות מתוך קוד ה Java.

מחלקת StegnoView

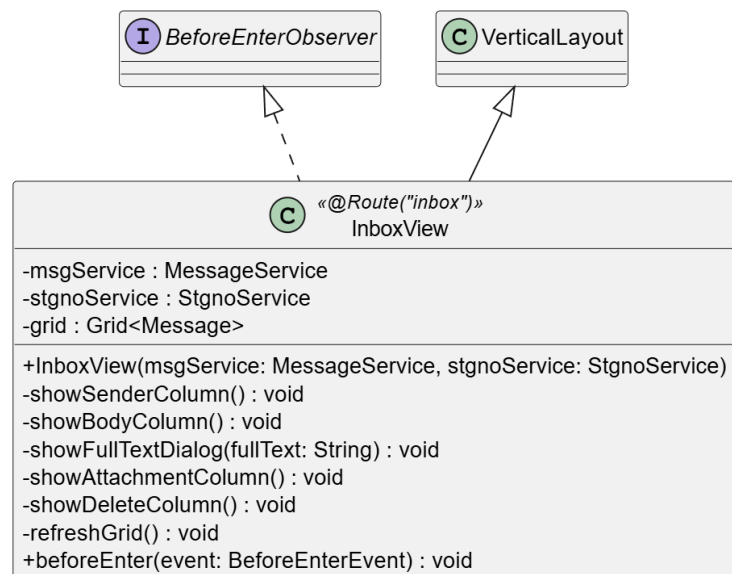
StegnoView היא זירת הפעולה המרכזית של המערכת. היא מספקת ממשק משתמש להעלאת קבצים, הקלדת מסרים סודיים, והפעלת מנועי הסטגנוגרפיה (הטמעה וחילוף). המחלקה מיישמת ארכיטקטורה אסינכרונית כדי למנוע קפיאה של הדפדפן במהלך חישובים מתמטיים כבדים (כמו הצפנת DSSS).

תכונות וירשה (Attributes & Inheritance)

- **ירשה ומימושים:** יורשת מ VerticalLayout ומממשת BeforeEnterObserver כ"שומר סף" (מונעת גישה אורחים).
- **stegnoService : StgnoService:** שירות הלוגיקה העסקית המוזרק למחלקה.
- **ui : UI:** שומר מופע מדויק של ה UI הנוכחי. שדה זה **קריטי** מכיוון שפעולות ההצפנה רצות ב Thread נפרד ברקע. גישה אליו באמצעות ui.access() מאפשרת לדחוף בבטחה עדכונים לדפדפן כמו סגירת ה"ספינר" והצגת התוצאה מבלי לגרום להתנגשויות חריגה (IllegalStateException).
- **uploadedFile / uploadedMimeType:** ניהול הסטטוס (State) של הקובץ שהמשתמש העלה כעת.

מתודות עיקריות (Key Methods)

- **UploadComponent():** מאתחלת את רכיב העלאת הקבצים. משתמשת ב UploadHandler חכם ששומר את הקובץ בזיכרון זמני (Temp File) בדיסק במקום ב RAM-ומגנה על השרת ממתקפות הצפה (הגבלת משקל ל-15 MB).
- **embedMsgAndAddImgToDB / extractMsg:** מתודות הליבה. הן חוסמות את המסך עם חלונית טעינה מודאלית, שולחות את הקובץ לעיבוד אסינכרוני בשכבת ה Service-, ומתמודדות עם תגובת ה Callback-.
- **createMediaCard:** מתודה המייצרת תצוגת השוואה ("לפני ואחרי"). היא משתמשת בטכניקת **Data URI Base64** כדי להזרים את התמונה או האודיו המוצגים ישירות לדפדפן, ללא יצירת קבצים פיזיים קבועים על כונן השרת, מה שמבטיח פרטיות מרבית למשתמש.

InboxView מחלקת

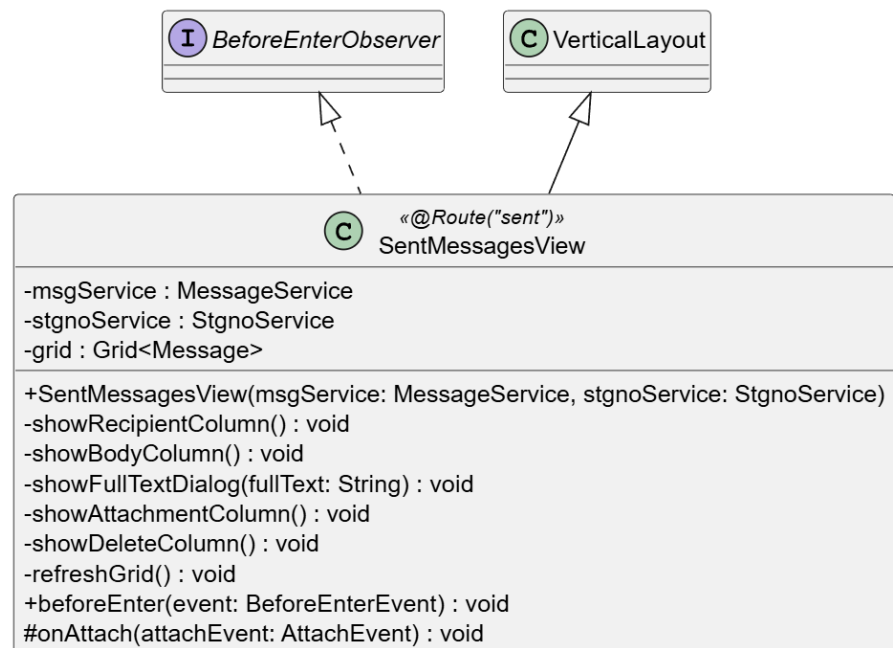
מחלקת InboxView אחראית על ניהול והצגת תיבת הדואר הנכנס של המשתמש. היא מרכזת הצגה טבלאית (Grid) של ההודעות הגלויות והנסתרות, ומספקת ממשק מהיר למחיקה ולכתיבת הודעות חדשות דרך הפעלת Dialog חיצוני.

תכונות וירשה (Attributes & Inheritance)

- **ירשה ומימושים:** יורשת את תכונות הפריסה (Layout) וממשת אבטחת ניתוב . BeforeEnterObserver
- **שירותים מוזרקים:** תלויה ב MessageService (לשליפת נתוני דואר) ו StgnoService שמועבר הלאה לכפתור חילוץ המדיה במידת הצורך.
- **grid : Grid<Message>:** רכיב טבלה מתקדם וגמיש של Vaadin. הוגדר ללא יצירת עמודות אוטומטית (false), כדי לאפשר שליטה הנדסית מלאה ברנדור של כל תא.

מתודות (Methods)

- **refreshGrid():** פונה ל Service למשיכת ההודעות מהמסד (מסוננות לפי הנמען – המשתמש הנוכחי) ומאכלסת את הטבלה.
- **showBodyColumn():** בונה עמודה חכמה לטקסט גלוי. אם תוכן ההודעה ארוך מ-30 תווים, היא חותכת אותו באופן אוטומטי לטובת תצוגה נקייה, ומאפשרת למשתמש ללחוץ על הטקסט המקוצר כדי לפתוח חלונת מודאלית (showFullTextDialog) הכוללת אזור גלילה (TextArea) לקריאה נוחה.
- **showDeleteColumn():** יוצרת כפתור מחיקה לכל שורה. פעולת המחיקה מיושמת בצורה מיטבית היא נפתחת ב Thread נפרד ברקע כדי למחוק את הרשומה ממסד הנתונים (מבלי להקפיא את המסך), ולאחר מכן משתמשת ב UI.access כדי לרענן את הטבלה ולהעלים את השורה מהעין בצורה חלקה ומאובטחת.

מחלקת SentMessagesView

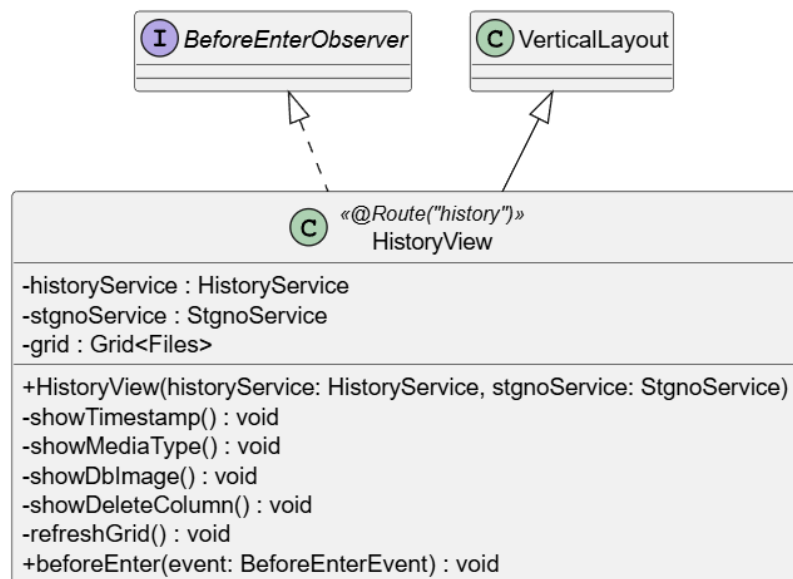
מחלקת SentMessagesView אחראית על ניהול והצגת תיבת "הדואר היוצא" של המערכת. היא דומה במבנה שלה לתיבת הדואר הנכנס, אך מציגה את ההודעות תוך סינון לפי השולח. תצוגה זו כוללת מנגנון **Polling** (דגימה מחזורית) המרענן את הטבלה אוטומטית.

תכונות וירושה (Attributes & Inheritance)

- המחלקה יורשת מ VerticalLayout וממשת BeforeEnterObserver לחסימת משתמשים לא מזוהים.
- נעזרת בשירותי ההודעות והסטגנוגרפיה, ומנהלת טבלת נתונים מסוג Grid<Message>.

בנאים ומתודות (Constructors & Methods)

- **showAttachmentColumn: ishowBodyColumn**: עמודות המיישמות תצוגה חכמה. טקסט ארוך נחתך ומוצג בחלונית מודאלית בלחיצה, וקובץ מצורף מוצג באמצעות רכיב מותאם אישית לחילוץ ותצוגה.
- **showDeleteColumn()**: מיישמת מחיקה אסינכרונית עם תצוגה אופטימית (Optimistic UI). המחיקה מתבצעת בשרת ברקע כדי לא להקפיא את הממשק, ולאחר ההצלחה מתעדכן ה-DOM.
- **onAttach(attachEvent)**: מתודה ייחודית הנדרסת (Overridden) ממחזור החיים של Vaadin. ברגע שהמסך מתחבר לדפדפן הלקוח, מופעל מאזין הדוגם את השרת (Polling) כל 15 שניות לצורך רענון אוטומטי של הטבלה, כך שהמשתמש רואה עדכונים בזמן אמת ללא צורך בריענון ידני.

מחלקת HistoryView

מחלקת HistoryView מספקת ממשק מעקב מקיף עבור כלל הקבצים שהמשתמש עיבד במערכת. היא מאפשרת צפייה מקדימה במדיה, הורדה מחדש, חילוך מסרים מקבצים ישנים, וניהול ארכיון חכם.

מתודות מרכזיות (Key Methods)

- **refreshGrid()**: פונה ל historyService למשיכת קבצי ההיסטוריה **הפעילים בלבד** (ללא אלו שנמחקו לוגית), וללא תוכן הבייטים עצמו כדי לחסוך בזיכרון (Optimization).
- **showDbImage()**: עמודת התצוגה המקדימה. היא מזריקה לטבלה את רכיב MediaPreviewButton, המבצע שליפה אמיתית של הקובץ ממסד הנתונים אך ורק כשהמשתמש דורש זאת באופן פעיל (On Demand).
- **showDeleteColumn()**: מנהלת מחיקה בטוחה הכוללת דיאלוג אישור. מחיקה זו אינה מוחקת פיזית את הקובץ ממסד הנתונים, אלא מבצעת מחיקה לוגית (Soft Delete) על ידי שינוי דגל isDeleted=true, תוך עדכון הדרגתי ובטוח של הממשק מתוך תהליכון רקע.

NewMessageDialog מחלקת

C NewMessageDialog
-messageService : MessageService -stegoService : StgnoService -afterSendAction : Runnable
+NewMessageDialog(messageService: MessageService, stegoService: StgnoService, afterSendAction: Runnable) -buildWindow() : void

NewMessageDialog הוא רכיב חלון מודאלי (Modal) עצמאי המשמש ליצירת הודעות חדשות מכל מקום במערכת. הרכיב תומך במשלוח הודעות טקסט רגילות או צירוף קבצים והטמעת מסרים סודיים עליהם.

תכונות וירשה: (Attributes & Inheritance)

- **ירשה מ Dialog:** רכיב קופץ של Vaadin המוצג מעל תוכן הדף הנוכחי.
- **afterSendAction : Runnable (Private):** פונקציית Callback המאפשרת למחלקה שפתחה את הדיאלוג (כגון טבלת הדואר היוצא) לרענן את עצמה מיד עם סיום המשלוח.

בנאים ומתודות (Constructors & Methods)

- **void : buildWindow():** בונה את פנים החלון. גולת הכותרת כאן היא מנגנון ה **שגר ושכח** בלחיצה על כפתור "שלח". במקום לגרום למשתמש להמתין למסך טוען ארוך (שכן הצפנת קבצים דורשת זמן חישוב ממושך), החלון נסגר באופן מיידי, ופעולת הקידוד והשמירה נפתחת בתהליכון (Thread) עצמאי ברקע. בסיום ההצפנה והשמירה בשרת, המערכת משתמשת בפקודת UI.push() כדי "לדחוף" באופן אקטיבי את הודעת ההצלחה (Notification) ישירות לדפדפן של הלקוח, ללא בקשה יזומה מצידו (WebSocket).

מחלקת MediaPreviewButton

C MediaPreviewButton
-steganographyService : StgnoService
+MediaPreviewButton(buttonText: String, fileSupplier: Supplier<Files>, stgnoService: StgnoService)
-openMediaDialog(attachedFile: Files) : void
-convertBytesToDataUri(fileData: byte[], mimeType: String) : String
-createMediaElement(mimeType: String, dataUri: String) : Component
-createDownloadLink(attachedFile: Files, mimeType: String, dataUri: String) : Anchor
-addExtractionTools(layout: VerticalLayout, mediaElement: Component, fileData: byte[], mimeType: String) : void
-addRegularFileIndication(layout: VerticalLayout, mediaElement: Component) : void
-startExtractionProcess(fileData: byte[], mimeType: String) : void
-showExtractionResult(isSuccess: boolean, secretMessage: String, loadingSpinner: Dialog) : void
-createLoadingSpinner(messageText: String) : Dialog

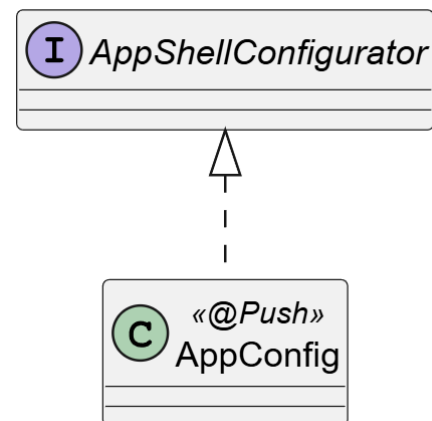
מחלקת MediaPreviewButton היא רכיב כפתור מותאם אישית (Custom Component) מהמתקדמים במערכת. היא נועדה לפתור את בעיית העומס על הרשת ועל הזיכרון בעת טעינת תיבות דואר או היסטוריית קבצים מרובות תמונות. המחלקה אחראית על שליפה נקודתית של המדיה, בניית דיאלוג תצוגה וניהול פעולות כגון חילוץ והורדה.

תכונות וירושה (Attributes & Inheritance)

- ירושה מ Button משמשת ויזואלית ותפקודית ככפתור לחיץ.
- fileSupplier : Supplier<Files>: שימוש בתבנית עיצוב פונקציונלית. הכפתור מקבל "הוראה איך לשלוף את הקובץ", ומוציא אותה לפועל רק בעת לחיצה (Lazy Loading).

מתודות (Methods)

- לחיצה על הכפתור פותחת ספינר מודאלי עם אופציית ביטול (AtomicBoolean). היא פונה אסינכרונית ל DB, ואם הלחיצה לא בוטלה, היא מפעילה את openMediaDialog.
- createMediaElement ו convertBytesToDataUri: ממירות את המידע הבינארי ישירות למחרוזת Base64 וטוענות אותו לתגית או <audio> של HTML5, ללא יצירת קבצים פיזיים בשרת.
- addExtractionTools: משלבת לחצנים וכן תפריט הקשר (Context Menu) בלחיצה על כפתור ימני בעכבר מעל המדיה, כדי לאפשר חילוץ סטגנוגרפי מהיר אל תוך חלונית צפה.

מחלקת AppConfig

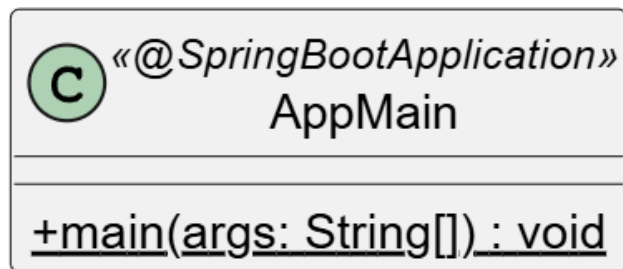
מחלקת AppConfig משמשת כרכיב התצורה המרכזי של מעטפת אפליקציית ה Web ב Vaadin. הפרדת רכיב זה ממחלקת האתחול מאפשרת לנהל את התנהגות הדפדפן, מחזור החיים של חלון האפליקציה (App Shell), וטכנולוגיות התקשורת של הממשק תחת מחלקה ייעודית אחת.

תכונות ואנוטציות (Attributes & Annotations)

- **מימוש AppShellConfigurator:** המחלקה מיישמת ממשק זה כדי לקבל הרשאה לנהל את הגדרות המעטפת הטכניות של ממשק המשתמש (כגון הגדרות מטא-דאטה, כותרות, או טעינת אלמנטים גלובליים).
- **<<@Push>>:** אנוטציה ארכיטקטונית קריטית עבור מערכות זמן אמת אסינכרוניות. אנוטציה זו מפעילה ארכיטקטורת תקשורת דו-כיוונית רציפה (Server Push) בין שרת ה Java-לבין דפדפן הלקוח, המבוססת על פרוטוקול **WebSockets**. יכולת זו חיונית עבור המערכת, שכן היא מאפשרת למחלקות התצוגה והדיאלוגים (כמו מסך ההצפנה או חלונית כתיבת ההודעה) לקבל עדכון ויזואלי מיידי ויזום מהשרת ברגע שתהליכוני הרקע (Threads) מסיימים פעולות כבדות של אלגוריתמי סטגנוגרפיה, ללא צורך בפעולת ריענון (Refresh) מצד המשתמש.
- למחלקה זו אין שדות גלובליים משלה, שכן היא משמשת כקובץ הגדרות תשתיתי בלבד.

בנאים (Constructors & Methods)

- המחלקה משתמשת בבנאי ברירת מחדל ריק לצורך קריאה ורישום על ידי מנוע האתחול של Vaadin בזמן עליית האפליקציה.

מחלקת AppMain

מחלקת AppMain מהווה את נקודת הכניסה הראשית (Entry Point) של המערכת. תפקידה הבלעדי הוא להניע את תהליך האתחול של האפליקציה כולה, להקים את התשתיות הנדרשות ולשחרר את השליטה לשילוב של Vaadin ו-Spring Boot.

תכונות ואנוטציות: (Attributes & Annotations)

- **<<@SpringBootApplication>>**: אנוטציית המעטפת המרכזית של התשתית. אנוטציה זו משלבת שלוש יכולות מובנות: הפעלת מנגנון ההגדרות האוטומטיות (Auto-Configuration), סריקת רכיבים גלובליים (Component Scan) ברחבי ה-Packages כדי לאתר את כל רכיבי ה-Services, Repositories ו ה-Views, וכן הגדרת המחלקה כמקור מרכזי לניהול הזרקת תלויות.
- המחלקה אינה מחזיקה בשדות גלובליים (Attributes), שכן היא מתפקדת כרכיב אתחול סטטי בלבד.

מתודות (Constructors & Methods)

- **main(args: String[]): void**: פונקציית הצירוף הראשית של ריצת ה-Java Virtual Machine (JVM) המתודה מפעילה את הפקודה SpringApplication.run, אשר טוענת את מכולת ה-loC, מפעילה את שרת ה-Tomcat הפנימי, ומאפשרת למערכת להתחיל להאזין לבקשות תקשורת.

מחלקת SecurityConfig

מחלקת SecurityConfig אחראית על הגדרות אבטחת המידע וניהול ההצפנה במערכת. היא מסומנת באנוטציה <<@Configuration>>, המסמנת ל-Spring-שמחלקה זו מייצרת אובייקטים (Beans) שצריכים להישמר בקונטיינר הגלובלי ולהיות זמינים להזרקה (Dependency Injection) ברחבי האפליקציה.

מתודות (Methods)

- **passwordEncoder(): PasswordEncoder**: מתודה המייצרת ומחזירה מופע של BCryptPasswordEncoder. מתודה זו מסומנת באנוטציה <<@Bean>>. הבחירה באלגוריתם BCrypt היא קריטית לאבטחת המערכת, שכן הוא אינו מסתפק רק בפונקציית גיבוב (Hashing) חד-כיוונית, אלא מוסיף "מלח" (Salt) "אקראי לכל סיסמה ומבצע מספר רב של סבבי חישוב. פעולה זו מאטה במכוון את תהליך הגיבוב, ובכך מחסנת את מסד הנתונים של המערכת) אוסף ה-Users מפני התקפות כוח גס (Brute-Force) והתקפות טבלאות צבעוניות (Rainbow Tables). האובייקט המיוצר כאן מוזרק ישירות לתוך ה-UserService שראינו בשכבת הלוגיקה העסקית, לביצוע אימות והרשמה בטוחים.

12.3 מבני נתונים בהם נעשה שימוש

1. מבנה נתונים `byte[]` :

מערך חד-ממדי של בייטים (`Byte Array`) בגודל קבוע.

מיקום השימוש:

- `Files.java`
- `DsssUtils.java`
- `F5Utils.java`
- `StegnoView.java`

משמש לצורך אחסון הנתונים הבינאריים הגולמיים (`Raw Data`) של קבצי המדיה לשמירה בתוך מסד הנתונים. בנוסף, הוא משמש לייצוג מחרוזת המסר הסודי לאחר המרתה מטקסט, כדי לאפשר את פירוקה לביטים בודדים לקראת הטמעה באלגוריתמים השונים.

2. מבנה נתונים `short[]` :

מערך חד-ממדי של מספרים שלמים קצרים (`Short Array - 16 bit`).

מיקום השימוש:

- `DsssUtils.java`

משמש לצורך שמירה ועיבוד של דגימות שמע (`Audio Samples`) מתוך קובץ `WAV`. סוג הנתונים `short` נבחר מכיוון שדגימות שמע סטנדרטיות מקודדות ב-16 ביט, מה שמאפשר ייצוג מתמטי מדויק בזיכרון וביצוע חישובים אלגבריים לעיבוד אותות באלגוריתם `DSSS`.

3. מבנה נתונים `int[]` :

מערך חד-ממדי של מספרים שלמים (`Integer Array`).

מיקום השימוש:

- `DsssUtils.java`
- `F5Utils.java`

משמש לצורך מספר מטרות:

1. ייצוג רצף ביטים של המסר וה-Header (מערכים המכילים רק 0 ו-1) לצורך פעולות ההטמעה.

2. ייצוג רצף פסאודו-אקראי (`PN Sequence`) באלגוריתם ה-`DSSS` המורכב מ-1 ו-(-1) לצורך פיזור המידע על פני הדגימות.

4. מבנה נתונים `int[][]` :

מערך דו-ממדי (`2D Array / Matrix`).

מיקום השימוש:

- `PvdUtils.java`

משמש לצורך ייצוג טבלאות ומטריצות סטטיות. באלגוריתם `PVD` הוא משמש לייצוג טבלת טווחי הקוונטיזציה (`RANGE_TABLE`), אשר מגדירה עבור כל טווח של הפרשי פיקסלים מהו הגבול העליון, הגבול התחתון, וכמה ביטים (`Capacity`) ניתן להטמיע בו.

5. מבנה נתונים BitSet :

מערך סיביות דינמי וחכם (java.util.BitSet).

מיקום השימוש:

- PvdUtils.java

לאיזה צורך הוא משמש? ניהול חסכוני ויעיל של סיביות (Bits) בזיכרון, ללא בזבז של בתים שלמים. משמש באלגוריתם PVD להמרת מחרוזות טקסט של המשתמש בקידוד UTF-8 לרצף בינארי טהור לצורך פיצול והטמעה בפיקסלים, וכן לתרגום הביטים שחולצו בחזרה לטקסט קריא.

6. מבנה נתונים ArrayList :

רשימה דינמית (Dynamic List).

מיקום השימוש:

- UserService.java

משמש לצורך ניהול אוספי נתונים שגודלם משתנה בזמן ריצה. במקרה זה, הרשימה משמשת לשליפה ולאחסון של רשימת כלל המשתמשים (User) המוחזרת ממסד הנתונים.

7. מבנה נתונים: ישויות מסד נתונים (Entity / POJO)

מחלקות מודל נתונים (Domain Models) עם אנוטציה @Document.

מיקום השימוש:

- Files.java
- IMessage.java
- User.java

משמש לצורך משמש כ"תבנית נתונים" למיפוי אובייקטים (ODM) מול מסד הנתונים MongoDB. המבנה מאגד תחת אובייקט לוגי אחד את כל התכונות של ישות במערכת - למשל משתמש (שם וסיסמה מוצפנת), או קובץ (נתונים בינאריים, חותמת זמן, וסוג קובץ) - ומאפשר למערכת לשלוף ולשמור אותם בקלות כיחידה אחת בתוך ה DB

8. מבנה נתונים: מחלקות מעטפת לוגיות (Wrappers)

מחלקת קפסולציה (Custom Object / Static Inner Class)

מיקום השימוש:

- DsssUtils.java

משמש לצורך אריזת מספר פריטי מידע שתלויים זה בזה ליחידה אחת AudioData. מאגד את דגימות השמע הגולמיות יחד עם תבנית הקידוד (AudioFormat) שלהן, כדי לאפשר שחזור מדויק של תדר קובץ ה WAV- המקורי בסיום תהליך ההטמעה, ללא עיוותים בסאונד.

9. מבנה נתונים Grid :

רשת נתונים תצוגתית מורכבת (UI Component / Vaadin Grid).

מיקום השימוש:

- HistoryView.java

משמש לצורך תצוגה טבלאית אינטראקטיבית של אוסף רשומות (היסטוריית הקבצים של המשתמש). המבנה מנהל מאחורי הקלעים אוסף של אובייקטים ומאפשר ביצוע פעולות דינמיות על הנתונים, כגון מיון עמודות והסרת שורות מהתצוגה (בעת מחיקה לוגית) בזמן אמת וללא ריענון הדף.

10. מבנה נתונים AtomicBoolean :

משתנה בוליאני בטוח-הליכונים (Thread-safe Boolean)

מיקום השימוש:

• StegnoView.java

משמש לצורך סנכרון בטוח בין תהליכונים. כאשר מתבצעת פעולת הטמעה ארוכה ברקע (Background Thread) והמשתמש לוחץ על כפתור "ביטול" בממשק, (UI Thread) המשתנה הבוליאני משמש כ"דגל ביטול". מבנה ה Atomic-מבטיח שהקריאה והשינוי יתבצעו באופן מיידי בין התהליכונים השונים וללא התנגשויות או צורך במנגנוני נעילה כבדים.

11. מבנה נתונים Optional<T> :

מעטפת אופציונלית למניעת שגיאות (java.util.Optional)

מיקום השימוש:

• UserService.java בפונקציית האימות.

משמש לצורך התמודדות בטוחה עם ערכים שעשויים להיות חסרים בזיכרון. (Null Safety) בעת ניסיון התחברות, הפונקציה שולפת משתמש מה DB-אך ייתכן שהוא אינו קיים. במקום להחזיר null חשיביל לקריסת המערכת, (NullPointerException) מוחזר אובייקט Optional שכופה על הקוד לבדוק האם הנתון אכן קיים (על ידי שימוש בפונקציה isPresent()) בטרם הגישה אליו.

12.4 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים

1. אלגוריתם PVD להטמעת מידע בתמונות (Spatial Domain)

מחלקה: PVDStegnoService | פעולה: embed

• קוד התוכנית:

```
MainLoop:
for (int y = 0; y < height; y++)
{
    // מחשב הפרשים בין זוגות פיקסלים סמוכים PVD-קפיצות בזוגות, כיוון ש
    for (int x = 0; x < width - 1; x += 2)
    {
        // תנאי סיום: אם החבאנו את כל הביטים הדרושים, אין צורך להמשיך לסרוק את התמונה
        if (bitPointer >= totalBitsLength)
            break MainLoop;

        // של שני הפיקסלים (Blue) קריאת ערכי הצבע בערוץ הכחול
        int p1 = PvdUtils.getBlueValue(image, x, y);
        int p2 = PvdUtils.getBlueValue(image, x + 1, y);

        // PVD בין שני הפיקסלים כפי שנדרש באלגוריתם (d) חישוב ההפרש המוחלט
        int d = Math.abs(p2 - p1);

        // שייך בטבלת הטווחים (למשל: 0-7, 8-15) (d) חיפוש הטווח שאליו ההפרש
        int rangeIndex = PvdUtils.getRangeIndex(d);

        // קביעת מספר הביטים שניתן להחביא בזוג זה בהתבסס על הטווח
        int capacity = PvdUtils.getCapacity(rangeIndex);

        // (יהיה 8 1, לדוגמה: אם הטווח הוא 8-15) מציאת הגבול התחתון של הטווח
        int l = PvdUtils.getRangeStart(rangeIndex);

        // שיוכנס לתמונה (M) משיכת הביטים מההודעה והמרתם לערך עשרוני
        int m = 0;
        for (int i = 0; i < capacity; i++)
        {
            // אם יש עוד ביטים להחביא והביט הנוכחי הוא 1
            if (bitPointer < totalBitsLength && totalBitsToHide.get(bitPointer))
```

```
M בערך i-בינארית כדי להדליק את הביט ה OR פעולת  $m \mid= (1 \ll i)$ ;

// התקדמות למקום הבא ב-"סרט" הביטים הכללי
bitPointer++;
}

// לגבול התחתון (M) על ידי הוספת ערך המסר העשירוני (d') חישוב ההפרש החדש והמוצפן
int newAbsD = 1 + m;

// (d) להפרש המקורי (d') ההפרש בין ההפרש החדש והרצוי (error): חישוב שגיאת ההפרש
int error = newAbsD - d;

// --- תוקן: שגיאת העיגול המתמטי ---
// חלוקת שגיאת ההפרש לשניים. בג'אווה, חלוקת מספר שלילי (למשל -7) ל-2 מחזירה -3 (מאבדת את השארית)
// error!-מחושב כהפרש המדויק מ errorDown, כדי שלא נאבד את הביט הזה
int errorUp = (int) Math.ceil(error / 2.0);
int errorDown = error - errorUp;

int newP1, newP2;

// עדכון הפיקסלים: הרחבת או כיווץ הפער כך שההפרש החדש ייווצר
// החוקיות דורשת שמירה על מי שהיה גדול יותר - יישאר גדול יותר או שווה
if (p1 >= p2)
{
    newP1 = p1 + errorUp;
    newP2 = p2 - errorDown;
}
else
{
    newP1 = p1 - errorUp;
    newP2 = p2 + errorDown;
}

// (קטע טיפול בחריגות מעל 255 או מתחת 0 הוסר לקיצור)

// הטמעת ערכי הפיקסלים המעודכנים (לאחר ההצפנה וההגנה) בחזרה לתוך התמונה
PvdUtils.setBlueValue(image, x, y, newP1);
```

```

        PvdUtils.setBlueValue(image, x + 1, y, newP2);
    }
}

```

- **טענת כניסה:** התמונה המקורית (ממערך בייטים שהומר ל `BufferedImage`) ומערך הביטים של ההודעה הסודית (`totalBitsToHide`).
- **טענת יציאה:** עדכון התמונה בזיכרון, שלאחריו היא מומרת למערך `byte[]` מפורמט PNG.
- **מה מבצעת:** מסתירה מידע בתוך תמונה על ידי שינוי ההפרשים (`Difference`) בין צבעי ערוץ הכחול של זוגות פיקסלים סמוכים.
- **אופן פעולה מפורט:** הלולאה סורקת את התמונה בזוגות פיקסלים. לכל זוג מחושב ההפרש (`d`). לפי גודל ההפרש, האלגוריתם בודק בטבלת טווחים כמה ביטים ניתן להסתיר בו. הביטים נשלפים ומומרים לערך עשרוני שמוסף לגבול התחתון של הטווח ליצירת הפרש חדש (`newAbsD`). האלגוריתם מחשב את שגיאת ההפרש, ומחלק אותה בין שני הפיקסלים כך שההפרש ביניהם ישתנה לערך הרצוי.
- **סיבוכיות זמן ריצה:** $O(W \times H)$ כאשר W רוחב התמונה ו H גובהה. סריקה חד-פעמית עד לסיום הטמעת המסר.

2. אלגוריתם F5 להטמעה במקדמי תדר (Matrix Encoding)

מחלקה: F5StegnoService | פעולה: embed

- קוד התוכנית:

```
// הלולאה תרוץ כל עוד גודל מערך המסר גדול מהמיקום שאנו נמצאים בו
while (msgIndex <= bitsToEmbed.length - 1)
{
    m[0] = bitsToEmbed[msgIndex];
    if (msgIndex + 1 < bitsToEmbed.length)
        m[1] = bitsToEmbed[msgIndex + 1];
    else
        m[1] = 0; // ריפוד במקרה של ביט אחרון בודד

    int[] IdxInDCT_List = new int[3];
    int validCount = 0;

    // חיפוש 3 מקדמים שאינם אפס
    while (validCount < 3 && imgIndex < DCTlist.size())
    {
        if (DCTlist.get(imgIndex) != 0)
        {
            IdxInDCT_List[validCount] = imgIndex;
            validCount++;
        }
        imgIndex++;
    }

    b1 = DCTlist.get(IdxInDCT_List[0]);
    b2 = DCTlist.get(IdxInDCT_List[1]);
    b3 = DCTlist.get(IdxInDCT_List[2]);

    bLsb[0] = F5Utils.getLSB(Math.abs(b1));
    bLsb[1] = F5Utils.getLSB(Math.abs(b2));
    bLsb[2] = F5Utils.getLSB(Math.abs(b3));

    h[0] = bLsb[0] ^ bLsb[2];
    h[1] = bLsb[1] ^ bLsb[2];
}
```



```
if (h[0] != m[0] && h[1] != m[1])
{
    if (b3 > 0) b3 -= 1;
    else b3 += 1;

    if (b3 == 0) Zflag = true;
    DCTlist.set(IdxInDCT_List[2], b3);
}

else if(h[0] != m[0]) // נוריד 1 מהערך של המסר, נוריד 1
    מהערך המוחלט של אותו הביט הראשון
{
    if (b1 > 0) b1 -= 1;
    else b1 += 1;

    if (b1 == 0) Zflag = true;
    DCTlist.set(IdxInDCT_List[0], b1);
}

else if(h[1] != m[1]) // נוריד 1 מהערך של המסר, נוריד 1
    מהערך המוחלט של אותו הביט השני
{
    if (b2 > 0) b2 -= 1;
    else b2 += 1;

    if (b2 == 0) Zflag = true;
    DCTlist.set(IdxInDCT_List[1], b2);
}

if (Zflag)
{
    Zflag = false;
    imgIndex = IdxInDCT_List[0];
    continue;
}

msgIndex += 2;
bitIndex = msgIndex;
}
```

- **טענת כניסה:** רשימת מקדמי תדר ה DCT של התמונה (DCTlist) ומערך הביטים להטמעה (bitsToEmbed).
- **טענת יציאה:** רשימת מקדמי DCT מעודכנת להטמעה בקובץ ה JPEG.
- **מה מבצעת:** מסתירה מידע במרחב התדר של התמונה תוך שימוש בקידוד מטריצה (1,3,2) כדי להחביא 2 ביטים של מסר בתוך 3 מקדמי תמונה.
- **אופן פעולה מפורט:** המערכת מחפשת 3 מקדמי DCT רצופים שערכם אינו 0. היא בודקת את ה LSB שלהם ומחשבת פעולת XOR, אם התוצאה שונה מביטי המסר הרצויים, האלגוריתם מבצע "Shrinkage" תמיד מקטין את הערך המוחלט של המקדם המסוים (מתקרב ל-0). אם נוצר אפס חדש כתוצאה מכך, נדלק דגל (Zflag) והאלגוריתם פוסל את הבלוק ומנסה שוב מההתחלה.
- **סיבוכיות זמן ריצה** $O(B + Z)$ כאשר: B הוא מספר הביטים להטמעה, ו Z הוא מספר הפעמים שנוצר אפס ונדרשה קריאה חוזרת (בדרך כלל זניח).

3. אלגוריתם DSSS להטמעת שמע (Spread Spectrum Adaptive Alpha)

מחלקה: DSSSStegnoService | פעולה: embed

- קוד התוכנית:

```
// המיקום במערך הביטים להטמעה
for (msgIndex = 0; msgIndex < bitsToEmbed.length; msgIndex++)
{
    // בדיקת קיבולת: האם נשאר מספיק מקום בקובץ עבור הביט הבא
    if (sampleIndex + DsssUtils.SAMPLES_PER_BIT >= samples.length)
    {
        System.out.println("Error: Audio capacity too small for this message!");
        return null;
    }

    // --- חישוב אלפא מקומית (Local Alpha) ---
    // מונע אובדן נתונים במוזיקה קצבית (כמו תופים) ע"י התאמת עוצמת ההטמעה לעוצמת הבלוק הספציפי
    long localSum = 0;
    for (int i = 0; i < DsssUtils.SAMPLES_PER_BIT; i++)
    {
        localSum += Math.abs(samples[sampleIndex + i]);
    }
    double localAvg = localSum / (double) DsssUtils.SAMPLES_PER_BIT;

    // קביעת עוצמת ההטמעה (21% מהעוצמה הממוצעת של הבלוק)
    double localAlpha = localAvg * 0.21;

    // הגבלות קצוות: מונע עוצמה חלשה מדי (שתימחק) או חזקה מדי (שתרעיש)
    if (localAlpha < 150) localAlpha = 150;
    if (localAlpha > 3500) localAlpha = 3500;

    // המרת הביט מ-0 לפורמט ביפולרי (-1) לטובת האלגוריתם המתמטי
    int bitToEmbed = bitsToEmbed[msgIndex];
    int bipolarBit = (bitToEmbed == 1) ? 1 : -1;

    // דגימות SAMPLES_PER_BIT הביט הבודד על פני (Spread) פיזור
    for (int i = 0; i < DsssUtils.SAMPLES_PER_BIT; i++)
    {
```

```

// PN * אלפא מקומית * ביט) + הנוסחה המרכזית: דגימה חדשה = דגימה נוכחית
double mod = localAlpha * bipolarBit * pnSequence[sampleIndex];
double newVal = samples[sampleIndex] + mod;

// short-הבטחה שלא נחרוג מגבולות טיפוס ה: (Clipping) הגנת גלישה
if (newVal > Short.MAX_VALUE) newVal = Short.MAX_VALUE;
else if (newVal < Short.MIN_VALUE) newVal = Short.MIN_VALUE;

// שמירת הדגימה המעודכנת
samples[sampleIndex] = (short) newVal;
sampleIndex++;
}
}

```

- **טענת כניסה:** מערך דגימות האודיו של קובץ ה WAV (samples), מערך ביטי המסר, וסיסמה.
- **טענת יציאה:** מערך דגימות (samples) שעבר מניפולציה ומוכן לשמירה.
- **מה מבצעת:** מטמיעה ביטים בתוך גל קול באמצעות טכניקת Direct-Sequence Spread Spectrum תוך התאמה דינמית של העוצמה (Local Alpha).
- **אופן פעולה מפורט:** עבור כל ביט במסר, האלגוריתם מבצע ממוצע של עוצמת הקול בבלוק ספציפי ומגדיר את עוצמת ההטמעה בהתאם (כדי לשרוד מוזיקה קצבית). הביט מומר לערך ביפולרי ומוכפל בעוצמת ההטמעה וברצף פסאודו-אקראי (pnSequence). התוצאה מתווספת לדגימות האודיו, ובכך המסר "מרוח" על פני הבלוק כרעש לבן.
- **סיבוכיות זמן ריצה – $O(B \times S)$** כאשר: B הוא מספר הביטים להטמעה, ו S הוא מספר הדגימות לכל ביט.

13. תיאור מסד הנתונים

ארכיטקטורת ניהול הנתונים של המערכת מבוססת על מסד נתונים מסוג **NoSQL** המנוהל בענן (**MongoDB Atlas**), ומשמש כתשתית האחסון המרכזית. תשתית זו אחראית על ניהול כלל פעולות ה-CRUD (יצירה, קריאה, עדכון ומחיקה) של המערכת, כאשר התקשורת והעברת הנתונים בין צד הלקוח לשרת מתבצעות בפורמט **JSON** הסטנדרטי.

נימוקים לבחירת הטכנולוגיה:

גמישות סכמתית (Schema Flexibility): בניגוד למסדי נתונים רלציוניים (SQL) המחייבים סכימה קשיחה וטבלאות מקושרות בקפידה, MongoDB שומר את המידע כ"מסמכים" (Documents) גמישים. תכונה זו אידיאלית למערכת שלנו, בה אובייקטים (כגון הודעות) יכולים להכיל קבצים מצורפים ולעיתים לא.

תמיכה בנתונים בינאריים (BSON): יתרון עצום של מסד נתונים זה הוא היכולת לשמור אובייקטים בינאריים גדולים (Binary Data) ישירות בתוך המסמך. כך, המערכת שומרת את קבצי התמונה והשמע הנגועים ישירות בענן ללא צורך בשילוב שרתי אחסון חיצוניים (כמו AWS S3).

זמינות ושירות מנוהל בענן: השימוש במסד נתונים מנוהל תורם לזמינות גבוהה (High Availability), גיבויים אוטומטיים ורציפות תפקודית, ללא צורך בתחזוקה פיזית של שרתי האחסון.

לצורך הפרדת רשויות נכונה, מסד הנתונים חולק לשלושה אוספים (Collections) מרכזיים:

• אוסף משתמשים (Users Collection)

אוסף זה מנהל את הרשאות הגישה, נתוני ההתחברות והפרופילים של כלל המשתמשים הרשומים במערכת.

- **UserID** מטיפוס String: מזהה משתמש ייחודי במערכת, מיוצר אוטומטית.
- **Username** מטיפוס String: שם המשתמש. שדה זה הוא ייחודי (Unique) ומשמש גם כגורם מזהה להתחברות ולשליחת/קבלת דואר.
- **Password** מטיפוס String: סיסמת המשתמש. נשמרת אך ורק לאחר גיבוב (Hash) מטעמי אבטחת מידע, למניעת חשיפה במקרה של זליגת נתונים.
- **JoinDate** מטיפוס Date: חותמת זמן של מועד ההרשמה ההיסטורי למערכת.
- **Status** מטיפוס String: סטטוס המשתמש (למשל: פעיל, מושהה), מאפשר ניהול הרשאות עתידי.

• אוסף קבצים (Files Collection)

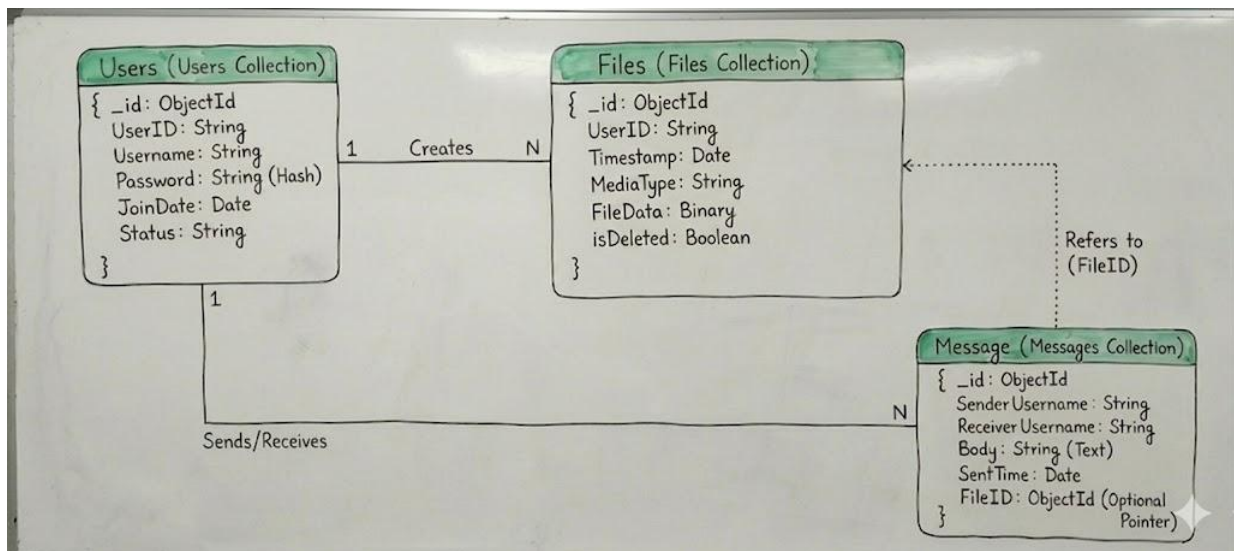
האוסף ה"כבד" ביותר במערכת. הוא מנהל את התיעוד של כל פעולות ההטמעה והחילוץ, ומאחסן פיזית את הקבצים הבינאריים שעובדו כדי לאפשר למשתמש להורידם מחדש מיומן הפעולות (History).

- **FileID** מטיפוס String: מזהה ייחודי של הרשומה והקובץ במערכת.
- **UserID** מטיפוס String: מזהה המשתמש שביצע את הפעולה (קישור לוגי ליוזם).
- **ActionType** מטיפוס String: מצוין האם הקובץ הוא מכיל מסר או לא.
- **Timestamp** מטיפוס Date: חותמת זמן מדויקת המתעדת את מועד העיבוד בשרת.
- **MediaType** מטיפוס String: חותמת הפורמט של המדיה למשל image/png או audio/wav.
- **FileData** מטיפוס byte[]: מערך הבתים של ה Stegno Object עצמו. כאן מאוחסן הקובץ הנגוע פיזית.
- **isDeleted** מטיפוס Boolean: דגל לביצוע "מחיקה לוגית" (Soft Delete). כאשר משתמש "מוחק" קובץ מההיסטוריה, שדה זה הופך ל True הקובץ מוסתר מהמשתמש אך לא נמחק פיזית מהמסד, כדי למנוע שבירה וקריסה של הודעות דואר ישנות המקושרות לאותו קובץ בדיוק.

• אוסף הודעות (Messages Collection)

אוסף זה מהווה את התשתית לערוץ התקשורת הפנימי והחשאי של המערכת (In-App Messaging). הוא מחבר בין המשתמשים השונים ובין הקבצים הנגועים.

- **MessageID** מטיפוס String: מזהה ייחודי של מסמך ההודעה במערכת.
- **SenderUsername** מטיפוס String: שם המשתמש השולח.
- **ReceiverUsername** מטיפוס String: שם המשתמש אליו מיועדת ההודעה. פונקציות ה-Inbox-שולפות הודעות על בסיס התאמה לשדה זה.
- **Body** מטיפוס String: תוכן ההודעה ה"גלוי" (הטקסט המקדים והרגיל שנשלח יחד עם הקובץ).
- **SentTime** מטיפוס Date: חותמת זמן המציינת מתי נשלחה ההודעה.
- **FileID** מטיפוס String: **מצביע (Pointer) לקובץ**. שדה אופציונלי. במידה וצורף קובץ נגוע להודעה, שדה זה יכיל את מזהה ה-FileID של הקובץ מתוך אוסף "היסטוריה וקבצים". דרך מזהה זה, המערכת מאחזרת את הקובץ המצורף בזמן אמת כשנמנע לוחץ על "הצג מדיה".



14. מדריך למשתמש

פרק זה מפרט את אופן השימוש במערכת **Stegno Mail**, החל משלב הכניסה ועד לניהול שוטף של תיבת הדואר והצפנת הקבצים.

- **רישום והתחברות למערכת (Register / Login):** בכניסה הראשונה לאתר, המשתמש מופנה למסך ההתחברות. משתמש חדש ילחץ על קישור "הרשמה" (Register) ויזין שם משתמש וסיסמה. לאחר שהמערכת מוודאת ששם המשתמש פנוי ושומרת את הנתונים באופן מאובטח, המשתמש מועבר חזרה למסך ההתחברות. לאחר התחברות מוצלחת, יוצג סרגל הניווט הראשי (תפריט עליון/צדדי) המאפשר גישה לכלל מודולי המערכת.

- **הטמעת מסר סודי (Embed):**

1. המשתמש מנווט ללשונית "הטמעה (Embed)" בתפריט הראשי.
2. לחיצה על כפתור "העלאת קובץ" לבחירת קובץ ה"כיסוי" (תמונה מסוג PNG/JPG או קובץ שמע מסוג WAV במידה ויועלה פורמט לא נתמך כמו WebP, המערכת תבצע המרה אוטומטית ברקע).
3. בתיבת הטקסט המיועדת, המשתמש יקליד את המסר הסודי שברצונו להחביא.
4. לחיצה על כפתור "הצפן/הטמעה". המערכת תעבד את הקובץ. בסיום, תופיע הודעת הצלחה ויוצג כפתור להורדת הקובץ הנגוע (Stegno-Object) למחשב האישי.

- **חילוץ מסר סודי (Extract):**

1. המשתמש מנווט ללשונית "חילוץ"
2. מעלה את הקובץ הנגוע שקיבל או הוריד בעבר.
3. לחיצה על כפתור "חלץ". המערכת תסרוק את הקובץ, תשאב את הביטים הנסתרים, ותציג את המסר הגלוי בתוך חלונית קופצת (Dialog) למשתמש. אין צורך בהזנת סיסמה או אורך מסר מצד המשתמש – המערכת עושה זאת אוטומטית.

- **ניהול תיבת דואר ודואר יוצא (Inbox & Sent Messages):** במערכת מובנית תשתית הודעות סגורה.

- **שליחת דואר:** המשתמש ילחץ על "הודעה חדשה", יבחר נמען מתוך רשימת משתמשי המערכת, יקליד נושא/תוכן גלוי ויצרף את הקובץ הנגוע. לחיצה על "שלח" תעביר את ההודעה ישירות לתיבת הנמען.

- **קריאת דואר (Inbox):** במסך הדואר הנכנס תוצג רשימת ההודעות. המשתמש יכול ללחוץ על כל הודעה כדי לקרוא את הטקסט הגלוי. אם מצורף קובץ נגוע, המשתמש יכול להוריד אותו או ללחוץ על כפתור "הצג/חלץ מדיה" כדי לפענח אותו ישירות בתוך הדפדפן ללא צורך בהורדה.

- **היסטוריית פעולות (View History):** במסך ההיסטוריה מוצגת רשימה (Grid) של כלל הפעולות שהמשתמש ביצע בעבר (הטמעות וחילוצים). המשתמש יכול להוריד קבצים היסטוריים מחדש בלחיצת כפתור, או למחוק רשומות מהיומן על ידי לחיצה על "מחק" (המבצעת מחיקה לוגית מאובטחת במסד הנתונים).

15. בדיקות והערכה

בפרק זה יוצגו תרחישי הבדיקה המרכזיים שבוצעו במערכת כדי לוודא את תקינותה, יציבותה, ואיכות האלגוריתמים שיושמו בה. לכל תרחיש מוגדרים נתוני הקלט, התוצאה המצופה, התוצאה שהתקבלה בפועל, וציון הערכה מסכם (בסולם 1-10).

מקרה בדיקה 1: מנגנון אימות והרשאות משתמשים (Login)

- **מטרת הבדיקה:** וידוא כניסה מאובטחת ומניעת גישה ממשתמשים לא מורשים.
- **קלט נתונים:** הזנת שם משתמש חוקי (לדוגמה: "agent_01") וסיסמה תקינה במסך LoginView.
- **תוצאה מצופה (Expected Output):** המערכת תאמת את הנתונים מול מסד הנתונים MongoDB, תייצר סשן (Session) מאובטח, ותנתב את המשתמש למסך הבית HomeView. סרגל הניווט MainLayout ישתנה מתצוגת "אורח" לתצוגת "משתמש מחובר".
- **תוצאה בפועל (Actual Result):** האימות עבר בהצלחה, המשתמש הועבר למסך הבית, וכפתורי תיבת הדואר והסטגנוגרפיה נחשפו בסרגל העליון.
- **ציון/הערכה:** 10/10 מנגנון האבטחה והרנדור המותנה פועלים בצורה מושלמת.

מקרה בדיקה 2: הסתרת מסר בתמונה ללא פגיעה חזותית (PVD Algorithm)

- **מטרת הבדיקה:** בדיקת תקינות מנוע הסטגנוגרפיה המרחבי על קבצי PNG.
- **קלט נתונים:** קובץ תמונה מסוג image/png (ברזולוציה 1080x1920), והזנת מסר טקסטואלי: "Secret Project 2026".
- **תוצאה מצופה:** המערכת תנתב את הקובץ ל-PVDStegoService, תטמיע את המסר תוך שינוי מינימלי של הפרשי הפיקסלים (ללא עיוות ויזואלי לעין האנושית), ותפיק קובץ נגוע להורדה. בחילוץ חוזר, המסר יפוענח במדויק.
- **תוצאה בפועל:** התהליך הסתיים בהצלחה בתוך פחות משנייה. התמונה המופקת נראית זהה לחלוטין למקור, והמסר הסודי חולץ ללא שגיאות כתיב או אובדן נתונים.
- **ציון/הערכה:** 10/10 אלגוריתם ה-PVD מיושם באופן אופטימלי ומציג יכולת הסתרה יוצאת דופן.

מקרה בדיקה 3: יציבות מערכת והמרת פורמטים אוטומטית (Graceful Degradation)

- **מטרת הבדיקה:** וידוא שהמערכת אינה קורסת כאשר משתמש מעלה קובץ בפורמט שאינו נתמך טבעית על ידי האלגוריתמים.
- **קלט נתונים:** קובץ תמונה מסוג WebP או GIF, והזנת מסר להטמעה.
- **תוצאה מצופה:** פונקציית הבדיקה (checkValid) ב-StgnoService תזהה שהפורמט בעייתי, תבלום את השליחה הישירה לאלגוריתמים, ותפעיל את שירות ConvertService להמרה שקופה לפורמט PNG טרם ההטמעה.
- **תוצאה בפועל:** התהליכון (Thread) זיהה את הפורמט, המיר אותו אוטומטית ל-PNG ברקע, והשלים את ההטמעה ללא הצגת שגיאת קריסה למשתמש.
- **ציון/הערכה:** 10/10 – טיפול בחריגות וניהול שגיאות עובד בצורה חכמה ויציבה.

מקרה בדיקה 4: חריגה מקיבולת התמונה (Capacity Limit - F5)

- **מטרת הבדיקה:** וידוא מנגנוני הגנה המונעים השחתת קבצים כאשר המסר ארוך מדי.
- **קלט נתונים:** תמונת JPEG קטנה מאוד (לדוגמה 100x100 פיקסלים), ומסר טקסטואלי ארוך מאוד (מעל 10,000 תווים).

- **תוצאה מצופה:** אלגוריתם F5StegoService יחשב את כמות מקדמי ה-DCT שאינם אפס, יזהה שאין מספיק "מקום" להטמיע את כל הביטים, ויעצור את התהליך בצורה מבוקרת עם החזרת הודעת שגיאה ל UI.

- **תוצאה בפועל:** הלולאה האלגוריתמית זיהתה חוסר במקדמים ופסקה את פעולתה. המשתמש קיבל התראת חלופית המודיעה כי "קיבולת התמונה קטנה מדי עבור מסר זה", והמערכת המשיכה לרוץ כרגיל.

- **ציון/הערכה: 10/10** מנגנוני ההגנה והולידציה על הזיכרון פועלים כנדרש.

מקרה בדיקה 5: איכות שמע וחסינות נתונים באודיו (DSSS Algorithm)

- **מטרת הבדיקה:** הערכת הביצועים של פיזור ספקטרום (Spread Spectrum) בקבצי קול בעלי עוצמה נמוכה (שקט/דיבור אנושי).

- **קלט נתונים:** קובץ audio/wav המכיל דיבור שקט ללא מוזיקה, והזנת המסר: "Top Secret Audio".

- **תוצאה מצופה:** מנוע ה-DSSSStegnoService יטמיע את המסר בעזרת מנגנון עוצמה אדפטיבית (Local Alpha), ויאפשר חילוץ מדויק של הטקסט, תוך שאיפה למינימום הפרעות שמע.

- **תוצאה בפועל:** אמינות הנתונים הייתה מושלמת (המסר חולץ בהצלחה מלאה, 100% דיוק). עם זאת, בשל השקט היחסי בקובץ המקור, ניתן היה להבחין ברעש רקע עדין ("רעש לבן") שנוסף לקובץ הפלט כתוצאה מפיזור ה-PN Sequence. מנגנון האלפא האדפטיבית מנע רעש צורם, אך לא העלים אותו לחלוטין.

- **ציון/הערכה: 8/10** ביצועי ההצפנה והחילוץ מעולים (10/10 אמינות נתונים), אך נדרשת פשרה (Trade-off) אקוסטית מסוימת כאשר מדובר בקבצי שמע שקטים. (המנוע מתפקד בצורה מושלמת ושקופה על קבצי מוזיקה קצבית).

16. מסקנות

פיתוח מערכת **Stegno Mail** היווה אתגר הנדסי מרתק ומורכב, שדרש שילוב בין פיתוח Web מודרני (Full Stack) לבין עולם אבטחת המידע ועיבוד האותות ברמת ה Low-Level.

עמידה ביעדים: ניתן לקבוע בבירור כי הפרויקט עמד בכלל מטרותיו ויעדיו. פותחה פלטפורמת Web אוטונומית ועצמאית, המספקת ערוץ תקשורת סגור וחשאי המאפשר למשתמשים להסוות ולהעביר נתונים רגישים בצורה נגישה ומאובטחת. ההחלטה לפתח מערכת "Zero Footprint" (ללא צורך בהתקנת תוכנות מקומיות) הוכיחה את עצמה כפתרון מעשי וריאלי לסביבות עוינות.

האתגרים האלגוריתמיים והשגי הפיתוח: ההישג המרכזי ביותר בפרויקט, לראייתו, הוא המימוש העצמאי של מנועי הסטגנוגרפיה. במקום להסתמך על ספריות "קופסה שחורה" קיימות, המערכת נכתבה כך שתוכל לטפל באופן ישיר בפירוק והרכבה של זרמים בינאריים, המרות קידוד (UTF-8), מניפולציות של פעולות ביט (Bitwise), ומתמטיקה מורכבת של עיבוד תמונה וקול.

- ההתמודדות עם אלגוריתם ה **F5** ב JPEG דרשה התעמקות במבנה מתמטי מסובך של מקדמי תדר (DCT) וכיווץ מטריצות.

- המימוש של אלגוריתם **DSSS** בקבצי שמע חייב הבנה של מושגים מעולם פיזיקת הגלים, כגון "ספקטרום פרוש", הנחתת עוצמה (Attenuation), ואלגוריתמי נירמול רעשים (Zero-Mean Normalization). היכולת לקחת מאמרים אקדמיים ולהפוך אותם לקוד יציב ופועל ב Java-הייתה מספקת ביותר.

הערכת הארכיטקטורה: בחירת הטכנולוגיות התבררה כמוצלחת מאוד. תשתית **Vaadin** אפשרה יצירת ממשק משתמש חלק ודינמי הכתוב כולו ב Java, שמנע פערי שפה בתקשורת עם השרת. מסגרת **Spring Boot** סיפקה מעטפת יציבה לניהול בקשות רשת Threads (אסינכרוניים ששמרו על האתר מהיר למרות האלגוריתמים הכבדים). שימוש במסד הנתונים **MongoDB** היה בחירה מושכלת, שאפשרה טיפול גמיש ואחסון יעיל של אובייקטים בינאריים גדולים (BLOB) ישירות בענן.

סיכום אישי: העבודה על פרויקט הגמר הרחיבה משמעותית את ארגז הכלים המקצועי שלי כמפתח תוכנה. הפרויקט גישר עבורי על הפער שבין פיתוח אפליקציות CRUD רגילות, לבין הנדסת תוכנה עמוקה הדורשת ניהול זיכרון נכון, הבנה של מבנה מחשב (Bits & Bytes), תקשורת HTTP, וראייה מערכתית כוללת.

17. פיתוחים עתידיים

1. שכבת הצפנה מקדימה (Pre-Encryption): הוספת שלב של הצפנה קריפטוגרפית (כגון AES) לטקסט, הממוקם לפני תהליך ההטמעה הסטגנוגרפית. שלב זה ישמש כשכבת אבטחה שנייה ("הגנה לעומק"), המבטיחה שגם אם גורם עוין יצליח לחלץ את המידע מתוך הקובץ, תוכנו יישאר בלתי קריא ללא מפתח מתאים.
2. הרחבה לפורמטי מדיה נוספים: הרחבת יכולות המערכת לתמיכה בסוגי קבצים דיגיטליים נוספים מעבר לתמונה ואודיו סטנדרטיים. פיתוח זה יאפשר לנצל מאפיינים ייחודיים של פורמטים שונים (כגון יתירות מידע בקבצי ארכיון או מסמכים) כדי לגוון את אפשרויות ההסתרה ולהגדיל את הקיבולת.
3. פיתוח אפליקציית מובייל (Mobile Application): בניית יישום לסמארטפונים. פיתוח זה יגיש את המערכת לשימוש מיידי בשטח, ויאפשר למשתמשים להצפין ולפענח קבצים (כגון תמונות שצולמו במכשיר) ישירות מהנייד ללא תלות בגישה לאינטרנט.
4. תוסף דפדפן (Browser Extension): פיתוח רכיב צד-לקוח המתממשק ישירות לדפדפן. תוסף זה יאפשר למשתמשים לנסות ולפענח מסרים מתמונות או קבצי אודיו בזמן אמת במהלך גלישה באתרי אינטרנט שונים, ללא צורך בהורדת הקובץ והעלאתו הידנית למערכת.
5. ניהול הודעות מתקדם (Advanced Messaging): שדרוג תיבת הדואר הפנימית כך שתכלול חיווי סטטוס קריאה.

18. ביבליוגרפיה

כללי

<https://he.wikipedia.org/wiki/%D7%A1%D7%98%D7%92%D7%A0%D7%95%D7%92%D7%A8%D7%A4%D7%99%D7%94>

F5

Westfeld, A. (2001). *F5—A Steganographic Algorithm: High Capacity Despite Better Steganalysis*.

<https://www2.htw-dresden.de/~westfeld/publikationen/f5.pdf>

Computerphile. (2015). *JPEG Files and Colour (JPEG Pt1)* .YouTube.

https://www.youtube.com/watch?v=n_uNPbdenRs

YouTube. *F5 Algorithm Tutorial* .

<https://www.youtube.com/watch?v=LFXN9PiOGtY&t=1s>

YouTube. *Steganography F5 Demonstration* .

<https://www.youtube.com/watch?v=yBX8GFqt6GA>

YouTube. *F5 Method Overview* .

<https://www.youtube.com/watch?v=TWEXCYQKyDc&pp=0gcJCQQLAYcqIYzv>

YouTube. *F5 Steganography Explained*.

<https://www.youtube.com/watch?v=Q2aEzeMDHMA&t=8s>

Wu, D. C., & Tsai, W. H. (2003). *A Steganographic Method for Images by Pixel-Value Differencing*.

<https://www.researchgate.net/profile/Yu-Shuan-Jhang/post/Can-anyone-implement-Pixel-value-differencing-and-RS-steganalysis-in-BMP-image/attachment/5e5e16723843b0499fec19d7/AS%3A864920119808002%401583224434853/download/PVD.pdf>

Kondo, K. (2012). *A Data Hiding Method Using Direct Sequence Spread Spectrum*. arXiv.

<https://arxiv.org/pdf/1205.6775>